

Proxmox Storage DRS

Balancing disk I/O across shared storages on Proxmox VE 9.2

Bernd Zeimetz bernd@bzed.de

2026-09-25

Rendered from IMPLEMENTATION_PLAN.md, SHA-256 9bd1c14b7162f641cdeabc26455052b9769b6867aca61be18fb5194aa334e3b9
(sha256sum --check docs/IMPLEMENTATION_PLAN.pdf.sha256 in the repository must succeed).

Copyright © 2026 Bernd Zeimetz. Licensed under the GNU Affero General Public License, version 3 or later.

Contents

1. Scope	4
Goal	4
Non-goals	4
Relationship to the PVE 9.2 Dynamic Load Balancer	4
Requirement interpretation: “minimal number of migrations”	4
Requirement interpretation: “spread data evenly”	5
Operating assumptions	5
2. Architecture	5
Module layout	6
2.1 Implementation, deployment and operations	6
2.2 Packaging and continuous integration	8
2.3 Logging	8
The two audiences, and the one rule that separates them	9
Levels	9
Format	9
Where the handler is attached	10
An error is one fact, logged once	10
The event catalogue	10
What this does <i>not</i> add	12
Verification	12
The timer	12
2.4 Monitoring status file	12
3. Data acquisition	13
3.1 Where the per-disk data comes from	13
3.2 Two paths deliberately not taken	14
3.3 Metric name mapping and verification	15
3.4 PromQL	15
3.5 PVE API	17
3.6 Which disks can actually be moved online	20
3.7 Disks with snapshots are excluded, loudly	23
3.8 Disks with an unapplied pending change are excluded	24
4. The load model	24
5. The optimization problem	26
5.1 Sets and data	26
5.1.1 Computing U^{set}	27
5.2 Variables	27
5.3 Constraints	27
5.3.1 Configuring the free-space requirement: <code>free_space</code>	29
5.4 Objective	33
5.5 Solver backends	34
6. Gating — deciding whether to act at all	35
7. Migration cost and the payback rule	36
7.1 Cost	36
7.2 Benefit	38
7.3 Acceptance	39
8. Migration ordering	42
8.1 The transient invariant	43
8.2 Scheduling algorithm	44

8.3 Deadlock and staging	45
9. Execution	45
9.1 Modes	45
9.2 Performing one move	45
9.3 Locks, and why a completed move is not a finished move	48
9.4 Failure handling	49
9.5 Output	50
10. Forecasting	51
10.1 Each forecaster owns its data range	51
10.2 Implementation warnings	52
11. Configuration	53
11.1 Validation rules	54
11.2 state.json	55
11.3 Global command-line options	56
11.4 Storage name patterns in groups[] .storages[]	57
12. Implementation phases	58
13. Failure modes and safety	62
14. Worked example	65
14.1 Input	65
14.2 Initial state	65
14.3 Solution	66
14.4 Ordering	67
14.5 Payback	67
14.6 Companion fixture: when the reserve and the balance objective disagree	67
14.7 Companion fixture: affinity repair under the payback rule	68
14.8 Companion fixture: the free-space repair mandate	69
15. Requirements traceability	71
15.1 Config knob → formula	72
16. Diagnostic bundles: collect-testdata and --replay	73
16.1 The bundle	73
16.2 Capture: the superset, not the configured path	74
16.3 Anonymization	76
The governing rule: allowlist, never denylist	76
The pseudonym function, and the salt	76
Per-kind mapping	77
The configuration in the bundle	78
What is deliberately preserved	79
Threat model, stated honestly	79
16.4 The command	79
16.5 Replay: the --replay global option	80
16.6 The corpus: tests/corpus/	81
What the suite asserts	81
The variant matrix	82
Size, and bundles too big to commit	82
16.7 Modules, phases and the honest status	82

A specification for a VMware Storage DRS replacement targeting **Proxmox VE 9.2**. It is written to be implementable without further research: every external fact it relies on is stated inline, and section 14 is a worked numeric example that doubles as a test fixture.

1. Scope

Goal

Given configurable **groups** of shared storages, continuously equalize **disk I/O load** across the storages within each group by live-migrating individual VM disks, subject to:

- a disk may only move between storages in **its own group**;
- every storage must always retain free space for snapshots — by default **2× its largest disk** — and this must hold *during* migrations, not merely before and after;
- every storage must retain its **configured free space** — an absolute byte count or a percentage of its capacity, per storage or per LUN family (§5.3.1) — and when a new volume has eaten it, the engine migrates disks away until it is free again: the requirement is repaired, never traded (§5.3), and the larger of it and the snapshot reserve always wins;
- the number of migrations must be **minimal**;
- a VM's disks should stay **together** on one storage unless space or I/O forces otherwise — a preference weighted by the VM's own I/O (§5.4) and backed by the cost model: reuniting a VM counts as benefit in the payback test, and disks too small to cost anything move for free (§7.1–§7.3);
- data is spread evenly across a group's storages as a **second priority to I/O** — no single storage should hold a disproportionate share of the group's bytes, so the failure of any one storage costs a bounded share of the data;
- a migration's own I/O cost must not exceed the benefit it delivers, judged over a horizon that reflects how long the placement will actually last — a move's cost is paid in days of degraded I/O, its benefit accrues for as long as the VM keeps running there (default horizon: one year, configurable).

Non-goals

Compute/CPU/memory DRS, VM-to-node placement, HA, backup scheduling, storage provisioning, thin-pool overcommit management, and any modification of guest configuration beyond disk location.

Relationship to the PVE 9.2 Dynamic Load Balancer

PVE 9.2 ships a built-in Dynamic Load Balancer — a dynamic mode for the Cluster Resource Scheduler that continuously live-migrates **HA-managed guests between nodes** to even out node CPU/memory utilization. It is **complementary, not overlapping**: it balances guests across *hypervisors* and has no notion of storage or of the FC LUNs behind it. Nothing in PVE balances disk I/O across storages, which is the gap this tool fills.

There is, however, a real interaction: the built-in balancer may live-migrate a VM to another node *while a Storage DRS plan is executing*, invalidating the {node} in a queued `move_disk` call. The pre-move re-validation in §9.2 exists precisely to catch this — it re-reads the VM's current node before every move and re-plans on mismatch. Implementers must not cache the node across moves.

Requirement interpretation: “minimal number of migrations”

The requirement that migrations be minimal is implemented as a **tunable preference** (the β term in §5.4), not as a strict lexicographic minimum. A strict minimum would refuse a second cheap move that halves the remaining imbalance, which is not what is wanted. β sets the exchange rate between “one more migration” and “this much less imbalance”; §14.3 demonstrates β selecting a two-move plan over a three-move plan on the same input (shown there at $\delta = 0$; at the default δ the capacity term reaches the same two-move plan for its own reason).

Requirement interpretation: “spread data evenly”

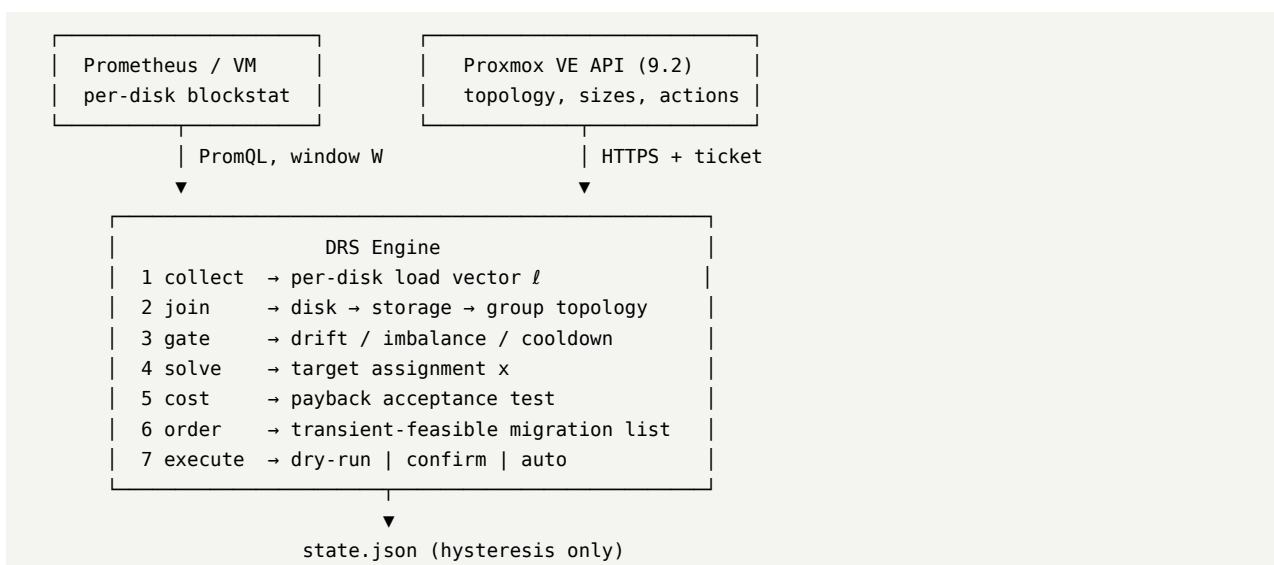
Even spread of **data** across a group’s storages is a risk-reduction requirement — the failure of one storage costs the group whatever was on it — and it is subordinate to the I/O requirement: the load model of §4, in-flight I/O time by default with the ops/bytes terms available where the operator wants them, remains the primary quantity. Like the migration count, even spread is implemented as a **tunable preference** (the δ term in §5.4) rather than a strict “I/O first, bytes second” lexicographic order: two assignments’ load objectives are essentially never exactly equal, so a strict order would reduce the capacity term to a tie-breaker that never acts. The default weight keeps I/O the first priority in every comparison on an imbalanced group, gives δ the deciding vote among plans the load objective is nearly indifferent between, and lets it act on its own — behind §6’s capacity gate — on a group whose I/O is already balanced but whose data is concentrated on few storages.

Operating assumptions

- PVE 9.2, shared LVM (typically over FC) or any shared storage supporting `move_disk`.
- Only **running** VMs are considered by default; a stopped VM generates no I/O to balance, and its disks are moved only if a capacity constraint requires it.
- Disks are thick-provisioned by default, so migration cost is proportional to *provisioned size*.
- **A VM is never stopped, suspended or reconfigured.** The only write the engine issues is `move_disk`. This is what rules out the offline path for snapshotted disks (§3.7), which are movable only with the guest down — a maintenance-window decision for a human.
- Every bus is in scope and every one of them moves online: `ide`, `sata`, `scsi`, `virtio`, plus `efidisk0`, `tpmstate0` and unused volumes (§3.6). A VM’s disks are not all `scsi*`, and anything not enumerated is capacity the model cannot see (§3.5).
- Storage-side cleanup is part of a migration, not an afterthought: with LVM `saferemove` the source volume is zeroed at a throttled rate after the mirror completes, and neither its space nor the VM is available until that finishes (§7.1, §9.3).

2. Architecture

The engine is **stateless with respect to metrics** — all history lives in Prometheus. It keeps one small local state file purely for hysteresis.



Two data sources are genuinely required, and neither substitutes for the other:

- **Prometheus** knows the load of `vmid=101`, `device=scsi0` but has **no idea which storage that disk**

lives on.

- **The PVE API** knows the topology, sizes and capacities, and is the only way to act.

The join between them is the disk identity (`vmid`, `device`), which both sides expose.

Module layout

Module	Responsibility
<code>config.py</code>	Load/validate YAML, defaults, unit parsing (24h, 200MiB), <code>/.../</code> storage patterns (§11.4)
<code>metrics.py</code> <code>pve.py</code>	Prometheus client, PromQL construction, <code>verify-metrics</code> API client: auth, topology, storage status, <code>move_disk</code> , task polling
<code>topology.py</code>	Build the disk/storage/group model, eligibility rules, storage-pattern expansion (§11.4)
<code>loadmodel.py</code>	Reduce raw series to the scalar load vector ℓ
<code>forecast.py</code>	Pluggable forecaster (quantile, seasonal-naive, Holt-Winters)
<code>optimize.py</code>	MILP formulation (CBC via <code>pulp</code>)
<code>heuristic.py</code>	Dependency-free greedy + local search fallback
<code>payback.py</code>	Migration cost model and acceptance test
<code>schedule.py</code>	Ordering under the transient reserve invariant
<code>execute.py</code>	Three execution modes, task supervision
<code>anonymize.py</code>	Keyed pseudonyms and field allowlists for diagnostic bundles (§16.3). Pure
<code>collect.py</code>	<code>collect-testdata</code> : capture, anonymize and write a diagnostic bundle (§16)
<code>replay.py</code>	Read a bundle back and serve it through the <code>metrics.py/pve.py</code> interfaces (§16.5)
<code>cli.py</code>	<code>plan</code> , <code>apply</code> , <code>verify-metrics</code> , <code>verify-storages</code> , <code>show-load</code> , <code>explain</code> , <code>collect-testdata</code>

The installed executable is **pve-storage-drs** — one `[project.scripts]` entry point onto `cli.py`. Every command in this document is written as `pve-storage-drs <subcommand>`; the manpage is `pve-storage-drs (1)`, and the Debian source and binary package are `pve-storage-drs` too. The Python package keeps its distribution name `proxmox-storage-drs` and its import name `proxmox_storage_drs`. The word **storage** is load-bearing: PVE 9.2’s own Dynamic Load Balancer moves guests between nodes (§1), and a shorter name would suggest this tool replaces it.

2.1 Implementation, deployment and operations

Language: Python 3.11+. The decision is driven by the solver and forecasting libraries — `pulp` driving CBC, and `statsmodels`, have no usable equivalent in Go or Rust without substantial reimplementations, and PVE hosts already ship Python, so operators can read and patch the tool. Single-binary deployment is not a requirement here; if it ever becomes one, the dependency-free heuristic path (§5.5) is the portable subset worth porting.

Dependencies come from Debian. The deployment target runs Proxmox VE 9.x, which is built on Debian trixie, so a module that trixie packages is a module the operator already trusts, already patches through their normal update path, and already has on a host with no outbound network. Every dependency below is therefore chosen for being in trixie, and the tool is packaged as a `.deb` (§2.2). A module that is *not* in Debian is only acceptable if it is pure Python and small enough to vendor into our own source package; anything else must be optional, with a code path that works without it.

Package	Purpose	In Debian trixie
requests	Prometheus HTTP, and the transport proxmoxer's https backend uses	python3-requests 2.32
proxmoxer	PVE API client (§3.5)	python3-proxmoxer 2.2
ruamel.yaml	Config (round-trips comments)	python3-ruamel.yaml 0.18
jsonschema	Config validation (§11.1)	python3-jsonschema 4.19
pulp	MILP via CBC — the default, packaged solver path	python3-pulp 2.7 + coinor-cbc 2.10 (both Depends)
statsmodels	Holt-Winters, optional	python3-statsmodels 0.14
pytest, pytest-cov, pytest-xdist	Tests; groups are independent so they parallelize	python3-pytest*

There is deliberately no ortools row. A CP-SAT backend built on it was specified here once, and removed (REVIEW.md AL-02): Debian does not carry ortools, it is not a candidate for vendoring (a large C++ extension, not a pure-Python module), and it would not be installed by hand on PVE hosts — so on every deployment `solver.backend: auto` resolved to CBC regardless, and a second model builder existed only for a backend production could never run. The MILP is solved by **CBC through python3-pulp**, with the dependency-free heuristic below that.

Make `statsmodels` an **optional extra** in `pyproject.toml`; `pulp` stays one too, for a plain `pip install` outside Debian, but `debian/control` treats it differently: `pulp` and `coinor-cbc` are `Depends`, not `Recommends`, so `apt install pve-storage-drs` always gets a real MILP solver by default — CBC-through-pulp is the primary solver on the deployment target, not a bonus for whoever remembers to add it. This is a packaging default, not a claim that the *code* needs a solver to run: the heuristic of §5.5 remains a genuine, exercised fallback — used whenever `pulp/CBC` genuinely are not importable or executable (a non-Debian install with no solver extra, or a broken one), whenever a solve fails or times out, or whenever `solver.backend: heuristic` is configured explicitly — and the code path that reaches it is still tested (`test_heuristic.py`, and `test_optimize.py`'s own “solver unavailable” branches), just no longer by installing the Debian package without its `Depends`, since that configuration no longer exists. The `autopkgtest` in §2.2 still imports every module of the installed package with only the binary package's `Depends` present — now including `pulp/coinor-cbc` — so it verifies the tool's real, default solver path rather than proving heuristic-only operation; an optional dependency (`statsmodels`) imported at module level still fails it.

Licence and contribution rules. The project is **AGPL-3.0-or-later**, copyright Bernd Zeimetz bernd@bzed.de; every source file carries the two-line SPDX header. `AGENTS.md` and `.agents/` hold the working agreement that any implementer — human or model — is expected to follow: `black` + `isort` + `flake8` + `mypy` with configurations written so the formatter and the linter cannot disagree, an enforced 85% coverage floor, feature branches merged only when `make check` is green, and the list of domain invariants (dry-run default, reserve never traded, no auto-delete) that a refactor must not quietly remove. A dependency whose licence is incompatible with AGPL-3.0-or-later cannot be added.

Deployment. Runs on a management host — not necessarily a PVE node — needing outbound access to the PVE API (tcp/8006) and to Prometheus. It holds credentials and must be treated accordingly; prefer an API token over username/password for unattended operation.

Cadence. Invoked by a `systemd` timer (or `cron`) every 15–30 min. Note the separation of concerns: the *invocation* cadence is independent of `execution.time_windows`, which constrain only when **moves execute**. The engine may plan at any time and simply decline to act outside the window. The drift and imbalance gates (§6) make frequent invocation cheap — most runs exit at a gate having issued only read queries.

Logging. Structured lines to `stderr`, never `stdout`, so that `--json`'s machine-readable report (section 9.5)

can be captured from stdout alone with nothing interleaved. What is logged, at which level, in which format, and the audit trail an auto run owes a human afterwards are all specified in **section 2.3**.

2.2 Packaging and continuous integration

The tool is delivered as a Debian package, `pve-storage-drs`, built from `debian/` in this repository (source format 3.0 (native), since upstream and packaging are the same tree). It installs the executable, the manpage, the example configuration and the generated documentation. Two rules follow from that and are not negotiable:

- **debian/control is the single source of truth for dependencies.** Build-Depends and Depends are updated in the same commit as the code or documentation that needs them. CI installs the build dependencies *from* `debian/control` with `mk-build-deps`, so a stale declaration fails the build rather than working by accident on a developer's machine.
- **The autopkgtest asks the only question the build cannot.** The build chroot has the Build-Depends installed and therefore cannot notice a missing runtime dependency. `debian/tests` installs the built package on a system that has only its Depends, then runs `pve-storage-drs --version`, `pve-storage-drs --help`, and an import of every module in the package.

Two pipelines, deliberately different:

	Runs	Covers
GitHub Actions	<code>debian:trixie</code> containers	lint, types, tests and coverage with Debian's packaged tooling; the document build against trixie's older <code>pandoc</code> ; <code>dpkg-buildpackage</code> , <code>lintian</code> , and <code>install-then-run</code>
GitLab CI (Salsa)	Debian's Salsa CI pipeline	<code>sbuild</code> in an unshare chroot with no network , then <code>lintian</code> , <code>piuparts</code> , <code>reprotest</code> and <code>autopkgtest</code>

The Salsa build having no network is the load-bearing part: it is what proves the package builds from trixie alone. If a Python module ever has to be fetched during a build, the answer is to vendor it into the source package; enabling `--enable-network` for `sbuild` is the fallback, and it is a deliberate, visible change to `debian/.gitlab-ci.yml`, not a default.

2.3 Logging

Status: implemented (phase 11). The event catalogue below is the built one; the three level corrections and the retraction at the end of this section have all landed.

Why this section exists. The first implementation got the plumbing right and the policy wrong. Streams are correct (report on stdout, log on stderr — verified), `--mode` overrides are logged exactly as section 11.3 requires, and the JSON formatter is sound. But a plain `pve-storage-drs explain` on a healthy cluster prints this to stderr before a single line of its own report:

```
{
  "event": "config_loaded",
  "level": "INFO",
  ...
}
{"event": "config_warning", "level": "WARNING", ... "no saturation_load configured ..."}
{"event": "config_warning", "level": "WARNING", ... "no saturation_load configured ..."}
{"event": "storage_pattern_expanded", "level": "INFO", ...}
{"event": "optimize_backend_unavailable", "level": "WARNING", ... "solver.backend=cpsat requested but ortools is not importable"}
```

Five machine-readable lines, on every run, for a run in which nothing whatsoever went wrong. Worse, the requirement that actually matters — “in auto mode this log is the only record a human will see” — is **not implemented at all**: `gates.py`, `loadmodel.py`, `schedule.py` and `payback.py` contain no logging calls

of any kind, and `execute.py` never logs the UPID of a `move_disk` it issued. A timer running `apply --mode auto` today records that it started and nothing about what it did to the cluster. The policy below replaces the “Logging.” paragraph of section 2.1.

The two audiences, and the one rule that separates them

A person at a terminal and a journal read three weeks later want opposite things. The person wants **silence unless something is wrong** — their report is the output, the log is not. The journal wants **every decision that changed the cluster**, in a form `journalctl` can filter and a script can parse. Both are served by the same records at different levels and in different formats; no record exists for only one of them.

The rule that follows, and the one to apply when adding any new log call: **a log record is a fact about this run’s decisions or a problem with this run. A fact about static configuration belongs in a report, not in a log that repeats it every 15 minutes forever.**

Levels

Level	Contents	Seen by default
ERROR	The run failed and is exiting non-zero	yes
WARNING	Something degraded but the run continued: a fallback, a skipped move, a lock waited out, an orphaned volume, corrupt state	yes
INFO	The audit trail — every decision that changed, or was allowed to change, the cluster	only where mandated below, or with <code>-v</code>
DEBUG	Per-query, per-candidate, per-poll detail. Third-party library logs	<code>-vv</code> only

The ladder is `--quiet` (ERROR) < default (WARNING) < `-v` (INFO) < `-vv` (DEBUG), with one mandatory exception:

A run that can change the cluster logs its audit trail whether or not anyone asked for it. When the effective mode is `confirm` or `auto`, `apply` raises its own floor to INFO. This is not a convenience: section 2.1’s “in auto mode this log is the only record a human will see” cannot be satisfied by an option the operator has to remember to pass, and an unattended timer that silently migrated 400 GiB is not acceptable output regardless of how the unit file was written. `--quiet` still wins — an operator may insist on silence — but the manual must state plainly that `--quiet` on an auto timer discards the only record of what was moved and why. `--quiet` is the *only* escape: an explicit `--log-level warning/error` on a `confirm/auto` run is raised back up to the floor rather than silently discarding the audit trail too (as built, REVIEW.md X-06 — this was, for a time, a second, undocumented way to lose it).

Read-only commands (`plan`, `explain`, `show-load`, `verify-metrics`, `verify-storages`, `collect-testdata`) and `apply --mode dry-run` keep the WARNING default: they change nothing, so there is nothing to audit, and their report already says everything.

Format

`--log-format {auto,text,json}`, default auto: **text when stderr is a TTY, json otherwise**. A person gets WARNING: `<message>;journal`, a pipe and a redirect all get the JSON object that is already implemented. This is the whole fix for “JSON in my terminal”, it needs no flag in the common case, and it falls out correctly for a `systemd` unit without the unit having to know anything. Both explicit values override the sniffing, in both directions, for the operator whose terminal wants JSON or whose pipeline wants text.

`--log-level {error,warning,info,debug}` sets the level explicitly and wins over `-v/-quiet`. Automation states a level; it should not have to count vs to get one. The one exception is the mandatory floor above: a level below it on a `confirm/auto` run is raised back up, not honoured.

Where the handler is attached

As built (REVIEW.md X-06): the single handler is attached to the **root** logger, not to `proxmox_storage_drs` — the opposite of what an earlier draft of this section specified, and a deliberate choice, not a drift: `handler-on-package plus propagate = False` was tried first and hides every record from `pytest`'s `caplog` and from anything embedding this package as a library, since neither talks to a handler nested under `proxmox_storage_drs` that never propagates up. `root.handlers = [handler]` sidesteps that. The separation this section actually needs — `-v/-vv` raising *this tool's* verbosity without also turning on `urllib3/proxmoxer/statsmodels` debug output — comes from the *levels* instead of from which logger owns the handler: `proxmox_storage_drs`'s own logger carries the run's level, `root` (which every third-party logger inherits from, having no handler or level of its own) stays clamped to `WARNING` except under `-vv`. A record from either still reaches the one handler by propagating up to `root`, so nothing is emitted twice and nothing is silently dropped. The behavioural requirements below (third-party loggers quiet until `-vv`, no duplicate emission) hold either way; this is the mechanism that delivers them.

Every module keeps `logging.getLogger(__name__)` with one exception: **`cli.py` must use the explicit name `proxmox_storage_drs.cli`**, because `__name__` there is `__main__` whenever the module is executed rather than imported, and a `journalctl` filter keyed on a logger name that changes with how the process was started is not a filter. (Observed in the field: the same event appearing as both `"logger": "__main__"` and `"logger": "proxmox_storage_drs.cli"`.)

An error is one fact, logged once

A failing command currently emits the `command_failed` record *and* a human `pve-storage-drs: <msg>` line — the same failure twice, in two formats, on the same stream. In text format the log record *is* the human line and the separate print is dropped; in json format the human line is dropped and the record carries it. Exit codes are unaffected either way.

The event catalogue

Event names are an interface — `journalctl ... | jq 'select(.event=="move_started")'` is a supported way to use this tool — so they are specified here rather than left to whoever writes the call. **Every log record carries an event** – enforced by a test that parses `src/` with `ast`, so a call site cannot silently add an unnameable member.

Event	Level	When	State
<code>run_started</code>	INFO	Once per invocation: command, effective mode, config path + sha256, version	built (replaced config_loaded)
<code>run_summary</code>	INFO	Once per invocation: groups visited, moves issued/succeeded/failed, bytes moved, wall time, exit code	built
<code>gate_decision</code>	INFO	Per group: act, the computed drift/imbalance fractions and the thresholds they were compared against	built — section 2.1 required it; nothing implemented it
<code>load_digest</code>	INFO	Per group: total load, per-storage <code>u_s</code> , disk count, how many disks were coverage-rejected	built — section 2.1 required it; nothing implemented it
<code>plan_selected</code>	INFO	Per acting group: backend, solver status, move count, the six objective terms, before/after spread	built — section 2.1 required it; nothing implemented it

Event	Level	When	State
payback_verdict	INFO	Per acting group: benefit, cost, ratio, the configured minimum, and the accept/reject outcome	built — section 2.1 required it; nothing implemented it
move_started	INFO	Immediately after move_disk returns, carrying the UPID , disk key, source, target, bytes	built — section 2.1 required it; nothing implemented it
move_finished	INFO	Per move: UPID, outcome (moved/failed/replan_needed/drainning), duration	built
replan	INFO	Section 9.2's re-plan loop fired: which group, which attempt, why	built
deadlock	WARNING	Section 8's scheduler could not order a move set	built
mode_override	INFO / WARNING	--mode differs from the config; WARNING when it escalates (section 11.3)	as built, correct
config_warning	INFO	Configuration advisories (e.g. a storage with no saturation_load)	built: INFO
storage_pattern_expanded	DEBUG	A /regex/ storage id matched a set	built: DEBUG
optimize_backend_unavailable	DEBUG under auto, WARNING when that backend was explicitly configured	An optional solver dependency is not importable	built: DEBUG under auto, WARNING otherwise
solver_fallback	WARNING	A configured backend produced no plan and the heuristic took over	as built, correct
forecast_fallback, forecast_backtest_failed	WARNING	Section 10's forecaster degraded to the quantile	as built, correct
vm_locked, orphaned_volumes, orphan_check_failed	WARNING	Section 9.4's hazards	as built, correct
state_corrupt, state_read_failed	WARNING	Section 11.2's state file is unusable	as built, correct
inflight_found, inflight_reconciled, inflight_check_failed, cluster_task_scan_failed	WARNING (INFO for inflight_reconciled)	Section 13's crash recovery	as built, correct
apply_lock_held	INFO	Another instance holds the lock; exit 0 quietly (section 11.2)	as built, correct
salt_rotated	WARNING	--new-salt discarded an existing mapping (section 16.3)	as built, correct
command_failed	ERROR	The run is exiting non-zero	built: printed once, per the format in use
status_file_write_failed	ERROR	monitoring.status_file could not be written (section 2.4); the run and its exit code are unaffected	built

Three level corrections deserve their reasons stated, since each one is a judgement that could otherwise be quietly reverted:

- **optimize_backend_unavailable under auto is not a warning.** solver.backend: auto *means* “use

the best solver installed here”; probing for an optional solver and not finding it (when this was written, CP-SAT; today CBC on a plain `pip install` without the solver extra) is that option working, not degrading. It then logged at WARNING once per group per run, forever, on every cluster that did not install the optional dependency — and its message said `solver.backend=cpsat` requested, which was false: auto requested it, not the operator. `cli._solve_group()` already suppresses its own fallback warning under auto; the inner probe must learn the same distinction, and must say not available rather than requested but ... when nobody requested it.

- **config_warning is an advisory about a file, not an event in a run.** “Storage X has no saturation_load” is equally true on every run until someone edits the config; repeating it at WARNING every 15 minutes trains operators to ignore warnings. It drops to INFO (so the audit trail still records what configuration was in force) and `verify-storages` — the command whose entire job is auditing storage configuration — reports it where an operator will act on it.
- **storage_pattern_expanded is a detail of topology building**, emitted once per group per topology build (twice per `collect-testdata` run). Which storages a pattern matched is already in every report that lists storages. DEBUG.

What this does *not* add

Section 2.1 previously promised “plus an optional file sink”. No such sink, and no logging section in the configuration schema, was ever implemented. **The promise is retracted rather than built:** `StandardOutput=/StandardError=` in a unit file, a shell redirect, or `systemd-cat` already put this stream anywhere an operator wants it, and a second, in-process copy of that mechanism would be one more thing to test, rotate and get wrong. An operator who wants a file gets it from the shell. *(Flagged explicitly because it is a removal from the specification, not an omission.)*

Verification

The tests that existed before phase 11 covered the formatter’s mechanics and nothing about policy, which is why the policy drifted. These now exist, in `tests/unit/test_logging_setup.py`, `test_cli.py`, `test_execute.py` and `test_optimize.py`:

- A clean read-only run against a `--replay` bundle (section 16) emits **nothing on stderr** at the default level, and its report on stdout is byte-identical to the same run with `--quiet`.
- `apply --mode auto` emits `gate_decision`, `plan_selected`, `payback_verdict`, `move_started` (with a UPID), `move_finished` and `run_summary` **without** `-v`, and `apply --mode dry-run` does not.
- `--quiet` suppresses the mandatory INFO floor; `--log-level` overrides both `-v` and `--quiet` for the ladder itself, but a level below the floor on a `confirm/auto` run is raised back up to it rather than suppressing the audit trail (X-06 of REVIEW.md section 29).
- `--log-format text` on a non-TTY emits no JSON; `--log-format json` on a TTY emits only JSON.
- Third-party loggers are not raised by `-v` (only by `-vv`).
- `--json` on stdout parses as a single JSON document with every log level and format combination — the stream separation property, asserted rather than assumed.
- Every `logger.*()` call in `src/` passes an event in `extra=` (a repository-wide `grep` test, in the spirit of `tests/unit/test_documentation.py`), so the catalogue above cannot silently gain an unnamed member.

The timer

Section 2.1 assumes a `systemd` timer and `--quiet`’s help text names one, but `debian/` ships no unit. When it lands it must set `StandardError=journal`, rely on auto mode’s mandatory INFO floor rather than passing `-v`, and must **not** pass `--quiet` — for the reason given above.

2.4 Monitoring status file

A timer-driven tool that fails quietly is worse than one that fails loudly, and the journal is not a monitoring system. `monitoring.status_file` (default `null`: write nothing) names a file that `apply` rewrites at the

end of **every** run that produced a result, in the format of the `check_statusfile` plugin (`monitoring-plugins-contrib, /usr/lib/nagios/plugins/check_statusfile`) — so Nagios, Icinga, or anything that runs a Nagios plugin, can watch it with no code of ours on the monitoring side.

The format is the plugin’s, read from its source rather than assumed:

- **Line 1** is exactly OK, WARNING, CRITICAL or UNKNOWN (case-sensitive; anything else is reported UNKNOWN).
- **Every later line** is the service output, printed verbatim; a file with nothing after line 1 is reported UNKNOWN (“Found no output”). Nagios takes `perfdata` from the first output line, so the summary (file line 2) ends with `| label=value ...`; the lines after it are detail.
- **Freshness is the file’s modification time**, not its content: older than the plugin’s `--age` (default 26 h) is reported WARNING. Hence *every* run rewrites it — including runs that moved nothing and dry runs — so that a stale file means “the timer stopped” and never “it was a quiet day”. A run that could not read its configuration writes nothing and the file goes stale, which is the right signal too.

Levels. CRITICAL when the run exits non-zero: a failed move, or a PVE API or metrics error the run could not plan around (§9.2, “Errors are not mismatches”), or an unexpected crash (the handler re-raises, but only after the file says CRITICAL — a bug must not leave the previous run’s OK in place). WARNING when the run exits 0 but left something for a human: it gave up after `execution.max_replans_per_run` re-plans (external churn — try again later); a failed move’s orphaned volume was reported (never deleted, §9.4); a source storage had not released a volume after a move (draining, §9.3); or a group still breaches its snapshot reserve or free-space requirement once this run’s plan has run (the tool never trades the reserve for balance, so it will not fix that by itself). Otherwise OK. UNKNOWN is never written by the tool. The summary line carries the first problem; detail lines list the rest, each on one line (newlines in messages are flattened, or the summary could be pushed off line 2), each line cut at 300 characters (monitoring systems truncate plugin output, and a PVE or Prometheus error can carry a whole request URL — the full text is in the run log), and capped at 20 lines with an “and N more” line.

Mechanics. Written to a temporary file in the same directory and renamed into place, so the plugin never reads half a report; mode 0644, because the monitoring user is not the user `apply` runs as; the parent directory is created if missing. The content is counts, group and storage names and volume ids — no credentials. Only `apply` writes it (`plan` and friends are interactive and would overwrite the timer’s last result), not `--replay`, and not a run that exited quietly because another instance held the lock (it produced no result; the file keeps the last real one). A failure to write it is logged (`status_file_write_failed, ERROR`) and changes neither the run nor its exit code — and a file that stops updating turns WARNING by age on its own.

Verified against the real plugin, not a paraphrase of it: `tests/unit/test_statusfile.py` and the CLI tests run `check_statusfile` over what the tool writes whenever the plugin is installed (OK → exit 0, WARNING → 1, CRITICAL → 2, missing → 3, stale → 1) and over the two shapes the tool must never write (wrong-case level word, nothing after line 1).

3. Data acquisition

3.1 Where the per-disk data comes from

This is the single most important implementation fact, so it is stated precisely.

`pvestatd` calls `PVE::QemuServer::vmstatus(undef, 1)` — note the `full=1` flag. With `full` set, `vmstatus` issues QMP `query-blockstats` and stores the result as:

```
$res->{$vmid}->{blockstat}->{$drive_id} = $blockstat->{stats};
```

where `$drive_id` is the QMP device name with the `drive-` prefix stripped — i.e. `scsi0`, `virtio0`, `ide2`, matching the VM config keys exactly. The whole QMP stats hash is kept without filtering, which includes:

```
rd_bytes  wr_bytes  rd_operations  wr_operations
rd_total_time_ns  wr_total_time_ns
flush_operations  flush_total_time_ns
rd_merged  wr_merged  failed_rd_operations  failed_wr_operations
invalid_rd_operations  invalid_wr_operations  account_invalid  account_failed  idle_time_ns
```

PVE/Status/InfluxDB.pm then flattens the nested hash so that the **first** nesting level becomes the InfluxDB *measurement* (blockstat), the **second** becomes an *instance tag* (instance=scsi0), and the leaves become *fields*. Combined with the base tags from `update_qemu_status` (object=qemu,vmid=...,nodename=...,host=<vm name>), a series arrives in Prometheus roughly as:

```
blockstat_rd_operations{vmid="101", instance="scsi0", nodename="pve01", host="db-01"}
```

Everything needed for true per-disk IOPS, throughput **and latency** is therefore already present in an existing PVE → InfluxDB → Telegraf → Prometheus pipeline. **No new exporter or collector is required.**

Present at the source is not the same as present at the query endpoint. The InfluxDB protocol InfluxDB.pm emits carries string fields and fixes a field's type at its first write; Prometheus/OpenMetrics has no string sample type. A transport between the two therefore *can* drop one of the six counters, for one series, permanently and silently — Telegraf with a `prometheus_remote_write/prometheus_client` output and no deliberate processor configuration being the common case. §3.3's step 2 exists because of this, and it is the reason every metric name here is configuration rather than a constant.

A backend that ingests **the InfluxDB protocol exactly as PVE exports it** and serves PromQL itself removes the bridge entirely — no Telegraf, no output plugin, nothing in the path that can drop a field on type grounds. **gigapipe on ClickHouse** is that backend here: it is what this project is dogfooded against and what both tests/corpus/ bundles were captured from. `gigapipe` also ingests OpenTelemetry, but that path is untested by this project and is not a route around §3.2 — PVE's OTel metric server bakes the drive id into the metric name regardless of what consumes it. `docs/manual/05-metrics-pipeline.md` is the operator-facing version of all of this.

Verification status of the above. The `vmstatus(undef, 1)` call, the `blockstat->{$drive_id}` assignment, the `s/drive-//r` prefix stripping and the InfluxDB.pm nesting behaviour were read from the `pve-manager` and `qemu-server` sources, not inferred from documentation. Independently, the operator of the target cluster confirmed empirically that their existing Prometheus already carries `rd_operations`, `wr_operations`, `rd_bytes`, `wr_bytes`, `rd_total_time_ns` and `wr_total_time_ns` per disk. The *existence* of the data is therefore settled.

What remains genuinely unconfirmed is the **Telegraf-side naming** — the measurement/field join character, and whether the instance tag survived the collision described in §3.3. That varies per deployment and is exactly what `pve-storage-drs verify-metrics` exists to pin down. Treat §3.4's metric names as defaults to be confirmed, not as constants.

3.2 Two paths deliberately not taken

Do not use RRD. `GET /nodes/{node}/qemu/{vmid}/rrddata` exposes only VM-aggregate `diskread/diskwrite` in bytes/s — no per-disk breakdown and no operation counts. `GET /nodes/{node}/storage/{storage}/rrddata` exposes only used and total, with no I/O metrics at all. Prometheus supersedes RRD entirely here.

Do not switch to the OpenTelemetry metric server (new in PVE 9.1), despite it being the more modern-looking option. Its `_convert_node_metrics_recursive` handles a nested hash by appending the key to the metric **name**:

```
if (ref($value) eq 'HASH') {
  push @metrics, $class->_convert_node_metrics_recursive(
    $value, $ctime, "${metric_prefix}_${key}", $attributes, # <-- name, not attribute
  );
}
```

so `blockstat.scsi0.rd_operations` becomes the metric `proxmox_vm_blockstat_scsi0_rd_operations_total`. The device is baked into the metric name rather than carried as a label, which makes PromQL aggregation across devices impossible without `label_replace` gymnastics and produces unbounded metric-name cardinality. The InfluxDB path puts the device in a **label** and is strictly superior for this use case. (The plugin also only supports OTLP/JSON encoding, not protobuf.)

3.3 Metric name mapping and verification

Telegraf naming is deployment-specific, so **every metric and label name is configuration**, not a constant. Implement `pve-storage-drs verify-metrics` as a first-class command that must be run before anything else. It shall:

1. query `/api/v1/label/__name__/values` and confirm each configured metric name exists;
2. run one instant query per metric and print a sample series with all labels. Reusing these same six results at no extra Prometheus cost, cross-check the full (`vmid`, `device`) set each metric reports against the other five's, and warn naming the metric and the disk(s) whenever one metric's set is a strict subset of another's — the one common, real-world way this happens: InfluxDB's line protocol fixes a field's type from its first write, and Telegraf's Prometheus-compatible output silently drops a field the moment it sees a non-numeric value for it (Prometheus/OpenMetrics has no string sample type), independently of the other five fields for the same disk. Step 5 below (checking only `read_ops`) cannot catch this alone if a *different* field is the one silently dropped;
3. confirm the configured `vmid`, `device` and `node` labels are present and non-empty;
4. warn loudly if the device label is literally `instance` — **PVE's instance tag collides with Prometheus's own scrape-target instance label**, and many Telegraf configurations rename or overwrite it. The implementer must confirm the real label name here before proceeding;
5. report per-disk sample coverage over the configured window, so gaps are visible up front;
6. measure the **observed sample spacing** of a live series (the modal delta between consecutive timestamps in a short `query_range`) and compare it against `metrics.pvestatd_push_interval`. That interval is a PVE-side setting the tool cannot read from the API, so it is declared in config; this step is what stops a stale declaration from silently invalidating the `rate_window ≥ 4 × interval` rule of §11.1. Error if the two disagree by more than 20%, and error if `rate_window` is below four times the *observed* spacing regardless of what config claims.

3.4 PromQL

Let `R` be `metrics.rate_window` (default 5m) and `W` be `window.lookback` (default 24h). Three raw quantities per disk, all keyed by (`vmid`, `device`):

Read and write are fetched **separately** so that `read_factor/write_factor` (§4) can be applied engine-side — six queries per group, not three:

```
# in-flight I/O, read and write (divide by 1e9 to get s/s)
sum by (vmid, device) (rate(blockstat_rd_total_time_ns[5m])) / 1e9
sum by (vmid, device) (rate(blockstat_wr_total_time_ns[5m])) / 1e9

# operations per second
```

```
sum by (vmid, device) (rate(blockstat_rd_operations[5m]))
sum by (vmid, device) (rate(blockstat_wr_operations[5m]))

# bytes per second
sum by (vmid, device) (rate(blockstat_rd_bytes[5m]))
sum by (vmid, device) (rate(blockstat_wr_bytes[5m]))
```

Reduce each to one scalar over the decision window with a robust quantile rather than a mean, so a single spike neither triggers nor suppresses a migration:

```
quantile_over_time(0.95, ( <expression above> )[24h:5m])
```

Notes for the implementer:

- `rate()` already handles the counter resets that occur when a VM reboots or migrates between nodes.
- The `sum by` collapses the `nodename/host` labels, which is what we want: a VM that live-migrated between nodes during the window must remain **one** disk, not two.
- Use the **range** (`query_range`) form as well when the forecaster needs a series rather than a scalar; the same expression works with a step.
- Reject a disk whose sample coverage over `W` is below `window.min_coverage` and fall back to its last known load from `state.json`, flagging it in the plan output. Never treat missing data as zero load — that would silently invite migrations *onto* a busy storage.
- **Scope every query to this cluster’s own nodes.** None of the expressions above restrict which series they match beyond the metric name itself, which is fine when one Prometheus serves exactly one PVE cluster but silently wrong the moment it serves more than one (or anything else emitting a same-named metric): `vmid` is only unique *within* a cluster, so an unscoped query would sum a same-numbered `vmid` from somewhere else into this one’s load without any error or warning. Add a matcher inside the vector selector, before `rate()`, in this order:
 1. `metrics.extra_selector`, verbatim, when the operator set one — needed when Telegraf’s tagging doesn’t carry PVE’s own node name verbatim, the operator wants to scope by a cluster-naming tag their own deployment happens to carry, or the restriction needed is something else entirely.
 2. `<metrics.labels.node>=~"pve01|pve02|..."` otherwise, built from the cluster’s own node list (`GET /nodes`, §3.5) every run rather than hand-maintained, so a node added to the cluster is covered with no config edit. This is the default tier.

A run-time symptom of a scoping mismatch (whichever tier produced it) — a resolved selector that matches literally nothing across all six raw queries — is named explicitly, as “the resolved query filter matched no series at all”, rather than being reported as an idle group (`loadmodel.compute_group_load’s no_series_matched`, REVIEW.md W-07).

`pve-storage-drs verify-metrics` applies only tier 1, never tier 2: it is deliberately independent of the PVE API, so it has no node list to build tier 2 from.

An earlier revision of this section had a middle tier here, matching a `cluster-naming` label (`metrics.labels.cluster`, defaulting to the literal `"cluster"`) against this cluster’s own name (`GET /cluster/status`, needing `Sys.Audit` at `/`). It was removed: the premise that this project’s deployments carry such a label “as standard practice” was an operator’s own mistaken assumption, not a real convention or a Prometheus finding — there never was such a tag, and the tier existed

only on that premise. Use `metrics.extra_selector` for a cluster-naming (or any other) label a deployment actually has.

As built (0.1.3): a gigapipe step $\geq$ range workaround. Two live-confirmed deployments of a recently updated gigapipe backend silently return **zero series** for any `query_range/subquery` call on a range-vector function (`rate()` included) whenever the query’s own step is $\geq$ the function’s own range-vector duration — binary-searched to the exact second, independent of window length or absolute step size. This project’s own defaults set `metrics.step == metrics.rate_window` (both 5m) — idiomatic back-to-back tiling, and exactly the failing boundary — so a fresh install with untouched defaults can hit this on an affected backend with no misconfiguration at all. `metrics.py`’s `safe_range_step_seconds(step, rate_window)` computes the largest whole-second step strictly below `rate_window` that divides `step` as evenly as a whole-second value can (150s at the 5m/5m default), and every `query_range/subquery` call this module issues — coverage, the forecaster’s raw range series, the quantile-over-time reduction’s subquery — uses that step instead, unconditionally (not only when the affected backend is detected: the symptom, zero series, is indistinguishable from genuinely absent data). `decimate_to_configured_step()` then recovers the originally configured grid by keeping every Nth point, so a range series’ retained points are byte-identical to what a plain `metrics.step` query would have returned on an unaffected backend — the workaround is a true no-op there, except for `quantile_over_time`’s own inner evaluation, which cannot decimate (an instant query returns one scalar): it evaluates its subquery on the denser, safe-step grid on *every* backend, a small but real shift in the reduced statistic (REVIEW.md Z-04). `docs/internals/30-metrics.md` has the full mechanism; `collect-testdata --estimate` and `support.max_series_points` account for the doubled (or more, at a wider `metrics.step/rate_window` ratio) point count this stores (REVIEW.md Z-05).

3.5 PVE API

pve.py is built on proxmoxer, not a hand-rolled ticket/CSRF client. `proxmoxer` implements the ticket exchange and API-token auth below itself, behind a `ProxmoxAPI` object whose attribute chaining (`proxmox.nodes(node).qemu(vmid).config.get()`) maps directly onto the endpoint table below. The decisive reason is its **backend abstraction**: the same `ProxmoxAPI` interface is available over plain HTTPS (`backend="https"`, the default and the only one this project uses today), or over SSH — either `openssh` (shells out to the system’s own `ssh + pvesh`) or `ssh_paramiko` (an in-process SSH client). A deployment that cannot or will not open `tcp/8006` to the management host can switch to an SSH-based backend as a **configuration change in one factory function** (`pve.build_client`), with no change to `PveClient`’s methods or to anything that calls them — exactly the shape a hand-rolled HTTPS-only client would not have offered. `python3-proxmoxer` is packaged for Debian trixie (§2.1).

Authentication, as `proxmoxer`’s `https` backend performs it (username/password as originally specified, API token preferred and what this project actually configures by default):

```
POST /api2/json/access/ticket      {username, password}
  → data.ticket                    → Cookie: PVEAuthCookie=<ticket>
  → data.CSRFPreventionToken       → header CSRFPreventionToken on every write
```

Tickets are valid ~2h; `proxmoxer` refreshes them itself on its own internal interval when using password auth. API tokens (`Authorization: PVEAPIToken=USER@REALM!TOKENID=SECRET`) need no CSRF header, no refresh, and are preferred for unattended auto mode; `proxmox.auth.token_id`’s `user@realm!tokenname` format is split into `proxmoxer`’s separate `user/token_name` arguments at the one place `pve.py` constructs the client.

Read path:

Endpoint	Used for
GET /cluster/resources?type=vm	VM inventory: vmid, node, status, name, tags

Endpoint	Used for
GET /cluster/resources?type=storage	Storage inventory, shared flag, used/total per node
GET /storage	Storage definitions: type, content, shared, nodes restriction, and per-storage saferemove / saferemove_throughput — see §7.1 and §9.3
GET /nodes	Every node in the cluster, by name — §3.4’s PromQL node-scoping filter, independent of which nodes currently host a VM or shared storage
GET /nodes/{node}/qemu/{vmid}/config	disk → storage mapping and size
GET /nodes/{node}/storage/{storage}/status	authoritative total/used/avail
GET /nodes/{node}/storage/{storage}/content	per-volume real allocated sizes, owner vmid
GET /nodes/{node}/qemu/{vmid}/snapshot	detect existing snapshot/volume chains (§3.7)
GET /nodes/{node}/qemu/{vmid}/pending	detect an unapplied pending config change per disk (§3.8)
GET /nodes/{node}/qemu/{vmid}/status/current	lock state immediately before a move (§9.3)

Use GET /storage (the list form), never GET /storage/{storage}, for a storage’s own config including saferemove. Verified empirically against a live PVE 9.2.11 cluster: an API token granted only Datastore.Audit can list every storage’s full config via GET /storage, but the same token gets 403 Forbidden (Datastore.Allocate) calling GET /storage/{storage} for the exact same storage — the single-item form apparently backs an edit-UI code path gated by the ability to change the config, not merely read it. The list form returns the identical per-storage object for every storage in one call, so nothing is lost by preferring it; it is also one call instead of |storages| calls.

GET /nodes/{node}/storage/{storage}/content itself needs Datastore.Allocate, not merely Datastore.Audit, contrary to what an “Audit-only, read-planning tool” design would hope. Also verified empirically on the same live cluster: with only Datastore.Audit granted, the call succeeds (HTTP 200) but returns an empty list on every storage, node and content-type filter tried, with no error at any level — a false negative that looks exactly like “this storage has no content” unless you already know to be suspicious of it. Granting Datastore.Allocate (bundled with Datastore.Audit in a custom role, scoped per-storage — see below) on the same storages made /content immediately start returning real data, retested and confirmed. So the tool’s true minimum privilege per storage in scope is Datastore.Audit **and** Datastore.Allocate, not Audit alone; move_disk needs nothing beyond that on the storage side (the VM side needs VM.Config.Disk and VM.Migrate or equivalent, out of scope for this note). Document this precisely in the operator manual rather than repeating the more comfortable but wrong “Audit is enough” claim — an operator who grants only Audit will see the tool silently treat every storage as empty of tracked volumes, which corrupts the `Usext` accounting of §5.1.1 without ever raising an error.

Two more things this exposed about Proxmox’s ACL model, worth knowing before configuring a token: first, API tokens have their own ACL entries, separate from their owning user — with privilege separation enabled (the default), a token’s effective permission is the *intersection* of the user’s permissions and whatever is granted to the token specifically, so granting a role to the user alone has no effect on the token, and granting it to the token alone has no effect either unless the user has it too. Both need the grant. Second, ACL entries at /storage (the parent path, with propagate: 1) were not sufficient in practice to make Datastore.Allocate retroactively work on already-listed sub-paths in one observed sequence during testing — granting the role explicitly at each /storage/{id} resolved it; whether the parent-path grant would have converged given more time was not conclusively isolated, so the operator-facing guidance below grants per-storage explicitly rather than relying on propagation from /storage.

Parsing a disk from the VM config: a key matching

```
^(?:ide[0-3]|sata[0-5]|scsi(?:[0-9]|[12][0-9]|30)|virtio(?:[0-9]|1[0-5])|efidisk0|tpmstate0|unused\d+)$
```

whose value looks like `san-a:vm-101-disk-0,size=512G,iothread=1`. The storage id is the part before

the first `:`; the size comes from the `size=` parameter, cross-checked against `/storage/{storage}/content`, which is authoritative for what is actually allocated. **Every bus counts** — a VM’s boot disk on `ide0` or `sata0` occupies and loads a storage exactly as a `scsi0` does, and a tool that enumerates only `scsi*` will silently mis-account capacity and produce plans that cannot reach balance. §3.6 covers which of these can actually be moved.

Skip only entries whose value contains `media=cdrom`. That covers ISO mounts, empty drives (`none,media=cdrom`) and cloud-init drives (`san-a:vm-101-cloudinit,media=cdrom`). A cloud-init volume does occupy real bytes on the storage, so it is not in `D` but it **is** counted in `Usize` (§5.1.1) like any other volume DRS does not manage.

Disk format. Detect the source format from the volume returned by `/storage/{storage}/content` (`format: raw|qcow2|...`), falling back to the storage type’s default (`raw` for LVM, `qcow2` for directory storages, `raw` for ZFS zvols). (C2) fixes `x_{d,s} = 0` when `s` cannot store that format. `move_disk` without an explicit format preserves the source format, which is what we want: **format conversion is out of scope and must stay disabled by default**. Only pass `format=` when an operator has explicitly opted in — converting `raw`→`qcow2` on shared LVM is what enables volume-chain snapshots, but it is a deliberate storage-policy change, not something a balancer should do silently.

pve-storage-drs verify-storages. A companion to `verify-metrics` (§3.3), run once per storage before relying on any plan. For every storage in every group it reports type, shared, content, `saferemove`, `saferemove_throughput`, total/used, the largest disk currently on it, and the **resolved** free-space requirement `soft_s/hard_s` with the level each came from (§5.3.1); then it derives the implied wipe time `z_max / |saferemove_throughput|` (§7.1 on that sign) and warns when that exceeds `migration.max_single_move_duration` or `gates.cooldown_per_storage` (§9.3). It also prints the expansion of every `/.../` storage pattern (§11.4) — the entry and the storages it matched — and lists cluster storages matched by no group, so an over-broad or dead pattern is visible before any plan relies on it. The resolved requirement is there for the same reason the pattern expansion is: a percentage resolves against *each* LUN’s own capacity, a pattern entry’s `free_space` lands on every storage it matched, and the deprecated `min_free_bytes` folds in as a per-storage floor on top of all of it (§5.3.1) — three ways for one line of config to mean a different number per storage, and this is the one command where that derivation is visible before a plan depends on it. Since `soft_s` now drives §6’s override, §7.3’s exemption and §9.5’s shortfall lines, an operator who cannot predict it cannot predict the balancer. This is the command that turns “why has this balancer been stuck for two days” into a line of output before the first move.

Read-path cost and concurrency. The topology read is `O(number of VMs)`: PVE has no batch config endpoint, so `/qemu/{vmid}/config` must be fetched per VM. For a few hundred VMs, serial fetching dominates run time. Specify:

- a bounded thread pool (8–16 workers, configurable) for the per-VM config fetches, with a short per-request timeout and bounded retries on 5xx;
- a **per-run topology cache** — one snapshot at the start of the run, reused by the load model, solver and scheduler;
- the pre-move re-validation in §9.2 **must bypass this cache** for the specific VM and target storage it is about to touch, and only for those; everything else may be reused;
- `/cluster/resources` is a single call and gives the VM inventory, node placement and coarse storage usage, so fetch it first and use it to decide which per-VM configs are needed at all — in practice this means only a **stopped** VM, when `exclude.running_only` is set, can be skipped without fetching its config, since that is the only exclusion `cluster/resources`’s own fields (`status`) can decide. A VM excluded by `exclude.vmid/tags` still needs its config fetched: (C2) *pins* such a disk into `D` rather than dropping it (§5.1.1’s note on why), which needs its size. A disk turning out to be “ungrouped” (§3.6) is only discoverable *after* fetching the config that reveals which storage it is actually on, so

it costs a fetch too — the saving from this optimization is real but smaller than a naive reading suggests.

Expected call count per run: $4 + 3 \cdot |\text{VMs considered}| + 2 \cdot |\text{storages}| + |\text{extra content pairs}|$ — four cluster-wide calls (VM inventory, storage inventory, storage definitions, and GET /nodes for §3.4’s default node-scoping filter, skipped only when metrics.extra_selector is set), three per considered VM (config, which also carries lock per §9.3’s pseudocode so no separate /status/current call is needed at planning time; /snapshot, per §3.7; and /pending, per §3.8), and two per storage (status, content). |extra content pairs| is the amplification the managed-disk own-VM-node size fetch adds (below, “The actual mechanism...”): one additional content call per distinct (node, storage) pair a managed disk’s VM runs on, beyond the per-storage active-node pick already counted above — deduplicated by _needed_content_node_pairs(), so it is bounded by distinct nodes hosting managed disks, not by VM count, but is otherwise unbounded above (worst case $|\text{nodes}| \cdot |\text{storages}|$) and grows with how spread out VMs are across nodes, not with cluster size alone.

Write path:

```
POST /api2/json/nodes/{node}/qemu/{vmid}/move_disk
  disk=scsi0 storage=san-c delete=1 bwlimit=<KiB/s> [format=qcow2]
  → UPID string
GET /api2/json/nodes/{node}/tasks/{upid}/status → status=running|stopped, exitstatus=0K|...
```

bwlimit is in **KiB/s**. delete=1 removes the source volume after a successful mirror; without it the old volume is left behind as an unreferenced volume and the reserve math will silently drift.

3.6 Which disks can actually be moved online

Enumerating every bus (§3.5) is necessary but not sufficient: the set of disks that *exist* is larger than the set that can be relocated while the VM is running, and the difference is load-bearing for both the capacity model and the affinity objective.

Config key	In D (movable)?	Why
ide0-3, sata0-5, scsi0-30, virtio0-15	yes	ordinary QEMU block devices; move_disk mirrors them online
efidisk0	yes	movable online — verified on PVE 9.2. Tiny (a few MiB), so its migration cost is effectively zero
tpmstate0	yes	movable online — verified on PVE 9.2. Tiny. PVE may not use the drive-mirror path for it, since swtpm rather than QEMU owns the state; see the note below
unused0-N	yes , with $l_d = 0$	a real allocated volume detached from the VM. It occupies bytes and counts against the reserve, but generates no I/O
anything with media=cdrrom	no — not in D, counted in U^{set}	ISO mounts, empty drives, cloud-init volumes

efidisk0 and tpmstate0 move online. This was confirmed empirically on a live PVE 9.2 cluster by the operator. Older Proxmox forum threads (PVE 6.x/7.x era) report online moves of these two failing, and that history is worth knowing only so nobody re-derives an obsolete restriction from a search result: it does not apply to 9.2. They are ordinary members of D.

Two practical notes. First, they are **small** — an EFI var store is a few MiB, TPM state likewise — and below migration.tiny_disk_bytes (§5.4) that smallness is priced in: their β and γ charges are zero and §7’s payback test does not apply to them, so κ may drag a 528 KiB efidisk0 across the group for free. An

earlier revision of this section charged such a move a full β on the grounds that it is a task, with task overhead and a lock window, and called the \$7 cost “negligible” rather than zero; dogfooding overturned that judgement (found live, not by review): a four-move plan whose entire value was reuniting VMs — two of the moves 528 KiB EFI disks — was rejected wholesale by a payback arithmetic in which the affinity those moves bought was structurally worth zero (\$7.2), so the tool refused the one action the operator most wanted. A tiny move occupies its VM’s lock for seconds and transfers a rounding error of the mirror budget; pricing it as a migration priced affinity repair out of the tool. Second, PVE may not use the drive-mirror path for `tpmstate0`, because `swtpm` rather than QEMU holds that state. Do not depend on drive-mirror semantics for it. The transient invariant of §8.1 — the volume occupies **both** storages until the move completes — is the conservative assumption and stays correct under either mechanism, so no part of the model needs to know which one PVE picked. Be clear about which way that conservatism cuts: if `swtpm` does *not* use drive-mirror, the both-storages assumption over-reserves during the move. Over-reserving can never cause a reserve breach, so it is safe; the only cost is that a `tpmstate0` move onto a nearly-full storage may fail the transient check when it would physically have fitted. Treat that exactly like any other infeasible move — **defer it, never force it** — and let `pve-storage-drs explain` say that the transient check was the blocker. Given that TPM state is a few megabytes, a storage tight enough for this to bind is a storage with a much larger problem.

Metrics coverage differs by device type. `efidisk0` is a QEMU drive and should appear in `blockstat`; `tpmstate0` and `unused{N}` are not QEMU block devices, so no series will exist for them and $l_d = 0$ under the `min_coverage` rule of §3.4. That is correct rather than a gap — they generate no guest I/O worth balancing. Have `pve-storage-drs verify-metrics` report which config keys resolved to a series and which did not, and classify these as **expected-absent** rather than as errors, so a genuine coverage problem on a `scsi0` still stands out. Treat the exact per-device-type coverage as a thing to observe on your cluster, not to assume from this table.

Pinned disks are modelled, not ignored. Disks pinned by (C2) — snapshot-blocked (§3.7), config-excluded, or locked this run — enter the MILP as ordinary disks with $x_{\{d, \sigma(d)\}} = 1$ fixed. This is deliberately *not* the same as excluding them: their bytes must still count toward $\sum_d z_d \cdot x_{\{d, s\}}$ and toward Z_s in the reserve constraint (C5), and their load toward L_s . Treating them as foreign volumes instead would work for capacity but would lose the fact that they belong to a VM whose other disks we are placing.

Pinned disks count toward the affinity term by default. κ (§5.4) counts a VM’s spread over storages, and a disk that cannot move this run still *occupies* a storage — the VM is genuinely spread whether or not that particular volume is reachable. So the $y_{\{v, s\}}$ linking of (C3) ranges over all of D ; set `objective.affinity_counts_pinned_disks: false` to range over D^{mov} instead.

An earlier revision defaulted the other way, reasoning that an immovable disk would otherwise leave a VM permanently “fragmented” and let κ “veto good placements to chase a co-location that cannot be achieved this run”. Dogfooding overturned that (found by replaying a real bundle, not by review), and it is worth being precise about why, because the reasoning is seductive and wrong in two separate places.

The co-location usually can be achieved. A pinned disk’s storage is a **constant**, so κ ’s only marginal effect is a preference for putting the VM’s movable disks where it already has one. That is reachable this run — by moving the movable disk, which is the very thing the solver is choosing. Excluding pinned disks does not make an unreachable goal reachable; it hides a reachable one. On the bundle that found this, VM 717219 had two pinned disks together on one storage and a single movable `efidisk0` on another; the plan sent the `efidisk0` to a *third* storage, because with the pinned pair excluded the VM’s counted footprint was one disk and every target scored identically.

Worse, the exclusion can invert the sign. Take a VM with a pinned disk on a.

- **One movable disk, on b.** Under D^{mov} the debt is 0 wherever that disk goes — one counted disk, one storage — so moving it to a to reunite the VM scores exactly the same as leaving it or sending it to a third storage: repair is *unrewarded*. Under all of D the debt is 1 until it joins a, then 0.
- **Two movable disks, both on b.** Under D^{mov} the debt is 0; moving one of them to a to rejoin the pinned disk raises it to 1. The tool *charges* κ for taking a step towards reassembling the VM. Under all of D that step is neutral ($1 \rightarrow 1$), and moving both is a gain ($1 \rightarrow 0$).

The original worry does have a real residue: κ now charges a VM for a split it cannot fully undo, so a VM with pinned disks on two different storages carries a permanent debt floor. That floor is a *constant* — it shifts the objective's absolute value but not its argmin, and it cancels in §7.2's $A_{\text{before}} - A_{\text{after}}$ — so it changes no decision. And where κ does pull a movable disk toward a bad target, it remains what §5.4 calls it: a soft preference that (C5)'s free space or a strong imbalance can legitimately override.

Unused disks move only to repair the reserve, and that is correct. They carry $l_d = 0$ — no series exists for a volume QEMU has not opened — so relocating one yields zero imbalance benefit while incurring the full $\gamma \cdot z_d$ byte penalty and the full payback cost of \$7. The solver will therefore leave them alone until a capacity constraint forces the issue, which is exactly the desired policy. Set `exclude.include_unused_disks: false` to pin them instead; they then count via U^{set} .

An unusedN volume can be absent from GET /storage/{s}/content — not PVE's normal behaviour, but not rare enough to ignore either. Found on a live PVE 9.2.11 cluster on Ceph RBD storage: a VM's `unused0` entry named a volume (VM:vm-104-disk-2) that PVE's own config still tracked, but that volume did not appear anywhere in that storage's content listing, while the same VM's two active (`scsiN`) disks did. The operator confirmed the cause: the underlying volume had been removed directly on the storage backend, outside Proxmox, leaving `unused0` a dangling reference in the VM's config. This is not something PVE catches on its own — an `unusedN` entry is not validated at VM start the way an attached disk is, so the VM boots normally with the stale reference still sitting in its config. `topology.py` treats a content-listing gap the same regardless of cause (§3.5): it falls back to the VM config's own `size=` for that disk and logs a warning naming it as unauthoritative, which is the conservative direction to be wrong in here — believing a since-deleted volume still occupies its former space costs nothing but a temporarily pessimistic reserve calculation, while the reverse (silently dropping it) could let a real volume's bytes go uncounted. An operator seeing this warning should treat it as a prompt to check for, and if confirmed gone, clean up the dangling reference (e.g. `qm unlink <vmid> unusedN`) rather than something the tool should silently paper over.

A content item can also be present but missing its own size — a second, narrower variant of the same gap. Found on a live cluster (a `verify-storages` crash, `KeyError: 'size'`, not a warning): unlike the `unusedN` case above, the `valid` is in `GET /storage/{s}/content`, just without a `size` key on that entry at all. `_resolve_disk_size_and_format()` originally assumed any matched entry had one; it now falls back the same way an absent entry does (VM config `size=`, a warning naming which of the two gaps applies), still trusting the entry's own `format` since that field is independent of `size`. The equivalent sum over *foreign* (unreferenced) volumes in `_build_storages()` had the identical unguarded `item["size"]` and no VM config to fall back to for one — that one skips the volume from the reserve calculation instead (an undercount, the opposite conservative direction from the case above, for lack of any better number) and warns just as loudly.

approximate-size sits between those two, and is preferred over the VM config. A `size-less` content item can still carry `approximate-size` — PVE's own field for storage plugins where an exact size is expensive to determine — and both call sites above check for it before falling further back: still a live number from the storage plugin itself, not a static one recorded at disk-attach time and never revisited, so it is the better of the two imperfect answers. Only when *neither* `size` nor `approximate-size` is present does either function fall all the way back to the VM config (managed disk) or skip the volume (foreign

one).

The actual mechanism, per the operator who hit this live: `approximate-size` is a PVE 9.2+ feature for `qcow2` volumes on shared LVM storage specifically. PVE 9.2 added snapshot support on ordinary (non-thin) LVM by formatting the LV itself as a `qcow2` image rather than using it raw. Reading a `qcow2` image’s own logical size means opening it, which needs its LV *active* — cheap on the node already running the owning VM (PVE keeps that LV active there), expensive or outright impossible on a shared LVM storage’s other nodes, which may have no reason to have activated it at all and can collide with whichever node already has. `approximate-size` is PVE’s way of answering the content listing anyway, from LVM’s own metadata, without activating anything. This means `_pick_active_node()`’s one pick per *storage* (used for every other section 3.5 call) is the wrong node for *this* value specifically: size on a `GET /storage/{s}/content` response is only reliable from the one node guaranteed to have the LV active, which is the node the volume’s own VM is running on, not an arbitrary node that merely reports the storage as available. `_needed_content_node_pairs()/build_topology()` now fetch a managed disk’s content listing from *its own VM’s node* specifically, not `_pick_active_node()`’s pick, precisely so size (not `approximate-size`) is what gets used whenever it can be. The tier above still matters: a foreign (unreferenced) volume has no owning VM to pick a node from, and a managed disk’s own node can itself report `approximate-size` if PVE has not (yet) activated the LV there either.

Evacuating a storage completely is therefore possible online — every disk type in the table above except CD-ROM-media entries can be relocated with the guest running, and CD-ROM entries hold no storage-owned data except cloud-init volumes, which are regenerable. Where a full evacuation is *not* achievable it is because of a §3.7 snapshot or an explicit exclusion, never because of a device type. `pve-storage-drs explain` must name the actual blocker per VM — “106: cannot fully consolidate, 2 snapshots on `scsi0`” — rather than emitting a plan that quietly leaves a stray volume behind.

3.7 Disks with snapshots are excluded, loudly

`move_disk` is issued with `delete=1` throughout (§3.5), and PVE refuses that combination on a volume that has snapshots — “*you can’t move a disk with snapshots and delete the source*”. Even where a move is accepted, PVE does not carry the snapshots across: they are left behind or lost. Under PVE 9 volume-chain snapshots the situation is worse, because a snapshot is a *separate full-size volume* on the source storage, so a “moved” disk would leave most of its bytes behind and the reserve arithmetic would silently drift.

There is no safe automatic remedy. The two real options both belong to the operator: delete the snapshots (a data-retention decision the balancer must not make), or move the disk offline with the VM shut down (out of scope — this tool never stops a VM, §1). So the policy is **skip, and complain**:

1. **Detect per volume, not per VM.** `GET /nodes/{node}/qemu/{vmid}/snapshot` is authoritative for VM-level snapshots and pins *every* disk of that VM. Independently, cross-check `/storage/{s}/content` for volumes owned by the VM that its current config does not reference: under volume-chain storages those are chain members, and elsewhere they are orphans. Either way the disk is unsafe to move. Do not rely on volume-name patterns — they are storage-specific.
2. **Pin, do not drop.** An excluded disk stays in the model with $x_{\{d, \sigma(d)\}} = 1$ so its bytes and load remain accounted for in (C4)/(C5)/(C6); it is not demoted to a foreign volume.
3. **Complain, every run, at WARN.** List each affected VM with its pinned bytes and pinned load, and the group total of both. A one-line “3 VMs skipped” is not enough: the operator needs to know *which* snapshots to clear to unblock balancing.
4. **Say when the goal has become unreachable.** If pinned load exceeds `report.warn_pinned_load_fraction` of a group’s total (default 0.25), the residual imbalance may be structural rather than a planning failure. Report the best achievable spread *given the pins* alongside the actual one, so a stubborn 40% spread is attributable to snapshots rather than looking like a broken solver.

`exclude.skip_vms_with_snapshots` stays as a knob but its false setting does not make such moves work — it merely stops pre-filtering them, and PVE will reject them at the API. Keep it true; the pre-flight check of §9.3 runs regardless.

3.8 Disks with an unapplied pending change are excluded

Confirmed live against a real cluster (`config/drs.yaml`'s dev cluster, VM 102): `GET /nodes/{node}/qemu/{vmid}/config` — what §3.5's `vm_config()` calls, with no current parameter, which is every call this project ever makes — returns a key's **pending** value once one exists, not the value actually in effect. A disk edited through the PVE UI while the guest is running (a resize, an option change) shows up there as if the edit had already taken effect, even though PVE has queued it for the VM's next reboot. `GET /nodes/{node}/qemu/{vmid}/pending` (`vm_pending()`) is the only endpoint that exposes both: each entry carries key, the still-in-effect value, and — only when that key has an unapplied change — a pending field (an edit) or a truthy delete field (queued for removal).

Why this matters for a storage migration, specifically. `move_disk` acts on the *current* volume, not on the pending edit — moving `scsi0` when its pending change is, say, a queued resize to a larger size relocates the disk at its *current* size, then leaves that same pending resize entry sitting in the config afterwards, now describing a change relative to a volume that has since moved storage. PVE does not reconcile or drop a key's pending entry as a side effect of `move_disk` — there is no code path that would, since the two are unrelated operations from PVE's own point of view. The operator's next reboot then applies a resize computed against a value that predates the migration, on whichever storage the disk ended up on. This is not a hypothetical: it is what “the pending disk entry is not being updated when the disk is migrated” (an operator report against the dev cluster) actually is once traced to the API level, and PVE has no fix for it because nothing in its own model treats `move_disk` and a pending edit as interacting at all. The engine's only safe option is to keep the two from ever overlapping in the first place.

So, exactly like §3.7's snapshot handling:

1. **Detect per disk device**, from `vm_pending()`, not from `vm_config()`'s own merged view — the pending value is exactly what `vm_config()` cannot be trusted to distinguish from the real one.
2. **Pin, do not drop**. A disk with an unapplied pending change stays in $\mathcal{D}$ with $x_{\{d, \sigma(d)\}} = 1$ fixed (C2), the same as a snapshot-blocked or locked disk (§3.6's “pinned disks are modelled, not ignored”).
3. **Re-check immediately before issuing the move** (§9.2's step 6, below), not only at planning time: an operator can queue a pending change in the PVE UI in the gap between `plan` and `apply/auto` issuing the move, exactly as a lock (§9.3) or a new snapshot (§3.7) can.
4. **No knob disables this**. Unlike `exclude.skip_vms_with_snapshots`, there is no configuration escape hatch — a pending change is not a policy preference to override, it is PVE leaving a config key referring to state that would become wrong the moment the disk moves.

(C2)'s eligibility list and §9.2's re-check list both name this condition explicitly.

4. The load model

For each disk d , reduce the three raw quantities to a single scalar ℓ_d , expressed in **average in-flight I/O requests**. The three terms have wildly different magnitudes, so they are normalized against each other before weighting and the blend is then rescaled back onto the in-flight-I/O scale; the units matter downstream and are spelled out below.

Read and write are combined **before** normalization, using the configurable asymmetry factors $\rho = \text{load_weights.read_factor}$ and $\omega = \text{load_weights.write_factor}$ (both 1.0 by default). Reads and writes are therefore fetched as **separate series** and combined engine-side in `loadmodel.py`, not summed inside

PromQL — this keeps the factors in tested code rather than in deployed query strings, and makes them unit-testable in isolation:

```
raw_t(d) = p.rd_time_d + w.wr_time_d      (in-flight I/O, s/s)
raw_o(d) = p.rd_ops_d + w.wr_ops_d        (ops/s)
raw_b(d) = p.rd_bytes_d + w.wr_bytes_d    (bytes/s)
```

Each term is then normalized by its group total, because the three have wildly different magnitudes (in-flight I/O ≈ 0 –10, ops/s ≈ 0 –50 000, bytes/s ≈ 0 – 10^9) and unnormalized weights would be meaningless. Normalization makes the three terms **commensurable**; it must not be the last step, because on its own it also destroys the physical meaning of the result. So blend in normalized space, then rescale back onto the in-flight-I/O scale:

$$i_d = \frac{raw_t(d)}{\sum_{e \in g} raw_t(e)}, \quad o_d = \frac{raw_o(d)}{\sum_{e \in g} raw_o(e)}, \quad b_d = \frac{raw_b(d)}{\sum_{e \in g} raw_b(e)}$$

$$\ell_d = T_g \cdot \frac{w_t \cdot i_d + w_o \cdot o_d + w_b \cdot b_d}{w_t + w_o + w_b} \quad \text{with } T_g = \sum_{e \in g} raw_t(e)$$

with $w_t = 1$, $w_o = w_b = 0$ by default — **I/O time is the primary quantity**.

Units of ℓ , and why the rescale is not cosmetic. T_g is the group’s total average in-flight I/O, so $\sum_{d \in g} \ell_d = T_g$ exactly and ℓ_d is measured in **average in-flight I/O requests** — the same physical unit as raw_t . Under the default weights the rescale is an exact identity, $\ell_d = raw_t(d)$. Everything downstream depends on this absolute scale:

- §7 compares a plan’s benefit against a mirror’s cost, where $w = 1.0$ means *one* sequential reader or writer in flight. That comparison is only valid if ℓ is on the same absolute scale. Were ℓ left normalized to sum to 1, $w = 1.0$ would silently be T_g times too large — on a busy group with $T_g \approx 20$ every migration would look twenty times more expensive than it is and the payback test of §7.3 would reject almost everything.
- §7.3’s saturation guard and §5.3’s big-M bound both need an absolute load scale.
- The worked example in §14 uses raw loads ($\Sigma \ell = 7.4$, not 1.0) and is self-consistent only under this definition.

Only *ratios* between disks matter to the balance objective, so the rescale changes no optimal assignment; it changes every quantity that is compared against a physical constant. Guard the divisions: if a group’s total for a term is 0 that term contributes 0 for every disk rather than producing NaN, and if $T_g = 0$ the entire group is idle — skip it, there is nothing to balance.

The objective weights of §5.4 are therefore calibrated in units of in-flight I/O per storage, which is what makes $\alpha = 1.0$ against $\beta = 0.25$ a meaningful default pair: a migration must buy at least a 0.25-request reduction in summed deviation to be worth making.

Why I/O time is the right default. `rate(rd_total_time_ns + wr_total_time_ns) / 1e9` is, by Little’s law, the **average number of I/O requests in flight** for that disk. It is dimensionless, it is additive across disks on the same storage, and it is directly comparable between storages of different size and speed. Crucially it *self-weights*: a 1 MiB write that takes 8 ms contributes eight times what a 4 KiB read taking 1 ms does, without anyone having to guess a read/write weight. Pure IOPS treats those two operations as equal and so systematically under-counts large sequential load; pure throughput does the reverse and under-counts small random load. ops and bytes remain available as additional weighted terms for operators who want to express a policy the array’s own timings do not capture.

Because I/O time already embodies the real cost asymmetry between reads and writes, leaving `read_factor = write_factor = 1.0` is correct for the default configuration. The factors exist for the ops/bytes terms, where the asymmetry is *not* otherwise represented — a write to a RAID-6 array costs far more than a read of the same size, and neither an operation count nor a byte count knows that. Applying them to the I/O-time term as well is supported but double-counts, so do it only deliberately.

One caveat to document: I/O time is a *feedback* signal — it rises when the array is slow, including when the slowness is caused by some other tenant. Combined with the drift gate and cooldowns this is stable in practice, but an operator seeing oscillation should shift weight toward ops/bytes, which measure only what the guest requested.

Storage load and utilization, where c_s is the configured capability weight:

$$L_s = \sum_d \ell_d \cdot x_{\{d,s\}} \quad u_s = L_s / c_s$$

u_s — not L_s — is what gets equalized, so a storage with half the spindles can be given capability weight: 0.5 and will correctly be assigned half the load.

5. The optimization problem

Solved **independently per group** — groups share no variables and no constraints, so a cluster with four groups is four small problems, not one large one.

5.1 Sets and data

Symbol	Meaning
g	a storage group
S	storages in the group
D	movable disks currently in the group
V	VMs owning at least one disk in D
$v(d)$	the VM owning disk d
$\sigma_{\theta}(d)$	the storage disk d is on now
z_d	provisioned size of disk d (bytes)
ℓ_d	load of disk d (section 4)
C_s	total capacity of storage s
$U^{s \text{ ext}}$	bytes on s consumed by volumes DRS does not manage (§5.1.1)
c_s	capability weight of s
f_s	snapshot reserve factor for s (default 2.0)
soft_s	configured free-space requirement for s , in bytes — the plan endpoint (§5.3.1)
hard_s	transient free-space floor for s , in bytes, $\leq \text{soft}_s$ (§5.3.1, §8.1)
N_s	saturation load of s , in in-flight I/O requests; optional, §7.3 only — not part of the MILP

Provisioned size, never allocated size — on every storage type. z_d is what the volume was *provisioned* at, and $\sum_d z_d \cdot x_{\{d,s\}} + U^{s \text{ ext}}$ is what a storage is counted as holding, including on thin-provisioned storage (Ceph RBD, LVM-thin, ZFS), where the pool may report far less allocated. This tool never counts on over-provisioning: a plan that fits only while the disks stay thin is one a growing guest can turn into a full pool, and nothing here can bound that growth. So (C4)/(C5), the free-space requirement (§5.3.1), §8.1’s transient predicate and the cost model all read the same provisioned sums, and used from `GET .../status` is a display figure, never a model input — including at execution time, where §9.2 step 2 re-reads the provisioned figure live from the content listing. The consequence is deliberate: on a thin pool a `free_space.soft` can show a shortfall the pool’s own numbers do not.

5.1.1 Computing U^{ext}

$$U^{\text{ext}} = \sum \{ \text{size}(\text{vol}) : \text{vol} \in \text{GET } / \text{nodes}/\{\text{node}\}/\text{storage}/\{\text{s}\}/\text{content} , \\ \text{identity}(\text{vol}) \notin D \}$$

where $\text{identity}(\text{vol})$ is the (vmid, device) pair the volume belongs to, resolved by matching the volume id against the owning VM’s config, and D here means *every* disk (C2)’s eligibility pass tracks for this group, pinned or not — $\sum_{\{s \in S\}} x_{\{d, s\}} = 1$ holds for $d \in D$ regardless of whether a specific x is fixed. Everything that is **not** a member of D counts as foreign: templates, ISOs and backups, disks of stopped or ungrouped VMs (§3.6/(C2): never fetched, never entered into any D at all), disks belonging to another group that shares the storage, unreferenced orphans, and **orphaned target volumes left by a previously failed move** (§9.3). Counting those orphans is intentional — they really do occupy the LUN, and letting them inflate U^{ext} is what makes their cost visible rather than silently eroding the reserve.

Config-excluded disks (exclude.vmid/s/exclude.disks/tags/no-drs) are *not* on this list. Section 5.3 (C2) pins them into D rather than dropping them, precisely so their bytes still count toward $\sum_d z_d \cdot x_{\{d, s\}}$ in (C4)/(C5) and their fragmentation toward κ — see “Pinned disks are modelled, not ignored” in §3.6. Treating a config-excluded disk as foreign instead would double the inconsistency: it would still occupy the reserve calculation correctly by accident (foreign bytes are also subtracted from capacity) but would silently break the affinity accounting for the rest of that VM’s disks, which is exactly the failure mode §3.6’s rule exists to prevent. An earlier draft of this section listed “excluded” alongside “stopped” and “ungrouped” here, which was a direct contradiction of (C2) rather than a second valid path — fixed in the same commit that first implemented this join in `topology.py`.

Set $U^{\text{ext}} = 0$ only if `snapshot_reserve.count_foreign_volumes` is false, which is not recommended.

5.2 Variables

Variable	Domain	Meaning
$x_{\{d, s\}}$	$\{0, 1\}$	disk d is placed on storage s
$y_{\{v, s\}}$	$\{0, 1\}$	VM v has at least one disk on s
Z_s	≥ 0	size of the largest disk on s
e_s	≥ 0	absolute deviation of u_s from target (L1 objective)
d_s	≥ 0	relative deviation of s ’s fill fraction from the group mean fill $\bar{b}$ (capacity-spread objective, §5.3 (C7))
t	≥ 0	maximum utilization (min-max objective)
r_s	≥ 0	reserve violation slack

5.3 Constraints

(C1) Assignment. Every disk lands on exactly one storage in its group:

$$\sum_{\{s \in S\}} x_{\{d, s\}} = 1 \quad \forall d \in D$$

(C2) Eligibility. Fix $x_{\{d, s\}} = 0$ wherever placement is impossible or forbidden. This is where all the domain rules live, and doing it as variable fixing rather than as constraints keeps the model small:

- s does not have images in its content list;
- s is not shared, or is restricted to nodes that cannot see the VM;
- s cannot hold the disk’s format;
- d or $v(d)$ is excluded by config (exclude.vmid/s, exclude.disks, tags, no-drs) — also pin;
- $\sigma_0(d)$ belongs to **no** configured group — such a disk is unmanaged: it is not in any D , it is pinned

where it is, and its bytes count toward U^{next} of its storage. Report it in show-load as “ungrouped, not managed” so an unintended omission from groups is visible rather than silent;

- d , or any disk of $v(d)$, has a snapshot or an unreferenced companion volume on its storage (§3.7) — $\text{pin}_x\{d, \sigma_0(d)\} = 1$. `move_disk delete=1` cannot move such a volume and would not carry the snapshots if it could;
- d has an unapplied pending config change on PVE (§3.8, `GET .../pending`) — also `pin`. PVE does not reconcile a disk’s pending entry when that disk is moved, so migrating one leaves the entry referring to state that predates the move;
- d is within its per-disk cooldown — also `pin` to current;
- $v(d)$ is currently locked (§9.3) — `pin` for this run. A lock is transient, so this is a *planning-time* `pin` only and carries no cooldown; the next run re-evaluates it.

(C3) VM affinity linking. Couple y to x in both directions so the objective term is exact. Let $D^{\text{mov}} \subseteq D$ be the disks that are not pinned by (C2):

$$\begin{aligned} x_{\{d,s\}} &\leq y_{\{v(d),s\}} & \forall d \in D^{\text{mov}}, s \in S \\ y_{\{v,s\}} &\leq \sum_{\{d \in D^{\text{mov}} : v(d)=v\}} x_{\{d,s\}} & \forall v \in V, s \in S \end{aligned}$$

D^{mov} is the range only when `objective.affinity_counts_pinned_disks: false` is set explicitly. **By default the linking ranges over all of D** (D^{mov} replaced by D in both constraints above), so that a disk which cannot move this run — snapshot-blocked, config-excluded or locked — still anchors its VM: $y_{\{v, \sigma_0(p)\}} = 1$ is fixed for every pinned p , and κ then rewards bringing the VM’s movable disks to that storage instead of being blind to where they go. §3.6 carries the full argument, including why the opposite default silently penalized affinity repair.

(C4) Largest-disk linearization. $Z_s = \max\{z_d : x_{\{d,s\}}=1\}$ is not linear, but because the reserve constraint pushes Z_s down while this pushes it up, a one-sided bound is exact at the optimum:

$$Z_s \geq z_d \cdot x_{\{d,s\}} \quad \forall d \in D, s \in S$$

(C5) Capacity, snapshot reserve and free space. The core safety constraint. The reserve is the **larger** of the snapshot term and the configured free-space floor, so introduce $R_s \geq 0$:

$$\begin{aligned} R_s &\geq f_s \cdot Z_s \\ R_s &\geq \text{soft}_s & (\text{constant, §5.3.1}) \\ \sum_d z_d \cdot x_{\{d,s\}} + U^{\text{next}} + R_s &\leq C_s + r_s & \forall s \in S \end{aligned}$$

Two one-sided bounds are exact for $R_s = \max(f_s \cdot Z_s, \text{soft}_s)$ because (C5) pushes R_s down while both bounds push it up. The $f_s \cdot Z_s$ term is the “always keep 2× the largest disk free” rule, and (C4) is what makes it expressible in a linear model at all. `soft_s` is the storage’s **configured free-space requirement** (§5.3.1): the number of bytes that must be free on s when the plan has fully run, whether the operator asked for it as an absolute byte count or as a percentage of the storage’s capacity. It generalizes the old `min_free_bytes` floor — a storage whose largest disk is small needed one (with $f=2$ and a 10 GiB largest disk, the snapshot term alone would reserve only 20 GiB on a 20 TiB LUN), but so does a storage that must keep headroom for reasons the snapshot rule cannot see: a thin-provisioning safety margin, a quota for volumes this tool does not manage, or the operator’s plain policy that a LUN is not allowed to run full. **If the snapshot reserve is bigger than the configured free space, the snapshot reserve wins** — the `max()` is the whole integration, and neither term can erode the other.

`soft_s` is a **requirement, not a preference**. A storage that ends the plan below its configured free space is in violation, exactly as if it had breached the snapshot reserve, and the engine must migrate disks away until the requirement is met — the Storage-DRS cluster function: when an admin places a new VM on an overfull storage, the balancer takes care of the situation and moves things so the configured free space is

free again. Three properties make that mandate real rather than declared:

1. **The lexicographic stage 1 minimizes it.** Σr_s covers both terms of the $\max()$ — there is one slack per storage, not one per reason — so a free-space shortfall is minimized ahead of balance at the same weight, in the same stage, as a snapshot shortfall. The repair is *winning by construction*: no achievable balance gain can pay for a byte of it (§5.3’s two solve paths).
2. **It bypasses the gates.** §6’s reserve override fires for a free-space violation exactly as for a snapshot violation — safety is not subject to hysteresis, and neither is a mandate.
3. **It is exempt from payback.** §7.3’s aggregate test does not apply to a plan that repairs: the repair is the requirement, and a cost/benefit test would let a large disk’s mirror cost veto the very rule the operator configured. The exemption is **plan-level** — decided on the plan’s outcome, the Σr_s of the scheduled assignment it ends at (§7.3: the R-02 state the ordered moves actually reach, not the solver’s target) against the current assignment’s — plan-level in the same way the built `payback.py`’s `has_reserve_override` short-circuits the aggregate test for the whole plan, but on a deliberately **narrower trigger** than that flag’s “some move’s source was violating at scheduling time”. §7.3 states the difference and proves the new trigger is a strict subset of the built one; do not read “plan-level” as “already built”. Which moves *carried* the repair is reported per move by the **revert test**: a move is marked `repair: true` iff holding that one disk on its current storage would strictly raise the plan’s final Σr_s , evaluated by re-scoring Σr_s on the final assignment with that one $x_{\{d, \sigma(d)\}}$ held (no re-solve — the plan is fixed; the test asks what the *plan’s own* slack would be without the move). The revert test is what covers the indirect repair: a move that empties the destination another repair needs is marked, even though its own source was never in violation (§14.8 works exactly this case).

A group can be **unable** to satisfy `soft_s` — every storage full, or every candidate disk pinned by (C2). That is what `r_s > 0` means, and it is reported exactly as a snapshot shortfall is: loudly, with the byte amount, as an unfixable shortfall (§9.5). The mandate is to *migrate until the configured free space is free*, not to guarantee that it can be — and when it cannot, the tool says so rather than silently planning around the violation.

5.3.1 Configuring the free-space requirement: `free_space`

`soft_s` and `hard_s` come from the `free_space` config block, resolved per storage at run start — after `/.../` pattern expansion (§11.4), before the model is built, so the solver, the scheduler and every report see plain byte constants.

```
free_space:
  soft: 0           # bytes, byte-unit string, or "N%" — the plan-endpoint requirement
  hard: null        # same grammar; global null = soft (no transient dip); per-storage
                   # null = inherit the global value
```

The grammar. A value is either

- an **absolute size**: an integer byte count (1073741824) or a byte-unit string ("1 GiB", "512MiB") parsed by the same unit parser as `migration.bwlimit_bytes_per_sec`; or
- a **percentage**: a string ending in % ("10%"), resolved as $\text{round}(C_s \cdot N/100)$ against *that storage’s own capacity*, with $0 \leq N < 100$ — 100% is rejected by the grammar itself, at config-parse time, rather than accepted here and left to §11.1’s `soft_s < C_s` rule, which is an inventory-time check and a different error surface. A percentage is a per-storage number even when it comes from a global or pattern-level setting: one "10%" applied across a group of a 20 TiB and a 2 TiB LUN demands 2 TiB and 200 GiB respectively, which is the point — “a tenth of the LUN free” is one policy, not two configs.

Where it can be set, most specific wins:

Level	Key	Applies to
per storage	groups[].storages[].free_space.soft / .hard	that storage (a literal entry, or every storage a /.../ pattern matches — §11.4)
global	free_space.soft / free_space.hard	every storage in every group with no per-storage setting

A pattern entry’s `free_space` applies to every storage it matches, exactly as its `capability_weight` does — one entry reserves a whole LUN family, and a literal entry overrides it for the one exception. `null` at the global level means *no free-space requirement from this knob* (the snapshot reserve may still impose one); `null` at the per-storage level means *inherit the global value*, the same inheritance `reserve_factor` already has.

`snapshot_reserve.min_free_bytes` is **deprecated syntax for `free_space.soft`** — and it was never more than a global scalar: the built `config.py` carries it as one number on the `snapshot_reserve` block, with no per-storage form (unlike `reserve_factor`). `config.py` accepts it, warns, and folds it in **per storage, after percent-to-bytes conversion**: `soft_s = max(soft_s_resolved, min_free_bytes)` for every storage, where `soft_s_resolved` is what the inheritance above produced. Per storage, because that is what the key means in the built code — `reserve.py` applies it as a floor on every storage (`required = max(round(f_s * largest), min_free_bytes)`), and the built transient check charges it on every in-flight state the same way — and because a percentage has no global value to fold into: “10%” resolves to 2 TiB on a 20 TiB LUN and 200 GiB on a 2 TiB one, so a single global `max()` has no answer. Folded per storage, the deprecated floor cannot be lowered **as a plan-endpoint requirement** by the new knob — not by a smaller global `soft`, not by a per-storage override, not by a percentage landing on a small LUN — and the warning says exactly that: both keys are named, and the deprecated one is reported as a lower bound on every storage rather than as one resolved number. The transient floor is the one place the new knob may move it, deliberately and explicitly: see the hard sentence below. Two keys carrying different values are two requirements, and the union is the only resolution that cannot reduce safety — `max()` can only raise a floor, while any “winner” rule is a guess that can silently lower one. That is not hypothetical: an operator who adopts the new example config (which spells out `free_space: soft: 0`) while keeping an old `min_free_bytes: 1 TiB` hits the both-set case **on upgrade**, and a “soft wins” rule would drop a configured 1 TiB floor to zero behind a warning that reads like a deprecation notice. The transient side needs no rule of its own: `hard: null` (the default) means `hard_s = soft_s`, so the folded floor keeps the §8.1 predicate exactly as strong as the built `max(f_b * max(Z_b, z_d), min_free_bytes)` check, and an operator who then sets `hard` below it is using the new knob for the dip it exists to allow, on top of a floor the warning still names.

Validate as written, then fold. The fold is the *last* step of resolution: inheritance first, then percent-to-bytes conversion, then §11.1’s two `free_space` checks against the values the operator actually wrote, and only then `soft_s = max(soft_s_resolved, min_free_bytes)`. The order matters in both directions, which is why it is stated rather than left to the implementation.

- `hard_s ≤ soft_s` checked *after* the fold would be checked against a propped-up `soft_s`: a config whose written `hard` exceeds its written `soft` would pass for as long as a large `min_free_bytes` stayed in the file, and deleting the deprecated key — exactly what the warning asks the operator to do — would turn a running configuration into a startup failure. The typo has to be caught when it is written, not when the prop is removed.
- `soft_s < C_s` checked after the fold would error on an oversized *deprecated* floor, and that is the one deliberate non-promotion here: a `min_free_bytes` above some storage’s capacity has never been a startup failure (the built code reports it as that storage’s permanent shortfall, §9.5), and “the old key keeps working” cannot mean a running config refuses to start on upgrade. The error applies to the new knob’s own value; an oversized deprecated floor warns and reports as the unfixable

shortfall it always was.

The order has a third consequence, and it is the one an implementation gets wrong by following the list above too literally (REVIEW.md AH-01): the two validations run on *written* values, but a **null** hard is not a written value — it is defined as $\text{hard}_s = \text{soft}_s$, and that soft_s is the *folded* one. A null hard is therefore settled **after** the fold, from the folded soft_s ; resolving it before the fold (to $\text{soft}_s_{\text{resolved}}$) would leave $\text{hard}_s = 0$ for a config carrying only min_free_bytes and drop the floor from §8.1’s transient charge — silently, since that config gets no deprecation warning. A **written** hard stays as written: it is the operator’s explicit dip, validated against the written soft and never raised by the fold.

No schema_version bump: the old key keeps working, and the new block is additive — but the block is only *reachable* once `config_schema.json` carries it, because that schema is closed (`additionalProperties: false` throughout, deliberately: §11.1’s structural pass is where a typo’d key is caught). §12’s phase-13 row lists it first for that reason.

Soft is the plan endpoint; hard is the floor at every instant. soft_s is the requirement the finished plan must satisfy — (C5) enforces it, the lexicographic stage repairs it, and §6’s override bypasses the gates for it. hard_s is the floor the **transient** states may not cross: while moves are in flight a storage may sit below its soft requirement — a mirror target is fully allocated before its source releases anything (§8.1) — and hard_s is how far down it may dip. The scheduler’s feasibility predicate (§8.1) checks hard_s for every intermediate state; the solver’s (C5) checks soft_s for the endpoint. Defaults: $\text{soft}_s: 0$ (no requirement beyond the snapshot reserve — the pre-§5.3.1 behaviour, so the *model* is unchanged for a config that sets nothing: the same $\max(f_s \cdot Z_s, 0)$ floor, the same slack, the same §7.3 override). $\text{hard}_s: \text{null}$ at the **global** level means $\text{hard}_s = \text{soft}_s$ for every storage that does not override it, i.e. *no dip below soft at all* — the conservative reading, and the one that keeps §8.1’s invariant exactly as strong as it is today whenever an operator has not thought about transients. Per-storage null means *inherit the global value* (below), so the two nulls never collide: the global $\text{hard}_s: \text{null}$ is a value (“no dip”), the per-storage $\text{hard}_s: \text{null}$ is an absence (“use the global”).

Two validation rules (§11.1): $\text{hard}_s \leq \text{soft}_s$ is a **hard error** — a floor above the requirement would make every plan infeasible for a storage that already satisfies its soft requirement; and $\text{soft}_s < C_s$ — a requirement no disk could ever leave room for is a config typo, not a policy. A percentage of 100 or more never reaches that second rule: the grammar above rejects it at parse time.

An unsatisfiable requirement thrashes, and that is a deliberate cost. A soft the group cannot reach (say “30%” on a cluster that is 85% full) bypasses the drift and imbalance gates on *every* run (§6), and a plan that reduces the shortfall — the best a saturated group can do — is payback-exempt (§7.3). Both anti-thrash mechanisms are disabled for exactly the case that recurs forever, leaving only `cooldown_per_disk` between successive repair attempts. That is accepted on purpose: the alternative is hysteresis on a safety property, which §6 already forbids for the snapshot reserve. The mitigations are the reporting ones — the unfixable-shortfall line of §9.5 names the byte amount every run, so an operator staring at a permanent WARN fixes the config or the cluster — and the fact that a plan that moves nothing costs only read queries. Do not “fix” this with a back-off timer without revisiting §6’s no-hysteresis-on-safety rule first.

r_s is a **repair slack**, not a licence to overfill. A storage can already be violating the reserve when the engine first runs (see the worked example in §14), and a $\text{hard}_s \leq C_s$ would make the model infeasible and the tool useless exactly when it is most needed. Two ways to keep it effectively hard:

1. **Lexicographic, preferred and provably correct.** Solve in two stages: minimize $\sum_s r_s$ alone; then fix $\sum_s r_s$ to that minimum as a constraint and minimize the §5.4 objective. CBC supports this by re-solving. The reserve is then never traded against balance at any weight, and $\sum r_s > 0$ provably means *physically impossible*, not merely *unattractive*.

2. **Single-stage big-M**, simpler but requiring calibration: keep $P \cdot \Sigma_s r_s$ in the objective with P large enough that no achievable gain from the other terms can pay for a violation worth caring about. P must be **computed at model-build time, not taken from config as a fixed number**, because the bound depends on the group’s absolute load T_g (§4):

$$U_{\text{obj}} = 2 \cdot \alpha \cdot T_g + \beta \cdot |D| + \gamma \cdot \Sigma_d z_d + \kappa \cdot |V| \cdot (|S| - 1) + \delta \cdot 2 \cdot |S|$$

(upper bound on the
non-reserve objective;
 $\Sigma_s d_s \leq 2|S|$ by (C7))

$$P_{\text{min}} = U_{\text{obj}} / \varepsilon_r \quad \text{with } \varepsilon_r = \text{the smallest reserve shortfall we refuse to trade}$$

$\Sigma_s e_s \leq 2 \cdot T_g$ because every u_s lies in $[0, T_g/c_s]$ and the deviations from u^* sum to at most twice the total. Take $\varepsilon_r = 1$ MiB expressed in the same size unit as z_d (2^{-20} TiB if sizes are TiB), which says: never accept even a one-mebibyte reserve shortfall in exchange for balance. Config objective.reserve_violation_penalty is then a *floor*, not the value used: the model uses $P = \max(\text{configured}, P_{\text{min}})$ and logs a warning when it had to raise it. That warning must be *checkable*, not just an announcement: log $P_{\text{configured}}, P_{\text{min}}, P_{\text{used}}$, and the four inputs the bound came from — $T_g, |D|, \Sigma_d z_d, |V| \cdot (|S| - 1)$ — plus the ε_r granularity, and repeat them in pve-storage-drs explain. P_{min} moves with T_g , so the same config file legitimately yields different effective penalties on a quiet group and a busy one, and on the same group at different times of day. An operator who sets reserve_violation_penalty: 5000 and sees the engine using 2.3×10^7 needs to be able to reconstruct that number rather than take it on faith.

Worked against §14 ($T_g = 7.4, |D| = 6, \Sigma z = 6.5$ TiB, $|V| = 5, |S| = 3, \delta = 0.5$, sizes in TiB): $U_{\text{obj}} = 14.8 + 1.5 + 0.325 + 5.0 + 3.0 = 24.6$, so $P_{\text{min}} = 24.6 \cdot 2^{20} \approx 2.58 \times 10^7$. The configured default $P = 1000$ is **four orders of magnitude too small** to be provably dominant at mebibyte granularity — it is dominant for violations above roughly 25 GiB and silently tradeable below that. This is precisely why option 1 is the default and this option needs the computed P .

Use the lexicographic solve (option 1) by default. It needs no calibration, its correctness does not depend on T_g , and both backends support it by re-solving. Option 2 exists for a backend that cannot re-solve cheaply.

As built (REVIEW.md V-01): option 2’s P_{min} computation and $\max(\text{configured}, P_{\text{min}})$ floor above were never implemented — there is no build-time computation of a provably-dominant P anywhere in src/. Both MILP backends (optimize.py) implement option 1 only, and never consult objective.reserve_violation_penalty at all. The heuristic backend (heuristic.py) is the key’s one live consumer: it multiplies the *configured* value straight into its objective as $\text{reserve_penalty_term} = \text{objective.reserve_violation_penalty} \cdot \text{reserve_shortfall_tib}$, with no floor and no warning if it is too small to be provably dominant for a given group. Until option 2 is implemented, read this section’s $P = \max(\text{configured}, P_{\text{min}})$ machinery as the target design for the single-stage path, not current behaviour — see docs/manual/10-configuration.md’s objective.reserve_violation_penalty entry for what the key does today.

tests/fixtures/reserve-tradeoff.yaml (§14.6) is the fixture for this rule: a two-storage group in which the two options provably disagree, with the exact P at which big-M flips recorded alongside the computed P_{min} . Test both paths against it. The §14 fixture cannot do this job — there, every reserve-violating assignment is also worse on balance, so both options agree at any P .

Report any residual $r_s > 0$ prominently as an unfixable shortfall, with the byte amount.

(C6) Spread. With $u^* = (\Sigma_d \ell_d) / (\Sigma_s c_s)$ — a **constant**, since total group load is invariant under reassignment — the L1 form is fully linear:

$$u_s = (\sum_d \ell_d \cdot x_{\{d,s\}}) / c_s$$

$$u_s - u^* \leq e_s \quad \text{and} \quad u^* - u_s \leq e_s \quad \forall s \in S$$

The min-max alternative is $t \geq u_s \forall s$. L1 is the default: min-max only sees the single hottest storage and is indifferent to everything below it, which tends to produce plans that fix the worst storage and ignore a second nearly-as-bad one.

(C7) Capacity-spread linearization. Let $b_s = (\sum_d z_d \cdot x_{\{d,s\}} + U^{\text{ext}}) / C_s$ be the storage's **fill fraction** and $\bar{b} = (\sum_d z_d + \sum_s U^{\text{ext}}) / (\sum_s C_s)$ the group's mean fill — a constant, like u^* , because total bytes are invariant under reassignment. Then:

$$(b_s - \bar{b}) / \bar{b} \leq d_s \quad \text{and} \quad (\bar{b} - b_s) / \bar{b} \leq d_s \quad \forall s \in S$$

b_s is linear in x , so no variables beyond d_s are needed. Dividing by $\bar{b}$ makes the term **scale-free**: measured on absolute fractions, deviation shrinks as a group empties and the term would stop acting precisely where risk concentration is easiest to fix; relative to $\bar{b}$, a group filled to 5% that keeps 60% of its bytes on one storage deviates as much as a full one does. If $\bar{b} = 0$ the group holds no data: the term is inactive, and so is §6's capacity gate. The fill counts managed disks and foreign volumes but **not** the snapshot reserve — the quantity spread is *data at risk*, and free space is not data.

5.4 Objective

$$\begin{aligned} \min \quad & \alpha \cdot \sum_{\{s \in S\}} e_s && (\text{imbalance}) \\ & + \beta \cdot \sum_{\{d \in D^{\text{big}}\}} (1 - x_{\{d, \sigma_0(d)\}}) && (\text{number of migrations}) \\ & + \gamma \cdot \sum_{\{d \in D^{\text{big}}\}} z_d \cdot (1 - x_{\{d, \sigma_0(d)\}}) && (\text{bytes migrated}) \\ & + \kappa \cdot \sum_{\{v \in V\}} w_v \cdot (\sum_{\{s \in S\}} y_{\{v,s\}} - 1) && (\text{VM disk fragmentation}) \\ & + \delta \cdot \sum_{\{s \in S\}} d_s && (\text{data spread / failure risk}) \\ & + P \cdot \sum_{\{s \in S\}} r_s && (\text{reserve violation}) \end{aligned}$$

with $D^{\text{big}} = \{d \in D : z_d \geq \text{migration.tiny_disk_bytes}\}$ (default 64 MiB) and $w_v = \max(1, \ell_v / \ell)$ defined below.

$(1 - x_{\{d, \sigma_0(d)\}})$ is exactly 1 when disk d moves and 0 when it stays, so β directly implements “minimize the number of migrations” and γ biases against moving *large* disks specifically. Both terms range over D^{big} , not over D : a disk below `tiny_disk_bytes` transfers in under a second and holds its VM's lock for seconds, so charging it a migration count would price exactly the affinity-repair moves the κ term exists to enable out of the plan — a tiny disk rejoins its VM whenever that reduces spread, at zero objective cost. κ counts the number of **extra** storages a VM is spread across, so it is 0 for a VM whose disks are all together and grows by 1 per additional storage — a soft preference that free space (C5) or a strong imbalance can legitimately override, as required. Whether a VM *can* fit on one storage is (C5)'s business; where it cannot, κ still rewards keeping the spread as narrow as possible. Two strengthenings of the affinity term, both motivated by the same live finding (a plan of four pure affinity repairs rejected by the payback rule, §7.2):

- **The preference is weighted by the VM's own I/O.** $w_v = \max(1, \ell_v / \ell)$, where $\ell_v = \sum_{\{d \in D : v(d)=v\}} \ell_d$ is the VM's total load within the group (pinned disks included — their I/O is the VM's I/O) and $\ell = \tau_g / |V_{\text{all}}|$ the group's mean per-VM load, with V_{all} **every distinct VM owning a disk in the group, pinned-only VMs included** — deliberately wider than (C3)'s V , which by default ranges over movable-disk VMs only. A pinned-only VM's I/O is still real traffic on the group's storages, and diluting it out of the mean would make ℓ depend on which disks happen to be movable this run rather than on the group's actual load; the §14.7 fixture is worked with VM 309 pinned entirely out of V and still divides by all three of the group's VMs ($\ell = 6.0/3$, not $6.0/2$). A VM doing several times the average I/O is worth correspondingly more to keep together: its fragmentation is measured in the same in-flight-I/O unit as everything else (§4), and comparing

the full VM’s I/O against the single disks the balance term shuffles is exactly the comparison that decides whether the VM should move as a unit or be scattered. The floor of 1 keeps a quiet VM’s fragmentation worth the same as before — the observed failures were quiet VMs whose reunions the solver *did* want, killed later by payback — so only the ceiling is new; the default κ itself is deliberately unchanged and remains the knob to turn if quiet-VM cohesion should tighten further. w_v is data, not a variable, so this stays a plain per-VM coefficient, and the §14 fixture’s optima survive it (there the heaviest VM’s cohesion rises 2.7× and the split still wins — §14.3 works the new arithmetic).

- **Disks too small to matter are free to place.** With D^{big} excluding them from β and γ , and §7.1/§7.3 exempting them from cost and from the aggregate acceptance test, a tiny disk’s placement is decided by κ and the capacity constraints alone. Churn stays bounded: κ only rewards *reducing* spread, and per-disk cooldowns (§6) still apply.

κ measures **within-group** fragmentation only. Because the problem decomposes per group and a disk can never leave its group, a VM with disks in two different groups is not counted as fragmented — that spread is structural and no migration could ever repair it. This is a consequence of the decomposition, not an oversight.

δ spreads **bytes**, not load. The failure of a storage takes with it everything on it, so an even fill fraction bounds the share of the group’s data that any single failure costs, and it keeps peak fill — hence reserve headroom — uniform across the group. I/O balance stays the first priority: $\Sigma_s e_s$ and $\Sigma_s d_s$ are both sums of per-storage deviations from an equal-share target on comparable relative scales, δ defaults to half of α , and on an imbalanced group the α term dominates every plan comparison (in §14.2: $\alpha \cdot \Sigma e = 8.07$ against $\delta \cdot \Sigma d = 1.08$). δ decides among plans the load objective is nearly indifferent between, and acts on its own only when a group’s I/O is already balanced but its data is concentrated — the case §6’s capacity gate exists to reach.

Like β and κ , δ is an exchange rate, not a lexicographic order — a strict “I/O first, bytes only among exactly equal load optima” rule would never act, for the same reason §1 gives for refusing a strict migration minimum. It says how much summed utilization deviation one unit of summed relative fill deviation is worth (`objective.delta_capacity_spread`, default 0.5; 0 disables the term). Raise it to spread data more aggressively — §11.1 warns once δ exceeds α , the point where data evenness starts outweighing I/O evenness in every comparison. The term is L1 and stays L1 whatever `objective.spread_metric` is set to: the risk argument cares about every storage’s share, not only the fullest one.

Scaling matters: express z_d in TiB and ℓ_d in average in-flight I/O requests (§4) before applying the weights, so the defaults in the example config are meaningful.

5.5 Solver backends

CBC via PuLP — the one MILP backend. Direct transcription of §5.3’s constraint set and §5.4’s objective; continuous e_s , Z_s , r_s are fine, so there is no scaling discipline at all. Two unit rules still apply, both for conditioning rather than correctness: size-valued quantities are expressed in whole MiB (Z_s , R_s , r_s , z_d , C_s , U^{ext} , soft_s) and loads in average in-flight I/O requests (§4), so raw bytes ($\sim 10^{12}$ – 10^{14}) never sit next to load values (~ 1 – 10) in the LP matrix — CBC’s simplex does not error on a badly conditioned matrix, it silently returns a numerically poor “optimal” (confirmed on a real corpus bundle, where an unscaled model made CBC’s own post-plan spread almost 100× worse than the heuristic’s on the same weights).

Assert after solving that the unscaled objective recomputed in floating point from the returned assignment — `heuristic.evaluate_assignment()`, the one objective implementation every backend reports through — agrees with what the solve was told it achieved; a cheap guard against a modeling mistake silently producing wrong plans.

A **CP-SAT (ortools) backend was specified here once and removed** (REVIEW.md AL-02): not in Debian, excluded from vendoring by this section’s own dependency rules, and not something operators install

by hand on PVE hosts, so `solver.backend: auto` resolved to CBC on every deployment — the second model builder, its integral-coefficient discipline (per-coefficient constant folding, the κ/ω scales, the γ -quantization trap of REVIEW.md F-14, the int64 magnitude assertions) and its CI/packaging exceptions existed only for a backend production could never run. Those rules are CP-SAT’s own and were removed with it; a future integer backend must re-earn F-14’s analysis rather than inherit it. The \$14 fixture agreement that once read “CP-SAT and CBC agree on every β case” now binds CBC to the exhaustively-enumerated optimum alone.

Heuristic fallback (no dependency, and the path for very large groups).

1. **Seed** with the current assignment (not from scratch — we are minimizing *change*).
2. **Repair**: while any s violates (C5), move the disk from s that most reduces the violation per byte moved, to the feasible storage with the lowest u_s .
3. **Descend**: repeatedly evaluate every single-disk move, every pairwise swap, **and every whole-VM co-relocation** (every movable disk of one multi-disk VM moved to the same target storage together, in one trial); apply the one that most improves the full objective (including β , γ , κ , δ); stop when no move improves it or `heuristic_iterations` is reached. The third candidate family was added after the first two (confirmed live on a real production cluster, not found by review): whenever κ is large enough to make moving one disk of an N -disk VM a net loss on its own (it pays κ ’s fragmentation penalty before a later move could reunite it), neither a single move nor a swap can ever reach the state where relocating the whole VM together is a clear win — descend found *zero* improving moves at all on a real, 196%-imbalanced cluster without this candidate, which a “the path for very large groups” fallback must not do.
4. **Polish**: attempt to reunite fragmented VMs where doing so does not worsen imbalance beyond `imbalance_threshold`. With step 3’s whole-VM co-relocation above, this step’s remaining scope is narrower than originally written: only an *N-way rotation* across three or more storages (no disk’s own move improving alone, and no two disks sharing a VM) is still outside descend’s own reach.

Swaps matter and must not be omitted: when both storages are near their capacity limit, no single move is feasible, and only an exchange of two disks can improve the balance.

The heuristic must use the **same** feasibility and objective functions as the MILP path so the two backends are directly comparable; make them shared pure functions.

6. Gating — deciding whether to act at all

Applied in order, before the solver runs. Any gate that fails ends the run with “no action”.

Drift gate — the “minimum % of changed average traffic” requirement. Compare the current load vector against the one recorded at the last *executed* balance:

```
||l_now - l_last||1 / ||l_last||1 ≥ gates.drift_threshold (default 0.10)
```

Using the L1 norm over the whole vector, rather than a per-disk test, means many small correlated changes can legitimately trigger a re-plan while one noisy disk cannot.

Vector alignment. The two vectors are indexed over the **union** of disk keys present in either, with a missing disk contributing load 0 to the vector it is absent from. A disk created since the last balance therefore contributes its full current load to the numerator, and a deleted disk contributes its full former load — both are genuine changes to the group’s I/O profile and should be able to trigger a re-plan on their own.

Degenerate cases, which must be handled explicitly rather than left to produce a division by zero:

Condition	Behaviour
No ℓ_{last} recorded (first run ever, or state file reset)	Skip the drift gate , proceed to the imbalance gate
$\ \ell_{\text{last}}\ _1 = 0$ (group was entirely idle) and $\ \ell_{\text{now}}\ _1 > 0$	Treat as fully drifted, proceed
$\ \ell_{\text{last}}\ _1 = 0$ and $\ \ell_{\text{now}}\ _1 = 0$	No load, no imbalance — exit “no action”

ℓ_{last} is recorded **only on a run that actually executed at least one migration**, not on every run. Recording it on planning runs would let load creep past the threshold in sub-threshold steps without ever triggering.

Imbalance gate — the “% of IOPS difference over all storages in the group” requirement:

```
(max_s u_s - min_s u_s) / u* ≥ gates.imbalance_threshold (default 0.20)
```

Evaluated per group; a group that passes it — or the capacity gate below — is planned, others are skipped.

Capacity gate — the data-spread counterpart of the imbalance gate, on fill fractions rather than loads:

```
(max_s b_s - min_s b_s) / b̄ ≥ gates.capacity_spread_threshold (default 0.25)
```

with b_s and $\bar{b}$ as in §5.3 (C7). A group that passes it is planned even when its I/O is perfectly balanced — in that case the $\bar{b}$ term of §5.4 is what does the work — and it bypasses the drift and imbalance gates for the same reason the reserve override does: a stable workload is not a reason to keep data concentrated. Unlike a reserve violation this is a preference rather than a safety property: cooldowns, payback (§7) and the transient invariant (§8.1) all still apply, and `gates.capacity_spread_threshold: null` disables the gate — with `objective.delta_capacity_spread: 0`, the whole feature. The gate cannot fire when $\bar{b} = 0$: an empty group has nothing to spread.

Cooldowns — a disk moved within `cooldown_per_disk` (default 24h) is pinned in place; a storage involved in a migration within `cooldown_per_storage` accepts no new incoming moves.

Reserve override — a storage in violation of (C5) — snapshot reserve or configured free space (§5.3.1), at plan time — bypasses the drift and imbalance gates entirely. Safety is not subject to hysteresis, and neither is the free-space mandate: an admin’s new VM on an overfull storage must be reacted to on the very next run, not once the workload has drifted 10%.

7. Migration cost and the payback rule

This section implements the requirement that *migrating a very large disk may generate more traffic than it saves*. It is expressed as a **hard acceptance test on the finished plan**, not merely as a soft γ penalty, because a penalty can always be outweighed by a large enough imbalance term.

7.1 Cost

A `move_disk` on a running VM performs a QEMU drive-mirror: it reads the whole source volume and writes it to the target, then switches over. For thick provisioning the full provisioned size is transferred. But the mirror is only the first half of the operation — `delete=1` then removes the source volume, and on a storage with `saferemove` enabled that removal is a full-size **zeroing pass**, throttled and often far slower than the mirror it follows:

```
duration_mirror_d = z_d / min(bwlimit, headroom_src, headroom_dst)

duration_wipe_d   = z_d / |saferemove_throughput(σ₀(d))|   if saferemove is enabled there
                  = 0                                       otherwise

duration_d        = duration_mirror_d + duration_wipe_d
```

```
cost_d = duration_mirror_d · (ω_src + ω_dst) + duration_wipe_d · ω_wipe
```

ω_{src} and ω_{dst} (default 1.0 each) are the added in-flight I/O on source and target — a mirror is one sequential reader plus one sequential writer, so 1.0 each is the natural unit and is directly comparable to ℓ , which is measured in the same units (§4). $cost_d$ is therefore in **load-seconds**. The wipe is charged to the source only, because it is a sequential write over the old volume with nothing happening on the target; ω_{wipe} (`migration.wipe_load_weight`, default 1.0) is its own weight so an operator who knows their array shrugs off a throttled zeroing pass can lower it without touching the mirror weights. §7.3 charges the same quantity to the saturation guard for the whole draining window.

A disk below `migration.tiny_disk_bytes` costs nothing. $cost_d = 0$ when $z_d < tiny_disk_bytes$ (default 64 MiB — comfortably above an EFI var store or TPM state, and far below any disk the payback rule was written for), and §7.3’s aggregate test counts neither its cost nor needs its benefit. A 528 KiB `efidisk0` mirrors and wipes in seconds; asking it to justify itself against λ is a category error — this rule exists for the migration whose own traffic can outweigh its benefit, and no disk this small can generate such traffic. Together with §5.4’s D^{big} exemption this is what makes affinity repair of tiny disks possible at all.

Why the wipe term is not a rounding detail. PVE’s LVM `saferemove` (“Wipe Removed Volumes” in the UI) defaults to a throughput of **10 MiB/s**. At that rate the 1.5 TiB disk of the §14 example takes about **44 hours** to wipe, against roughly 2.2 hours to mirror it at 200 MiB/s — the cleanup is twenty times the move. A cost model that stops at the mirror understates such a migration by that factor and will happily schedule a plan that occupies the source array for two days.

`saferemove_throughput` is signed, and the sign is not part of the rate. PVE passes the configured value straight through to `cstream -t`, where the sign selects *how* the limit is enforced and the magnitude is the rate in bytes/second (`cstream(1)`, confirmed by the operator against a production cluster):

- **positive** — a session average. `cstream` accumulates its own error and may exceed the rate for a while to make good on earlier underutilization, so the whole session converges on it.
- **negative** — an upper limit on each individual read/write syscall pair, never exceeded.

So `-1073741824` means 1 GiB/s, not minus anything, and negative values are ordinary in PVE configurations — they are what an operator writes when they want a rate the wipe can never burst above. Hence the `| ... |` in the formula. $z_d / |throughput|$ is the right estimate under either sign; under the negative one it is additionally a hard floor, since the wipe cannot finish ahead of a rate that is never exceeded.

Dividing by the signed value is not a cosmetic error. `duration_wipe_d` would come out **negative**, `duration_d = duration_mirror_d + duration_wipe_d` would collapse towards zero — to *exactly* zero on the common configuration where the operator sets `|saferemove_throughput|` equal to `migration.bwlimit_bytes_per_sec` — and §7.3’s `max_single_move_duration` rejection, along with `verify-storages’ cooldown-versus-wipe` warning, would then be unable to fire for a disk of any size. This is not hypothetical: it was found by replaying a real bundle from a cluster whose three LVM storages all carry `saferemove_throughput -1073741824` against a `bwlimit_bytes_per_sec` of 1073741824, where a hypothetical 100 TiB move reported a total duration of 0 s and passed a 6 h limit.

Read `saferemove` and `saferemove_throughput` from `GET /storage` (§3.5 — the list form, not `GET /storage/{id}`) per storage; never assume, and never assume a sign either. `migration.account_saferemove_wipe: false` disables the term for an operator who has verified their storages do not wipe, but the default is to account for it. Note the knock-on effects, all covered in §9.3: the wipe also determines when the source’s space is actually released, and it holds a storage-level lock while it runs.

7.2 Benefit

The plan improves the three *persistent* parts of the §5.4 objective — load balance, data spread and VM affinity — from $(E_{\text{before}}, F_{\text{before}}, A_{\text{before}})$ to $(E_{\text{after}}, F_{\text{after}}, A_{\text{after}})$, where $E = \sum_s e_s$, $F = \sum_s d_s$ (§5.3 (C7)), and $A = \sum_{\{v \in V\}} w_v \cdot (\sum_{\{s \in S\}} y_{\{v, s\}} - 1)$ is the objective’s affinity debt, weights included — the same sum the κ term charges, from the one shared implementation (AGENTS.md §5). All three improvements persist for as long as the workloads keep running on the new placement, and we account them over the payback horizon H :

$$\text{benefit} = (\alpha \cdot (E_{\text{before}} - E_{\text{after}}) + \delta \cdot (F_{\text{before}} - F_{\text{after}}) + \kappa \cdot (A_{\text{before}} - A_{\text{after}})) \cdot H$$

also in load-seconds — δ converts relative fill deviation, and $\kappa \cdot w_v$ a busy VM’s fragmentation, into load-deviation equivalents (§5.4), so both sides of §7.3’s comparison stay in the unit that is the whole point of using I/O time as the load metric. β and γ charge the move itself and belong on the cost side.

H defaults to 365d (`migration.payback_horizon`), with $\lambda = 10$. The reasoning is the asymmetry of the two sides: a migration’s cost is paid once and early — days of mirror I/O (§7.1), plus a source wipe that can outlast the mirror by an order of magnitude — while its benefit accrues for as long as the placement lasts, which for infrastructure is months to years. Assume, by default, that a VM balanced today keeps running where it is for at least another year. A short horizon asks “does this pay back before the bruise heals?”, and at the previous 7d default the answer was *no* for every move worth less than $\lambda \cdot \text{cost}/H - 0.17$ of summed deviation per TiB moved, 0.69 for the 4 TiB disk of \$14.5 — rejecting, at the same λ , precisely the slow-accruing but real benefits a balancer exists to capture: a week of degraded I/O on the storages looks bad, and still pays for itself many times over across a year of better balance.

Three things H is **not**, to keep the semantics honest:

- **Not a prediction that nothing changes for a year.** Drift and operator action will re-plan long before that (§6), and a re-plan does not claw back benefit already accrued. H is the accounting period over which a recurring benefit pays for a one-time cost — the convention capacity planning has always used for exactly this shape of decision.
- **Not the brake on doing too much.** That role belongs to β/γ and the gates. With a one-year horizon the aggregate test’s remaining job is to reject plans whose persistent-objective improvement is genuinely negligible — $\Delta \text{obj} < \lambda \cdot \text{cost}/H$, about 0.008 for the two-move plan of \$14.5 — which is the right shape for a safety test: a guard against absurdity, not the main cost control.
- **Not universal.** It is an assumption about VM lifetime. Short-lived fleets (CI, render farms, lab clusters) should lower it toward the actual lifetime of their VMs; \$11.1 warns below 30d, where the test starts rejecting real benefits again.

κ belongs in this sum, and its earlier exclusion was a defect. The original wording here read “ κ is a preference, not a physical benefit, and appears in neither” — and dogfooding broke on exactly that sentence. A plan whose four moves were *all* affinity repairs (two of them 528 KiB EFI disks) scored $\text{benefit} = 9 \text{ load} \cdot \text{s}$ against $\text{cost} = 232 \text{ load} \cdot \text{s}$ — ratio 0.0388, reject — and the tool refused the one action the operator most wanted. No oversized balance move was involved; the rule rejected a plan it had never been given a way to value. Co-location persists exactly as long as balance does, and its worth is measurable — the fragmented VM’s own I/O, which is what w_v weighs. Two properties of the corrected term, stated so they are not quietly reverted:

- **ΔA may be negative**, and then it *reduces* the benefit: a balance move that splits a VM must pay for the fragmentation out of its α gain. The \$14 fixture’s two-move plan carries both signs at once (§14.5).
- **At $H = 365\text{d}$ any nonzero $\kappa \cdot \Delta A$ dwarfs any achievable cost** (a 4 TiB mirror costs $\sim 4 \times 10^4$ load·s; one reunited average-load VM is worth $0.5 \times 31\,536\,000$). For affinity-motivated plans the aggregate

test therefore defers to the solver’s own κ -versus- β/γ pricing — a genuine cost/benefit test in objective units, not a rubber stamp — and the test’s remaining job stays the one the second bullet above gives it: rejecting plans whose *physical* benefit is negligible.

7.3 Acceptance

```
accept plan  $\iff$  benefit  $\geq$  migration.payback_ratio  $\cdot \sum_d$  cost_d
```

with $H = 365d$ and $\lambda = 10$ by default. The sums — and the test itself — cover only disks with $z_d \geq \text{migration.tiny_disk_bytes}$: a smaller disk’s move carries $\text{cost}_d = 0$ (§7.1) and needs no verdict, while every hard rule below applies to it exactly as to any other move. Additional **hard** rules, applied per move, that reject individual migrations regardless of the aggregate test:

- $\text{duration}_d > \text{migration.max_single_move_duration}$ (default 6h) $\rightarrow$ reject the move;
- the move would push either endpoint above $\text{migration.saturation_ceiling} \cdot \text{saturation_load}$ during the mirror $\rightarrow$ defer the move to a later run rather than reject the plan (see below); skipped for a storage with no saturation_load configured;
- the move violates the transient reserve invariant of section 8 $\rightarrow$ reject.

A plan that repairs is exempt from the aggregate test. The trigger is the plan’s *outcome*: its final assignment’s $\sum r_s$ is strictly below the current assignment’s — the plan leaves the group with less reserve/free-space shortfall than it found, and no mirror cost may veto that (the Storage-DRS mandate). The “final assignment” is the **scheduled** one — the state the ordered moves actually reach, $\text{schedule_result.final_assignment}$, the same R-02 distinction the payback benefit already draws (cli.py evaluates its final_breakdown against it, not against the solver’s aspirational target): a partially deadlocked plan is scored on what it will really run, and a plan whose *target* repairs but whose schedule never gets there is not exempt. **“What it will really run” means after the hard per-move rules below have taken their moves out, not merely after ordering.** The two rules that fire at this gate rather than during scheduling — the $\text{max_single_move_duration}$ rejection and the saturation-ceiling deferral — drop moves from what is executed but not from $\text{schedule_result.order}$, so a repair move either of them refuses would otherwise sit in the order, satisfy the trigger, and buy a **plan-level exemption for the balance moves that survive it** — the economic test skipped on a plan that no longer repairs, and whose shortfall §9.5 then reports as unmet. Take both sums over the move set the gate will actually execute (the order minus the refused and deferred moves), which is well-defined because neither refusal depends on the exemption: both are per-move verdicts on $\text{cost}_d/\text{duration}_d$ alone, computed before the aggregate test is consulted. §8.1’s transient invariant needs no such treatment — $\text{order_moves}()$ enforces it while building the order, so a breaching move never reaches it in the first place. The outcome trigger, not a per-move flag, is what makes the mandate total: a plan of **redundant repairs** — two moves from a violating storage where *either one alone* repairs it — contains no move that passes the revert test below, yet the plan still repairs and is still exempt; a per-move trigger would let payback veto exactly that plan and leave the violation standing. Which moves carried the repair is reported per move by the **revert test**: a move is marked repair: true iff holding that one disk on its current storage would strictly raise the plan’s final $\sum r_s$ — its source ends the plan below its snapshot reserve or its configured free-space requirement (§5.3.1), or the move empties the destination another repair needs (§5.3, §14.8). The test is evaluated by re-scoring $\sum r_s$ on the plan’s final assignment — the same scheduled assignment the trigger scores — with that one $x_{\{d, \sigma(d)\}}$ held — no re-solve. Repairs are not priced: the requirement is the operator’s configured policy. The exemption is **plan-level**: one repairing plan skips the economic test for the whole plan, and every move in it carries its $\text{repair: true/false}$ marker in the plan output so the operator can see which move carried the repair. (The marker and the trigger are allowed to disagree, in either direction — the redundant-repair plan above repairs with no marked move, and the converse exists too: a move can be load-bearing on a plan that does not repair — holding it back raises the plan’s *final* $\sum r_s$ above zero

while the current assignment was already at zero, e.g. one half of a swap whose other half lands a new largest disk and needs the room this move frees. Such a move is marked `repair: true` — and, once §8.2’s exception 2 is implemented (it is spec-only today, as §14.8 records), scheduled first, because it frees space a later move needs — but its plan is *not* exempt — the plan leaves the group no better than it found it, and a plan that does not repair has nothing the mandate protects.) This is deliberately narrower in its trigger than the built `payback.py` already implements (`has_reserve_override` fires when *any* move’s source was violating at scheduling time — a plan that moves a disk off a violating storage but ends no less short is exempt today and will not be under the outcome trigger), and the free-space work replaces that detection with the outcome test plus the revert-test markers. The breadth cuts both ways, and the wide side is a deliberate cost too: the trigger is one byte of shortfall reduction, so a plan that repairs by a byte while carrying expensive balance moves is exempt **in full** — the economic test is skipped for the whole plan, not prorated, and on a permanently-short cluster (§5.3.1’s thrash paragraph) every shortfall-reducing plan is fully payback-exempt for as long as the shortfall stands. That is accepted for the same reason the narrower side is: the mandate does not grade repairs by size — a partial repair is still the operator’s configured requirement being met — and there is no principled way to price only part of a repairing plan, because the benefit is plan-level (§7.2’s before→after on the whole assignment), so “price the balance moves but not the repair” would mean inventing a per-move benefit that does not exist. What bounds it: the three hard rules above still apply to every move in an exempt plan, the plan output carries every move’s cost and its repair marker so the operator sees exactly what the exemption bought, and on a healthy cluster the trigger is rare — rarer than the condition §6’s override already singles out, in fact: the outcome trigger is a **strict subset** of both. Σr_s can only fall if some storage’s `used_s` or `Z_s` falls, which requires a disk to *leave* that storage — and a storage whose shortfall a move reduces was in violation when the move left it, so every outcome-exempt plan is also exempt under today’s `has_reserve_override` and §6’s override was already open for it. The change can only *remove* exemptions, never create one, which is what makes “deliberately narrower” a provable claim rather than a comparison. A repair that cannot finish inside `max_single_move_duration`, or that breaches the transient invariant, is rejected like any other move and the shortfall reported as unfixable.

The shipped `fc-tier1` fixture survives this unchanged, and its plan is exactly the redundant-repair case. `san-a` starts $r = 0.5$ (4.5 used + 4.0 snapshot reserve > 8.0 TiB); the §14 two-move plan ends at $\Sigma r_s = 0$, so the outcome trigger exempts it — as the built `has_reserve_override` already does (102:scsi0’s source is violating when it is scheduled). Neither move passes the revert test, though: holding 102:scsi0 back leaves `san-a` at $3.5 + 4.0 = 7.5 \leq 8$, holding 101:scsi1 back at $3.0 + 4.0 = 7.0 \leq 8$ — either move alone repairs, so both are marked `repair: false`. The fixture’s recorded payback numbers (`fc-tier1.expected.json`: cost 26 214.4, benefit 1.93×10^8 , ratio 7 344.4, accepted: true) are untouched — the sums are computed in full whatever the trigger, and this plan clears the aggregate test on its own anyway ($7\,344 \geq 10$). Only the per-move flag changes (102:scsi0 is `resolves_reserve_violation: true` before phase 13, unmarked under the revert test). **As built (AH-02):** `fc-tier1.expected.json` never recorded the old flag, but it does record the new surface — per-move `disk_key`, the plan’s `aggregate_ok`, the `reserve_shortfall_tib_before/_after` pair (0.5 → 0.0), `repair_exempt: true` (the outcome trigger fires; the plan would have cleared the aggregate test regardless) and `repair_markers` (both false, the redundant-repair case) — with every recorded payback *number* unchanged; the corpus expected files record the same fields.

A plan of nothing but tiny disks has no economic gate at all, and that is deliberate — but it means the objective is the only thing holding it. $\Sigma_d \text{cost}_d$ is then 0, so — outside the repair exemption above, which skips the test anyway — $\text{benefit} \geq \lambda \cdot 0$ reduces to $\text{benefit} \geq 0$ and any non-negative benefit passes; the implementation reports the ratio as `+inf`. That is the intended reading of “needs no verdict” above, and it is the whole point of §7.1’s exemption: a 528 KiB `efidisk0` rejoining its VM must not have to out-earn a rule written for multi-terabyte migrations.

State the consequence plainly, because it was not obvious and it cost a real dogfooding cycle to find. For such a plan, nothing downstream of the §5.4 objective asks whether the moves are worth making. If the objective scores a move at $+\epsilon$ for any $\epsilon > 0$, the move happens — and ϵ can be arbitrarily small, because no term in the objective has a materiality floor either. So the *correctness of the objective’s affinity term is load-bearing for tiny moves in a way it is not for any other kind of move*, and a defect in it surfaces directly as migrations that buy nothing.

Exactly that happened. With §5.3 (C3)’s affinity term excluding pinned disks — the pre-fix default, see §3.6 — two 528 KiB `efidisk0` moves on a real cluster scored a κ gain of precisely zero and were emitted anyway on a 3.6×10^{-7} capacity-spread difference, a relative improvement of about 6×10^{-8} , on which the three then-existing solver backends did not even agree (CP-SAT emitted two moves, CBC none; the CP-SAT backend has since been removed — AL-02). Each was a live migration with a VM lock, a PVE task and a `saferemove` wipe, and each burned `gates.cooldown_per_storage` on its target. Correcting (C3)’s default turned the same two moves into genuine reunifications worth a discrete 1.0 of objective, and the three backends into agreement.

No materiality floor is specified, and none should be added speculatively: any threshold would be a magic number standing in for a decision this model does not otherwise need to make, and the observed failure was a defect in the objective rather than a missing gate. But the branch is a real one, and a future bundle showing tiny moves emitted with $A_{\text{before}} = A_{\text{after}}$ is evidence to revisit it — the narrowest available fix being to require $A_{\text{before}} > A_{\text{after}}$ for a zero-cost plan, which needs no threshold because the affinity debt moves in discrete steps.

Defining “during the mirror”. u_s as used everywhere else is a p95 over the lookback window — a robust *statistic*, not an instantaneous reading — so adding an instantaneous ω to it would mix two different kinds of quantity. Define the check explicitly:

```
L_during(s) =  $\hat{L}_s(\text{duration}_d)$  +  $\sum_{\{m \text{ in flight at } s\}} \omega_{\text{role}(m,s)}$ 

check: L_during(s) ≤ saturation_ceiling · N_s      for s ∈ {src, dst}
```

where $\hat{L}_s(\Delta)$ is the **forecaster’s upper bound on L_s** — the storage’s *aggregate* load over a horizon equal to the move’s expected duration (§10), in average in-flight I/O requests, the same units as l (§4) and as ω . It is **not** the capability-normalized u_s ; mixing the two here was the original defect in this rule.

$\omega_{\text{role}(m,s)}$ depends on the move’s state, not only on its endpoints. A move stays in the in-flight set M until it reaches done (§8.2). That is right for capacity, but a *fixed* role charge would be wrong for load: once the mirror has switched over, this move writes nothing more to the target, while the source is being **zeroed** for as long as `saferemove` takes. So:

state	charge on src(m)	charge on dst(m)
-----	-----	-----
mirroring	ω_{src}	ω_{dst}
draining	ω_{wipe}	0
done	0	0

ω_{wipe} is `migration.wipe_load_weight`, default 1.0 — the zeroing pass is one sequential writer, so 1.0 is the natural value and it is the same quantity §7.1 charges for `duration_wipe_d`. With this, a 44-hour wipe on a busy source stays visible to the saturation guard for its whole duration instead of vanishing from the check the moment `move_disk` reports OK, which is the only way the guard can protect the *next* move scheduled onto that storage. The capacity invariant of §8.1 needs no change: it already holds the move in M until done, and the source-side byte accounting of `mirroring` and `draining` is identical.

Two honest caveats. The wipe is throttled by construction (10 MiB/s by default), so charging it a full ω is

conservative — deliberately so, because the alternative is to under-count a storage that is busy zeroing 1.5 TiB. And this remains a best-effort guard: a deployment with no `saturation_load` set skips it entirely, and there `execution.cooldown_per_storage`, sized against the wipe time (§9.3), is the blunter mitigation that still works.

N_s is what the check is measured against, and it is not c_s . c_s is a *relative* capability weight whose default is 1.0 and whose absolute value is meaningless — only the ratios between storages in a group affect the balance objective, so `saturation_ceiling ·  $c_s$` compares a physical queue depth against a dimensionless preference. That is dimensionally wrong in both directions: with $c_s = 1.0$ a storage carrying an entirely healthy $L_s = 6.5$ would fail a 0.85 ceiling outright, and a storage weighted $c_s = 0.5$ would be held to half the ceiling of its peer purely for being labelled less capable. Instead:

```
N_s = storages[s].saturation_load    - the number of concurrent I/O requests storage s services
                                     before queueing delay dominates. An absolute, physical
                                     property of the array (roughly its effective queue depth).
```

N_s has **no safe default and is null unless the operator sets it**, in which case the check is skipped for that storage and `pve-storage-drs explain` says so. We cannot infer it: the observed peak L_s is not a capacity (an idle storage would get a tiny N_s and reject every migration onto it, which is exactly backwards), and neither c_s nor the LUN size tells us anything about queue depth. Obtain it from the array’s documented queue depth, or empirically as the L_s at which measured latency starts climbing super-linearly. Sizing N_s in the same units as l is straightforward because both come from the same Little’s-law quantity.

This is deliberately a **best-effort guard**, not a physical limit: even with N_s set we have no model of the array’s true saturation point, only the load we can attribute to guests. `max_single_move_duration` and the transient reserve invariant of §8.1 are the hard bounds and are always active; this one exists to avoid the obviously bad case of starting a long mirror onto a storage that is already close to its service limit. A deployment that leaves every `saturation_load` unset is fully supported and loses only this one advisory check.

If the plan fails the aggregate test, re-solve with β and γ doubled and retry, up to three times. This naturally converges on the smaller subset of high-value moves rather than abandoning the run — usually the one or two disks with the highest l_d / z_d ratio, which is exactly the right thing to move.

As built: the three changes this section and §7.2 specify — the w_v weighting of κ , the `tiny_disk_bytes` exemptions, κ ’s place in the benefit — are implemented: `payback.py`’s `compute_benefit_load_seconds()` takes the three-term formula, both solver backends fold a per-vmid $\kappa \cdot w_v$ coefficient (`heuristic.compute_vm_weights()`, one implementation shared by both MILP backends and the heuristic), and the four-affinity-repair plan that previously scored `benefit 9 load · s vs cost 232 load · s` → ratio 0.0388 $\times$ and refused to act now prices the reunions correctly (see §14.7’s `affinity-repair.yaml`, the fixture built to prove exactly this). The β/γ -doubling re-solve above remains unimplemented — a separate, still-open scope decision (`payback.py`’s own module docstring lists it under “deliberately not implemented in this pass”), not part of this fix.

l_d / z_d — load per byte — is worth surfacing in `pve-storage-drs explain` output. It is the single best indicator of a good migration candidate: high I/O concentrated in a small disk.

8. Migration ordering

The solver produces a *target assignment*. It says nothing about the order of moves, and order matters: the requirement that the snapshot reserve holds **even during storage migrations** is a constraint on every intermediate state, not just the endpoints.

8.1 The transient invariant

During a `move_disk` of disk `d` from `a` to `b`, the volume exists on **both** storages — the mirror target is fully allocated before the switchover, and the source is only removed afterwards by `delete=1`. So while a single move is in flight, `b` must satisfy:

$$\text{used}_b + z_d + f_b \cdot \max(Z_b, z_d) \leq C_b$$

Note the $\max(Z_b, z_d)$: if the incoming disk is the new largest on `b`, the required reserve grows at the same moment the disk arrives. This is the case most likely to be missed, and the one most likely to fill a SAN LUN.

The transient free-space floor. The snapshot term above is not the only thing `b` must keep: a storage with a configured free-space requirement (§5.3.1) must not dip below its **hard** floor while the move is in flight. The full single-move predicate is therefore:

$$\text{used}_b + z_d + \max(f_b \cdot \max(Z_b, z_d), \text{hard}_b) \leq C_b$$

$\text{hard}_b \leq \text{soft}_b$ always (§5.3.1 validation), so the *floor component* of this predicate is a relaxation of the endpoint constraint (C5) — the endpoint demands $\max(f_b \cdot Z_b, \text{soft}_b)$ free, the transient state demands only $\max(f_b \cdot \max(Z_b, z_d), \text{hard}_b)$ of floor. The rest of the predicate is **stronger** than (C5), not weaker, and must not be read as implied by it: the snapshot term grows to $f_b \cdot \max(Z_b, z_d)$ the moment the incoming disk is the new largest, and the source is still charged in full — the two facts that make §8.1 a separate invariant rather than a corollary. With the default `hard: null` (i.e. $\text{hard}_b = \text{soft}_b$) the floor component does not relax at all and the transient check is exactly as strong as before §5.3.1; an operator who sets `hard` strictly below `soft` buys the scheduler room to land a disk on a storage that is *heading toward* its soft requirement — the target’s fill rises transiently past soft_b and the plan’s final state pulls it back down — without ever crossing the hard floor. The dip is bounded and planned, not discovered: the solver only ever emits moves whose *endpoints* satisfy (C5), so a transient dip below `soft` can occur solely on a storage the finished plan leaves compliant — or, when the plan ends with $r_s > 0$ (§5.3 permits it: physically unachievable), on a storage whose shortfall the plan already minimizes and reports.

The source `a` gets no relief until the move completes, so a plan that depends on freeing space on `a` to make room on `a` is simply infeasible and must be ordered around.

Generalized to concurrent moves. `execution.max_concurrent_migrations` may exceed 1, and then the single-move form above is **not sufficient**: several disks can be landing on `b` at once, and none of their sources release space until each completes. For an in-flight set `M`, every storage `b` must satisfy:

$$\begin{aligned} \text{used}_b &+ \sum_{\{m \in M : \text{dst}(m)=b\}} z_{\{\text{disk}(m)\}} \\ &+ \max(f_b \cdot \max(Z_b, \max_{\{m \in M : \text{dst}(m)=b\}} z_{\{\text{disk}(m)\}}), \text{hard}_b) \leq C_b \end{aligned}$$

Both the sum and the inner `max` are over the same in-flight set, and the $\max(\dots, \text{hard}_b)$ floor of the single-move form carries over unchanged. Implement this as the single feasibility predicate and call it with $M = \{m\}$ for the sequential case, so there is only one version of this rule in the codebase.

`concurrency_ok(state, m)` is then defined as: adding `m` to the current in-flight set

1. keeps the generalized invariant above satisfied on **every** storage;
2. keeps $|M| \leq \text{max_concurrent_migrations}$;
3. keeps the count of in-flight moves touching any single storage — **as either source or target** — at or below `max_concurrent_per_storage`;
4. respects the saturation check of §7.3, which sums $\omega_{\text{role}}(m, s)$ over **every** move still in `M` at that storage — including moves in draining, whose source is charged ω_{wipe} and whose target is charged

nothing;

- violates no per-disk or per-storage cooldown.

With the default `max_concurrent_per_storage: 1`, two moves targeting the same storage serialize automatically and the generalized form collapses to the single-move form. That is the recommended configuration, and raising it should be a deliberate act on a storage with ample headroom.

8.2 Scheduling algorithm

```

pending ← { moves implied by the target assignment }
state   ← current allocation
order   ← []

while pending:
    feasible ← { m ∈ pending : transient_invariant_ok(state, m)
                                   and concurrency_ok(state, m)
                                   and not in cooldown }

    if feasible is empty:
        if a staging move exists: order.append(staging_move); continue
        else: report deadlock with the blocking storages; break

    m ← argmax over feasible of (persistent-objective reduction) / cost_m
                                   # the α, δ and κ·w terms of §5.4 – the parts whose
                                   # improvement persists; β/γ are one-time costs.
                                   # cost_m = 0 (a tiny disk, §7.1) ranks first:
                                   # free value, delivered before anything pays

    order.append(m)
    state ← apply(state, m)          # target charged immediately; the source is charged
                                   # until its volume is observed gone (see below)

```

Ordering by **objective reduction per unit cost** means the plan front-loads its value: if the operator aborts halfway, or a maintenance window closes, the moves that mattered most have already run. Two exceptions take priority and are scheduled first regardless of ratio:

- moves that resolve a storage currently violating (C5);
- moves that *free* space on a storage which some later move needs.

The source is not freed when the task succeeds. `apply(state, m)` must not optimistically credit the source with `z_d` bytes back. With `saferemove` on the source storage the old volume still exists — fully allocated — for the whole duration of the zeroing pass (§7.1), which can be far longer than the move itself. A move therefore leaves the in-flight set M in two stages:

```

mirroring → draining → done

mirroring: target charged z_d, source still charged z_d  (the drive-mirror window of §8.1)
draining:  target charged z_d, source still charged z_d  (the move_disk task has reported OK,
                                                         but the source volume is still there)
done:      source volume absent from /storage/{a}/content

```

For the generalized transient invariant of §8.1 the *source-side* accounting of mirroring and draining is identical, so keep a move in M until it reaches done and the invariant needs no change at all. The *load* charge is not identical across the two states, and §7.3's $\omega_{\text{role}}(m, s)$ table is what distinguishes them: a draining move charges ω_{wipe} to its source and nothing to its target. Keeping the move in M is therefore what makes the wipe visible to the saturation guard as well as to the capacity check — which is the point, since the wipe is by far the longer of the two windows on a large disk. What does change is that ordering rule 2 — “moves that free space a later move needs” — cannot be satisfied within a run when the source wipes slowly. The scheduler must therefore treat a predicted free-space release as **unrealised until**

observed, and a plan whose feasibility depends on one is split rather than executed on faith (§8.3, option 2).

8.3 Deadlock and staging

Three storages each near capacity can produce a cycle in which no move is individually feasible though the target assignment is perfectly valid — the classic pebble-motion deadlock. Resolution, in order of preference:

1. **Staging move** — find any storage in the group with enough slack to temporarily accept the smallest disk in the cycle, move it there, complete the cycle, then move it to its target. Costs one extra migration; permit it only if the *whole* plan still passes the payback test with that extra cost included.
2. **Split the plan** — execute the feasible subset now, re-plan on the next run. Often the workload has changed anyway.
3. **Report** — emit the blocking set and stop. Never force a move that breaches the reserve.

Guard staging moves against loops: a disk may be staged at most once per plan, and a staged disk is exempt from the per-disk cooldown for the duration of that plan only.

9. Execution

9.1 Modes

Mode	Behaviour
dry-run (<i>default</i>)	Print the plan, the before/after balance, the payback arithmetic, and the exact API calls that <i>would</i> be issued. Change nothing.
confirm	Execute the plan one move at a time, prompting before each. Re-validate the transient invariant immediately before each prompt, since the cluster may have changed. Offer [y]es / [n]o skip / [a]ll remaining / [q]uit.
auto	Execute unattended, subject to concurrency caps and <code>execution.time_windows</code> .

auto must additionally: refuse to start a move that cannot finish inside the remaining time window; stop cleanly at window close rather than aborting an in-flight move; and honour `max_migrations_per_run`.

9.2 Performing one move

```
POST /nodes/{node}/qemu/{vmid}/move_disk
  disk={device} storage={target} delete=1 bwlimit={KiB/s}
  → UPID
poll GET /nodes/{node}/tasks/{upid}/status every execution.poll_interval_seconds
  until status == "stopped"; task success ⇔ exitstatus == "OK"
```

Task success is **not** the completion criterion — see §9.3. The move is done only once the source volume has actually disappeared and the VM's lock has cleared.

`bwlimit` is **KiB/s** in the API, while `migration.bwlimit_bytes_per_sec` is bytes/s; convert at the call site and nowhere else.

With `max_concurrent_migrations > 1` this loop launches up to the cap (subject to `concurrency_ok`, §8.1) and polls **all** in-flight UPIDs each cycle, starting the next queued move as each slot frees. With the default cap of 1 it degenerates to the sequential form above.

Before **every** move, re-read the live state rather than trusting the plan:

1. re-fetch `/nodes/{node}/qemu/{vmid}/config` and confirm the disk is still on the expected source — the VM may have been touched by an operator, or live-migrated to another node by the PVE 9.2 Dynamic Load Balancer (§1), changing `{node}`;
2. re-read the target storage live and re-check the transient invariant (§8.1) against *actual* current usage — in **provisioned** terms, like every other check (§5.1). `used` from `/nodes/{node}/storage/{target}/status` is the pool's *allocated* figure and is never an input: on a thin pool (Ceph RBD, LVM-thin, ZFS) it is far lower than what is provisioned, and a re-check built on it could only ever confirm the plan, never catch a pool that other provisioning has filled *in provisioned terms* since planning. The live `used_b` is instead $\sum \text{size}(\text{vol}) : \text{vol} \in \text{GET } /nodes/{node}/storage/{target}/content$ — the same quantity `schedule.transient_invariant_ok()` sums from the model ($\sum z_d + U^{\text{sex}}$), read fresh — while `C_b` still comes from `/status`'s total, since the LUN may have been resized. The third model input, `Z_b`, is **not** re-read: it stays the planning-time largest managed disk on the target, raised only by this run's own completed moves, because a listing entry cannot say whether a volume belongs to a managed disk (§5.3 (C4) defines `Z_b` over managed disks). A managed disk that something else lands on the target after planning is therefore counted in full in `used_b` but does not raise `Z_b`, so only the reserve multiplier's growth on the new largest disk, $f_b \cdot (Z_{\text{live}} - Z_{\text{model}})$, goes uncounted — never the disk's own bytes (AJ-03). A listing entry without size counts at its approximate-size; one with neither makes the figure unknowable, and the move is refused. Any failure to read either endpoint refuses the move too: the check never passes on a partial figure. Those two refusals are *errors*, not mismatches — see “Errors are not mismatches” below: they fail the run rather than re-plan. A target that is simply too full for the move is the ordinary mismatch, and is `replan_needed`.

No double counting under concurrency. With `max_concurrent_migrations > 1` the invariant sums a charge `z_m` for every move of this run already in flight onto the same target (§8.1), so those moves' own mirror targets must not also be counted from the listing — a new RBD image is listed at its full provisioned size the moment `move_disk` allocates it. The executor therefore records, when it launches a move, which volumes the target's listing held at that instant, and leaves out of the sum at most one volume per in-flight move: one that is *not* in that launch-time listing, belongs to the *same* VM, and has one of the two sizes `move_disk` allocates it at. As the operator describes PVE's behaviour (not read from PVE's source): between storages of the same thin kind the target is the same size as the source image (the size in its content listing); between different storage types, or from thin to thick, the target is allocated at the disk line's `size=` in the VM config. Either figure can differ from the other (a volume resized outside PVE, a storage that rounds sizes), so the pre-flight's parsed `size=` is kept alongside the listed size and a volume of either size matches. If PVE ever allocates a target at some third size, nothing matches, the target stays counted as well as charged, and the check is merely stricter than it needs to be. The same two sizes decide what the move itself is charged as `z_m`: the disk's listed size, except when the move changes the kind of volume made — between different storage types (`storage.type` differs), or a qcow2 disk landing on a target that cannot hold qcow2 so PVE writes it raw — where the target is allocated at the config's `size=` and the larger of the two is charged. A same-kind move keeps the listed size. (The tool never passes `format=` and (C2) keeps a qcow2 disk off a storage that cannot hold it, so the conversion branch is a guard; the plan's own §8.1 check keeps `z_d`, the listed size.) Nothing else is ever excluded — a foreign volume that appeared since, or a leftover of the same VM that was already there, still counts — and where nothing matches (the window between `move_disk` returning and the allocation) the move is charged by its `z_m` alone. Wrongly keeping a volume only makes the check stricter; wrongly dropping one would weaken it, so the match is deliberately narrow. The sequential executor has no in-flight set and excludes nothing;

3. confirm the VM is still running and untagged for exclusion;
4. confirm `config.lock` is empty — if not, wait per §9.3 rather than failing;
5. confirm no snapshot has appeared for the VM since planning (§3.7); if one has, drop the move and re-plan — `delete=1` would be rejected by PVE anyway;
6. re-fetch `/nodes/{node}/qemu/{vmid}/pending` and confirm this disk still carries no unapplied pending change (§3.8); if one has appeared since planning — an operator can queue one in the PVE UI at any time — drop the move and re-plan rather than risk `move_disk` leaving that entry referring to pre-move state.

These re-reads bypass the per-run topology cache (§3.5) for this VM and this storage only. Steps 4 and 5 are cheap: both come from the same `/qemu/{vmid}/config` response as step 1. Step 6 is its own call, since only `/pending` (not `/config`) distinguishes a key's pending value from the one in effect (§3.8).

Re-plan protocol. A mismatch is a *normal* outcome in a live cluster, not an error, and must not be allowed to loop:

1. Abandon the remaining moves in the current plan. Do not attempt to patch it — the state it was computed against no longer holds.
2. Record the moves already completed in `state.json` (including their cooldown timestamps) so the next pass sees them as history rather than re-deriving them.
3. Re-invoke the **whole** pipeline from the new observed state: gates, load model, solver, payback, ordering. The gates may well conclude no further action is needed, which is a correct outcome.
4. Cap re-plans at `execution.max_replans_per_run` (default 3). On exceeding it, stop and report — a cluster churning faster than the engine can plan is a condition for a human to look at, not to iterate against. This is a *bail-out*, not a failure: the run exits 0, the report and the `run_summary` say why it stopped, and the next run starts from freshly observed state. External changes — a VM moved, a snapshot or a new disk appearing, another tool filling the target — are exactly what this loop is for, so a run is only given up on when they keep breaking every plan it makes.
5. In auto mode, a re-plan inherits the remaining time window; if too little remains for the cheapest queued move, stop cleanly rather than starting one that cannot finish.

Errors are not mismatches. A mismatch is the cluster differing from the plan, and a re-plan can cure it. An *error* is the tool failing to observe the cluster at all, and re-planning would only run into the same wall — a plan built from data that could not be read is not a better plan. Errors therefore **fail the whole run**: it stops at once (no later group is planned or executed), exits 1, and reports what it could not read; moves already completed stay recorded in `state.json` and in the report. The errors are:

- the PVE API failing while the executor re-reads the VM's config or the target storage's status or content before a move, or the target listing a volume whose size cannot be established (steps 1 and 2 of the pre-move re-reads above) — the move outcome is failed, marked `abort_run`, never `replan_needed`;
- a metrics (Prometheus) error while computing a group's load, whether in the first plan or in a re-plan (re-plan protocol step 3) — a re-plan whose load model is unavailable is not “the gates concluded no action is needed”; it is a failed run.

`plan` and `explain` are read-only and continue through the remaining groups so the report is as complete as possible, but they too exit 1 when any group's load could not be computed. Every other PVE API error raised during a run already propagates and exits 1.

9.3 Locks, and why a completed move is not a finished move

Two distinct mechanisms can make a VM untouchable, and the engine must handle both. Neither is an error condition — both are ordinary states in a working cluster — so the response to both is to **wait**, not to fail.

1. The VM config lock. PVE writes a lock: line into the VM config for any operation that must not be interrupted. Its documented values are backup, clone, create, migrate, rollback, snapshot, snapshot-delete, suspending and suspended. Any of them makes `move_disk` fail with “VM is locked”.

```
before issuing move_disk for VM v:
  lock ← config(v).lock                # /qemu/{vmid}/config, or /status/current
  while lock is set:
    if waited > execution.locks.wait_timeout: → apply execution.locks.on_timeout
    sleep execution.locks.poll_interval
    re-read lock
```

Treat the value set as **open-ended**. Never whitelist “harmless” locks and proceed anyway: a future PVE version may add a value, and the failure mode of guessing wrong is a half-completed operation on someone else’s backup. Any non-empty lock means wait. Log which lock was seen and for how long — a VM stuck in backup for six hours is something the operator wants to know about, and it is the main reason the wait timeout is measured in hours rather than minutes.

The pre-flight check is also a *planning*-time input: a VM locked when the run starts is pinned by (C2) for that run, so the solver does not build a plan around a disk it cannot touch. The check here is the second line of defence, because a lock can appear between planning and execution.

2. The post-move wipe — the one that is easy to miss. When `move_disk delete=1` completes and the task reports `exitstatus: OK`, the migration is *not* over on a storage with `saferemove` enabled. PVE then zeroes the old volume at `saferemove_throughput` (default **10 MiB/s**, §7.1), which for a large disk runs for hours or days. During that window:

- the source volume still exists and its space is **not** reclaimed — §8.2’s draining state;
- the operation holds a storage-level lock, so other volume operations on that storage queue behind it, and a subsequent `move_disk` touching the same VM or storage can fail even though *our* move finished cleanly;
- the task list shows nothing running, so a naive “poll until the UPID stops” loop concludes the move is done and immediately issues the next one — straight into the lock.

This is why the executor’s completion criterion is deliberately stronger than task success:

```
move m from a to b is DONE ⇔ task(upid) exitstatus == OK
    ∧ volume(m) absent from GET /storage/{a}/content
    ∧ config(vmid(m)).lock is empty
    ∧ elapsed(drain) ≥ min_wipe_seconds(m)
```

`min_wipe_seconds(m)` is `size_bytes(m) / |storage.saferemove_throughput|` — PVE’s own configured wipe rate, already read live off the storage definition (§3.5), magnitude taken because the sign selects `cstream`’s throttling mode rather than the rate (§7.1) — or, when that throughput is not configured (the storage type has no such concept, e.g. Ceph RBD or ZFS, or `saferemove` is off there), simply absent from the criterion (the first three conditions alone still govern, exactly as before this term existed). **Found dogfooding against a real cluster: the first three conditions alone are not sufficient.** VM 101’s `scsi1` move completed, its source volume was gone from the content listing, and its config lock read empty — every one of the first three conditions held — yet the very next move for the *same* VM (`efidisk0`) still had its `move_disk` task fail immediately with PVE’s own

```
can't lock file '/var/lock/qemu-server/lock-101.conf' - got timeout
```

PVE's own wipe cleanup for the first move was still finishing at the OS level even though the API-visible signals this criterion polls had already cleared. `min_wipe_seconds` closes this in the one case it can be computed exactly, from the same formula §7.1's planning-time cost estimate already uses — one implementation, two callers, not a second copy of it.

Poll all conditions at `execution.poll_interval_seconds`, bounded by `execution.source_release.timeout` (default **48h**, sized for a multi-TiB wipe at 10 MiB/s). On timeout, do not fail the run: mark the storage draining, exclude it as both source and target for the remainder of the run, report it, and let the next run re-evaluate from observed reality. Set `execution.source_release.wait: false` only on storages verified not to wipe — with `saferemove` off, the volume disappears immediately and this condition costs one extra API call.

3. The `move_disk` task's own flock — a residual race `min_wipe_seconds` cannot always close. The `lock: config` attribute in **1** and the file `move_disk` itself must flock to start (`/var/lock/qemu-server/lock-<vmid>.conf`) are not the same thing, and `min_wipe_seconds` above is only as good as the configured throughput it is computed from — no throughput configured, an inaccurate one, or the same “can't lock file... got timeout” `exitstatus` surfacing for some other momentary reason entirely, and the race in **2** can still reach `move_disk` itself. `_wait_for_unlocked()` cannot catch it either way — there is nothing to poll that shows it in advance, since PVE clears the `lock: line` before releasing the flock. As a narrow, mechanical safety net under **2**'s proactive fix, the executor retries the `move_disk` task itself when its `exitstatus` matches this exact message: up to `execution.locks.task_retry_limit` extra attempts (default **2**), each preceded by `execution.locks.task_retry_backoff` (default **15s**). A retry that still exhausts the limit is reported “failed” exactly like any other task failure — this is a fallback for the residual race, not a substitute for **1**'s open-ended wait or **2**'s computed floor.

Sizing the knobs against each other. With `saferemove` at its default throughput, one move can occupy its source storage for far longer than a whole planning cycle. Two consequences the implementer should not have to rediscover:

- `gates.cooldown_per_storage` must exceed the expected wipe time for that storage's largest disk, or the next run will plan moves onto a storage that is still draining and stall in 9.3's wait loop. `pve-storage-drs verify-storages` computes `z_max / saferemove_throughput` per storage and warns when the configured cooldown is shorter, or when it exceeds `migration.max_single_move_duration`.
- Keep `execution.max_concurrent_per_storage: 1`. Two moves off the same source mean two concurrent wipes sharing one throttle, so both take twice as long while both sources stay fully allocated.

9.4 Failure handling

`move_disk` is atomic from the caller's perspective: on failure QEMU cancels the mirror and the source volume remains authoritative, so there is nothing to roll back. The realistic hazards are:

- **Orphaned target volume.** A failed or cancelled mirror can leave `vm-101-disk-0` on the target. After any failure, list `/nodes/{node}/storage/{target}/content` and warn about volumes owned by the VM that are not referenced in its config. Do **not** delete them automatically — report them. They count against the reserve until removed, so surfacing them matters.
- **Partial plan.** With `abort_on_failure: true` (default), stop and report; the cluster is in a valid intermediate state and the next run will re-plan from reality.
- **Task supervision loss.** If the engine dies mid-move, the PVE task continues. On startup, check for running `move_disk` UPIDs owned by the DRS user before planning anything.

9.5 Output

Every mode emits the same machine-readable plan (JSON) plus a human summary. The example below is the section 14 fixture at `beta_move_count: 0.5` (the two-move variant):

```
Group fc-tier1 - imbalance 255% (threshold 20%) → ACT
  san-a u=6.50 ██████████ used 4.5/8.0 TiB ▲ reserve short by 0.5 TiB
  san-b u=0.70 █ used 1.5/8.0 TiB
  san-c u=0.20 █ used 0.5/8.0 TiB

  1. 102:scsi0 san-a → san-c 1.5 TiB ~2.2h Δimbalance -4.53 ℓ/z 1.67
  2. 101:scsi1 san-a → san-b 1.0 TiB ~1.5h Δimbalance -2.00 ℓ/z 1.00

  after: san-a u=3.00 san-b u=1.70 san-c u=2.70 spread 53% (from 255%)
  payback: benefit 1.93e8 load·s vs cost 2.62e4 load·s → ratio 7344 (need 10) ✓
           plan repairs san-a's reserve (Σ r_s 0.5 → 0) - $7.3-exempt; both moves
           marked repair: false (either alone repairs - redundant repairs, $7.3)

  pinned (not movable this run):
    106 snapshots present (2) 1.0 TiB ℓ 0.9 on san-a → clear snapshots to unblock
    107 locked: backup 0.5 TiB ℓ 0.3 on san-b → waited 0s, re-check next run
    109 excluded by tag no-drs 0.2 TiB ℓ 0.0 on san-c
  pinned load 1.2 of 8.6 (14%, warn at 25%); best achievable spread given pins: 44.6%
```

As built (phase 13): the exemption note under the payback line and the per-move [repair] markers are printed by plan/apply, and the repair field reaches `--json` per move — computed by §7.3's revert test, not by the old “source presently violating” flag, which now survives only as `order_moves()`'s scheduling signal (§8.2 priority 1).

The **exemption itself needed a field of its own**, and phase 13 added it. `--json`'s payback block carried `benefit_load_seconds`, `total_cost_load_seconds`, `ratio`, `aggregate_ok`, `rejected_moves`, `deferred_moves` and `accepted` — none of which says *why* the aggregate test passed, so a plan accepted on merit and a plan accepted only because it repairs are the same object to a machine. (A consumer can re-derive it as `aggregate_ok` and `ratio < λ`, but only by supplying a `payback_ratio` the block does not carry, and re-deriving a verdict the tool already reached is not a specification.) The per-move repair markers do not close it either: §7.3's redundant-repair plan is exempt with no marked move, and §14.8 turns on exactly the distinction a bare accepted: `true` erases. Phase 13 therefore emits `repair_exempt` (the trigger's own verdict) alongside the pair it is computed from, `reserve_shortfall_bytes_before/_after` — the two Σr_s sums the trigger already has in hand at the call site. That pair is also the first instalment on §16.6's X-07/Y-04 gap: with it, check 4's “payback arithmetic” per variant records a verdict a regression diff can read, and check 2's Σr_s invariant (`final ≤ current`) becomes checkable from `plan --json` without the emitted order.

The pinned block is not optional decoration — it is the “complain” half of the skip-and-complain policy of §3.7, and it is the only place an operator learns which snapshots to clear.

As built (REVIEW.md V-02): `plan/apply` print none of the above beyond the move list and payback verdict — neither renderer carries a pinned field, a fragmentation line, or a pinned-load line. That narration lives in `pve-storage-drs explain` instead (`docs/manual/29-explain.md`'s pinned (not movable this run):/cannot fully consolidate:/pinned load ... sections, including the per-pin → action hints shown above); `show-load` also names each pin inline, per disk, as `[pinned: <reason>]`. Read every “pinned block”/“plan output” reference above as `explain`'s output, not `plan`'s or `apply`'s. There is likewise no dedicated data: fill-deviation line in any renderer's human output (an earlier revision of this example showed one, REVIEW.md AA-06): the fill deviation reaches a human only via `explain`'s own objective: line (its spread term, shown further down this section) and `plan/apply/explain --json`'s `before_capacity_spread/after_capacity_spread` scalars — the same asymmetry `show-load`'s spread-only human line

already has relative to its own --json.

As built, added later: none of the above covers the case where the gate decides to ACT and every disk is eligible (nothing pinned), yet the solver’s own optimum is still to move nothing – found dogfooding against a real cluster, where a two-disk VM sitting entirely on the busier storage was the only lever, and objective.kappa_vm_affinity’s fragmentation penalty for splitting it legitimately outweighed the imbalance it would fix. plan’s one-line verdict gives no way to tell that apart from “the solver didn’t try” from the outside. explain now reports it: when decision.act is true and the final assignment moves nothing, it evaluates every single-disk move (heuristic.best_single_disk_alternative() – every movable disk against every other group storage, via the same evaluate_assignment() the solver itself uses, section 5.3’s one-move neighbourhood) and names the one closest to being worth it, with the section 5.4 term-by-term arithmetic that rejected it:

```
objective: imbalance 0.576 + moves 0 + bytes 0 + fragmentation 0 + spread 0 + reserve 0 = 0.576
no moves made: the objective is lowest at the current assignment
closest alternative: 110:scsil VM-krbd → VM
    imbalance 0.576→0.0426, moves 0→0.25, bytes 0→0.00732, fragmentation 0→0.5, spread 0→0, reserve 0→0
    total 0.576 → 0.8 (worse by 0.223 -- rejected)
```

--json carries the same information as rejected_alternative (null unless this case applies), alongside disk_key/vmid/device/from_storage/to_storage, baseline and objective (each the six-term breakdown objective above already serializes), and worse_by. plan/apply still print none of it, for the same reason as everything else in this section (V-02, above) – it is narration for a human, not a machine-checked verdict.

10. Forecasting

The default decision statistic is the p95 of the trailing window, which is deliberately conservative and needs no model fitting. Forecasting is behind an interface so it can be strengthened without touching the optimizer:

```
class Forecaster(Protocol):
    def required_range(self) -> timedelta:
        """How much history this model needs. metrics.py serves exactly this."""

    def predict(self, series: TimeSeries, horizon: timedelta) -> Forecast:
        """Returns point estimate and an upper bound for the horizon."""
```

10.1 Each forecaster owns its data range

window.lookback is the **decision** window – the period whose load we are balancing. It is *not* the amount of history a forecaster needs, and conflating the two makes the seasonal models unreachable: Holt-Winters with seasonal_periods = 288 (24 h at a 5 m step) needs $2 \times 288 = 576$ samples, i.e. **48 h**, which a 24 h window can never supply. It would silently fall back to quantile forever.

So metrics.py must expose query_range over an **arbitrary** range, not just window.lookback, and each forecaster declares what it needs:

Implementation	required_range()	Point estimate	Upper bound
quantile (<i>default</i>)	window.lookback (24 h)	p95 over W	quantile_over_time(upper_quantile), default p99
seasonal_naive	max(lookback, seasonal_lookback_days) (7 d)	median across same-hour-of-day samples	p95 across those samples

Implementation	required_range()	Point estimate	Upper bound
holt_winters	$\max(\text{lookback}, 2 \cdot \text{seasonal_periods} \cdot \text{step})$ (48 h)	fitted forecast at horizon	point + $z \cdot \sigma$ of in-sample residuals, $z = 2$

The optimizer consumes the **upper bound**, never the point estimate. Being wrong in the direction of “this disk is busier than it looks” costs a slightly suboptimal balance; being wrong the other way migrates a disk onto a storage that is about to be saturated. For the default quantile forecaster this makes the distinction concrete rather than vacuous: the point estimate is `window.quantile(p95)` and the bound is `window.upper_quantile(p99)`, so the optimizer sees p99.

As built (REVIEW.md T-03): the decision statistic that actually drives the gates, solver, payback and ordering is `window.quantile` (the point estimate), computed once per group by `loadmodel.compute_group_load()`. The upper bound described above is real and exercised, but its only consumer is §7.3’s saturation-ceiling guard, and only for a storage that configures `saturation_load` — the guard calls a Forecaster (built here, gated by the §10.2 backtest below) to get $\hat{L}_s(\Delta)$. Wiring the upper bound into the optimizer’s own input, as this section describes, remains future work — **phase 14** (§12), tracked as REVIEW.md AL-01, records the design and its one deliberate default change; until then, treat every occurrence of “the optimizer consumes the upper bound” in this document as the target design, not the current behaviour.

Forecasts are produced **per disk**. Where §7.3 needs a per-storage bound $\hat{L}_s(\Delta)$, it is the sum of the per-disk upper bounds over the disks assigned to s in the state being evaluated: $\hat{L}_s(\Delta) = \sum_{\{d : x_{\{d,s\}} = 1\}} \hat{u}_d(\Delta)$. Summing upper bounds is conservative — it assumes the disks peak together — which is the right direction for a guard whose failure mode is starting a mirror onto an already-busy array.

Config validation (§11.1) must **reject** a configuration whose Prometheus retention or whose selected forecaster and window are mutually inconsistent, rather than silently degrading. Enabling a seasonal model is a statement that the history exists to support it.

10.2 Implementation warnings

- **Do not compute Holt-Winters in PromQL.** Prometheus’s `holt_winters` was renamed `double_exponential_smoothing` in Prometheus 3.x and requires `--enable-feature=promql-experimental-functions`. More importantly, despite the historical name it is **double** exponential smoothing — level and trend only, with **no seasonal component** — so it cannot learn a daily cycle. Seasonal forecasting must happen engine-side on data pulled via `query_range`.
- Require at least $2 \times \text{seasonal_periods}$ samples before trusting a Holt-Winters fit, and fall back to quantile otherwise — with a **logged warning**, since a silent fallback hides a misconfiguration.
- Validate by backtesting: fit on $[t-2T, t-T]$, predict $[t-T, t]$, compare against actual. Refuse to let a model whose backtest error exceeds the imbalance threshold drive migrations.

As built (bug fix): the live §7.3 saturation-guard fetch (`cli.py’s_saturation_forecast_inputs()`, via `loadmodel.compute_disk_load_series()`) used to issue one **unchunked** `query_range` over its whole computed range — $2 \cdot \text{window.lookback}$ from §10.2’s own backtest-gate floor above, combined with a fine `metrics.step`, confirmed live to exceed a VictoriaMetrics/gigapipe backend’s own max-points-per-timeseries limit (11,000 by default) with a 500 “exceeded maximum resolution” error, something §16.2’s `collect-testdata` capture path was already immune to. The live fetch (`loadmodel._fetch_raw_quantity_series()`, via the new `_issue_chunked_range_query()`) is now chunked exactly like §16.2’s capture path — day-sized sub-queries (`metrics.RANGE_QUERY_CHUNK_SECONDS`), boundaries falling on the range’s own start, never on wall-clock “now”, stitched back into one series (`metrics.stitch_range_results()`), factored out of `collect.py’s_own_stitch_range_captures` so both paths share one merge im-

plementation) — every plan/apply run with a backtested forecaster active, not just collect-testdata, so neither path depends on Prometheus retention or `metrics.step` staying small enough to fit one request.

11. Configuration

Where it lives: `/etc/pve/drs.yaml`. That path is on `pmxcfs`, the cluster filesystem, so the file is replicated to every node automatically: one edit, one config, no per-node drift, and no question about which copy is authoritative. It is also an ordinary path, so a management host that is not a cluster member can simply provide it at the same location and every command, example and manpage stays true on both kinds of host.

Resolution order, first match wins:

#	Source	On failure
1	<code>--config PATH (-c)</code>	Error and exit non-zero
2	<code>\$PVE_STORAGE_DRS_CONFIG</code>	Error and exit non-zero
3	<code>/etc/pve/drs.yaml</code>	Error naming the path and pointing at the shipped example

An **explicitly requested** config that is missing, unreadable or invalid is a hard failure. Never fall through to the default: a run that silently balanced a production cluster from a different file than the operator named is the worst outcome in this document. Every run logs the resolved path and the SHA-256 of the file it actually read, in the first line of output and in the JSON report.

Three consequences of living on `pmxcfs`, all of which the implementation must respect:

1. **An edit is cluster-wide and immediate.** There is no staging step and no per-node rollout. A typo is live everywhere the moment it is saved, which is why §11.1’s validation is a hard failure rather than a warning and why dry-run is the default mode.
2. **Do not put the state file there.** `state.json` is written on every run; `pmxcfs` is a small, quorum-gated, cluster-replicated store meant for configuration. Reads do not need quorum, so a node that has lost quorum can still read its config and plan, but writes do — keep `state.path` on local disk (§11.2).
3. **Treat the file as readable by the web server.** Files under `/etc/pve` carry group ownership `www-data` by default, which puts a plaintext password: within reach of anything running as the PVE web server. Prefer an API token restricted to the privileges of §3.5, and keep the secret out of the file entirely using `PVE_PASSWORD / PVE_TOKEN_SECRET` from the environment or a systemd credential. **Verify the ownership and mode on the target cluster** (`ls -l /etc/pve/drs.yaml`) before storing any secret in it — this document does not assume it.

A shared config does not make the tool cluster-aware. The obvious next step after putting the config on `pmxcfs` is to enable the systemd timer on every node, and that is wrong: `state.json` is node-local, so each node keeps its own lock, its own cooldowns and its own drift baseline, and the `fcntl` lock of §11.2 cannot see the other nodes at all. What does cross the cluster is the startup scan for in-flight `move_disk` UPIDs owned by the DRS user (§13) — it is the only reason two concurrent instances degrade to “slow and redundant” instead of “conflicting”. **Run the timer on exactly one host.**

See [config/drs.example.yaml](#) for the fully annotated reference. The requirement-to-setting mapping:

Requirement	Setting
Storage groups VMs may not leave	<code>groups[].storages[]</code> — literal ids or <code>/regex/</code> patterns (§11.4)

Requirement	Setting
2× largest disk free for snapshots	snapshot_reserve.factor (default 2.0), per-storage override
Keep N bytes / N% of each storage free	free_space.soft — global, per-storage or per-pattern (§5.3.1); snapshot_reserve.min_free_bytes is deprecated syntax for it
Min % changed traffic before migrating	gates.drift_threshold (default 0.10)
% I/O difference across the group	gates.imbalance_threshold
Timeframe considered	window.lookback (default 24h)
Minimal number of migrations	objective.beta_move_count
Keep a VM's disks together	objective.kappa_vm_affinity
Spread data evenly across storages (failure risk)	objective.delta_capacity_spread, gates.capacity_spread_threshold
Migration load accounted for	migration.*, objective.gamma_move_bytes_per_tib
Manual vs automatic	execution.mode
Forecasting	forecast.model

The config carries `schema_version: 1` at the top level. `config.py` rejects an unknown **major** version with an explicit message naming the version it understands, so a future incompatible change has a migration path instead of misinterpreting fields.

11.1 Validation rules

Validate with `jsonschema` for structure, then apply these semantic rules. Every one of them is a failure that produces a clear error and a non-zero exit, never a warning-and-continue — a misconfigured balancer moving production disks is worse than one that refuses to start.

Rule	Rationale
<code>schema_version</code> major matches	Forward compatibility
Every group non-empty, ≥ 2 storages after pattern expansion (§11.4)	A one-storage group has nothing to balance; with patterns the count that matters is storages matched, not entries written
No storage appears in two groups , after pattern expansion	A disk's group would be ambiguous; a pattern matching a storage another group also names is the same defect
Storage ids exist in the cluster; every <code>/.../</code> pattern matches at least one storage (§11.4)	Catches typos before they silently exclude disks or silently shrink a group
Every <code>/.../</code> pattern compiles as a Python regular expression, checked at load time	A malformed pattern must fail with the compiler's own message, not crash at match time (§11.4)
Within a group, no storage is matched by two pattern entries	Which entry's options apply would be arbitrary; a literal entry overriding a pattern is allowed and is not this error (§11.4)
<code>capability_weight > 0</code>	Appears in a denominator
<code>reserve_factor ≥ 0, <code>min_free_bytes</code> ≥ 0</code>	Negative reserve is meaningless; <code>min_free_bytes</code> is accepted as deprecated <code>free_space.soft</code> syntax (§5.3.1)
<code>free_space.soft/hard</code> : absolute values ≥ 0 and parseable (bytes or byte-unit string); percentages "N%" with $0 \leq N < 100$; <code>hard</code> $\leq$ <code>soft</code> after per-storage resolution and percent-to-bytes conversion, and before the deprecated <code>min_free_bytes</code> fold	§5.3.1. A hard above soft makes every plan for a compliant storage infeasible; a percentage of 100 or more is a typo, not a policy. Before the fold, because the fold can only raise <code>soft_s</code> : checked after it, a written <code>hard > soft</code> would hide behind a large <code>min_free_bytes</code> and surface as a startup failure the moment the operator deletes the deprecated key (§5.3.1, "validate as written, then fold")
<code>free_space.soft < C_s</code> for every storage, after resolution — the <code>free_space</code> value's own, not the folded <code>min_free_bytes</code> (see the resolution rules below)	A requirement no disk could leave room for is a typo; caught only once the inventory is loaded, like the pattern rules of §11.4
$0 \leq \text{drift_threshold} \leq 1, 0 \leq \text{imbalance_threshold} \leq 1$	They are ratios
<code>quantile</code> $\in (0,1)$, <code>upper_quantile</code> $\in (0,1)$, <code>upper_quantile</code> $\geq$ <code>quantile</code>	The bound must not sit below the point estimate
<code>min_coverage</code> $\in (0,1]$	A ratio; 0 would accept a disk with no data
Metric names non-empty; label names non-empty and pairwise distinct	A duplicated label name silently collapses series

Rule	Rationale
<code>rate_window ≥ 4 × metrics.pvestatd_push_interval</code>	Below this, <code>rate()</code> sees too few points. The interval is a PVE-side setting the tool cannot read, so it is declared in config (default 60s, PVE's own default) and <code>verify-metrics</code> cross-checks it against the observed sample spacing of a live series, erroring if the two disagree by more than 20%
<code>window.lookback ≥ forecaster.required_range()</code>	See §10.1 — otherwise the model can never run
<code>payback_ratio > 0, payback_horizon > 0</code>	Zero disables the safety test
<code>payback_horizon ≥ 30d (warn, not error)</code>	A horizon of days rejects slow-accruing but real benefits; it should approximate VM lifetime, not operator patience (§7.2)
<code>tiny_disk_bytes ≥ 0</code>	The size below which a disk moves free of β , γ and the payback test (§5.4, §7); 0 restores the old accounting
<code>delta_capacity_spread ≥ 0; warn when > alpha_spread</code>	A negative weight would reward concentration; above α , data evenness outweighs I/O evenness in every comparison and the tool is no longer an I/O balancer first
<code>capacity_spread_threshold > 0</code> where set, null disables	A ratio of fill fractions to the mean fill; it can legitimately exceed 1 (§14.2 measures 1.85)
<code>saturation_ceiling ∈ (0,1]</code>	A fraction of <code>saturation_load</code> , not of <code>capability_weight</code>
<code>saturation_load > 0</code> where set; warn once per run for each storage where it is unset	§7.3's guard is silently inactive without it
<code>max_concurrent_* ≥ 1</code>	Zero would deadlock the scheduler
<code>execution.locks.wait_timeout > 0, on_timeout ∈ {skip, abort}</code>	A zero timeout turns every ordinary backup window into a failed run
<code>execution.source_release.timeout ≥ z_max / saferemove_throughput</code> for every storage where <code>saferemove</code> is on	Otherwise every large move times out into draining (§9.3)
<code>gates.cooldown_per_storage ≥ z_max / saferemove_throughput (warn, not error)</code>	The next run would plan onto a still-draining storage
<code>report.warn_pinned_load_fraction ∈ (0,1]</code>	A ratio
Time windows: <code>start ≠ end</code> ; crossing midnight allowed and explicit	Ambiguity here silently disables auto
<code>execution.mode ∈ {dry-run, confirm, auto}</code>	Typo must not silently fall back to acting

Resolution rules (warn and continue). Everything above is a hard error. Two load-time situations are not errors but *resolutions* of a config that says two things at once, and they warn and continue by design — an implementer must not read them into the table above:

- `snapshot_reserve.min_free_bytes` and `free_space.soft` both set → `soft_s = max(soft_s_resolved, min_free_bytes)` per storage, after percent-to-bytes conversion, with a warning naming both keys and stating that the deprecated one applies as a lower bound on every storage. §5.3.1: both keys express the same quantity — a minimum-free floor — so the union is the only resolution that cannot silently lower a configured floor on upgrade; a “winner” rule is a guess. The `free_space` rows above that *are* errors stay errors.
- `min_free_bytes` above some storage's capacity → warn, do not error: the storage reports a permanent unfixable shortfall (§9.5), exactly as the built code treats it today. The `soft_s < C_s` error applies to the new knob's own value only (§5.3.1).

11.2 state.json

The only persistent state, at `state.path`, default `/var/lib/pve-storage-drs/state.json`. Local disk, one copy per host, deliberately **not** on `/etc/pve` for the reasons in §11. Small, versioned, and written atomically (temp file + `os.replace`):

```
{
  "schema_version": 1,
  "lock": { "pid": 12345, "host": "mgmt01", "acquired_at": "2026-09-04T02:00:00Z" },
  "last_balance": {
```

```

    "at": "2026-09-03T22:14:03Z",
    "load_vector": { "fc-tier1:101:scsi0": 3.0, "fc-tier1:102:scsi0": 2.5 }
  },
  "cooldowns": {
    "disk": { "fc-tier1:101:scsi1": "2026-09-03T22:41:55Z" },
    "storage": { "fc-tier1:san-b": "2026-09-03T22:41:55Z" }
  },
  "inflight_upids": [],
  "staged_disks": []
}

```

- Keys are "<group>:<vmid>:<device>" and "<group>:<storage>", so a vmid reused after a VM is destroyed and recreated in a different group cannot collide.
- last_balance is updated **only after a run that executed at least one migration** (§6).
- **Locking:** fcntl.LOCK_EX on the file for the duration of a run. If the lock is held but lock.pid is not alive on lock.host, reclaim it and log the reclamation — a killed run must not block the timer forever. A live PID means another instance is running: exit 0 quietly.
- inflight_upids is written *before* issuing each move_disk and cleared on completion, so a crashed run can be reconciled on the next start (§13).
- Losing this file is safe but not free: cooldowns and drift history reset, so the next run may migrate sooner than intended. Treat it as state to back up, not as a cache.

11.3 Global command-line options

Accepted before the subcommand, and shown by pve-storage-drs --help with their defaults:

Option	Default	Effect
-c, --config PATH	/etc/pve/drs.yaml	Read the configuration from PATH. Missing or invalid is a hard failure (§11)
--group NAME	all groups	Restrict the run to one group; repeatable. Groups are independent (§5), so this changes nothing about the result for the groups selected
--mode {dry-run,confirm,auto}	execution.mode	Override the execution mode for this run only
--json	off	Emit the machine-readable report of §9.5 instead of the human one
-v, --verbose	warnings only	-v adds this run's decision trail (INFO), -vv adds per-query detail and third-party library logs (DEBUG) — §2.3
--quiet	warnings only	Errors only. Note that on an auto run this discards the audit trail §2.3 otherwise logs unconditionally
--log-level {error,warning,info,debug}	—	Set the level explicitly; wins over -v/--quiet (§2.3)
--log-format {auto,text,json}	auto	auto = human text when stderr is a TTY, JSON otherwise (§2.3)
--version	—	Version, then exit
--manual	—	Show pve-storage-drs(1) (§8.5 of AGENTS.md)
--replay PATH	live cluster	Run against a collect-testdata bundle at PATH instead of the live cluster, with no network access at all — §16.5

Two rules the implementation must honour.

Every --mode override is logged, and an override toward less safety is a warning. Order the modes

dry-run < confirm < auto. Moving *down* that order — auto to confirm, confirm to dry-run — is logged at info: an operator being more careful than their config needs no ceremony. Moving *up* it is the operator deliberately removing a barrier they themselves configured, whether that barrier is the dry run or the per-step confirmation, so dry-run → confirm, dry-run → auto and confirm → auto all log at **warning** level, naming both values. In auto mode the log is the only record a human will see (§2.1), and “why did it move disks when the config said confirm” must be answerable from it.

No option may set a value that config.py would have rejected in the file. The command line goes through the same validation (§11.1), because a knob that is only checked on one of its two paths is a knob that is not checked.

11.4 Storage name patterns in groups[].storages[]

A storages[] entry may name storages by regular expression rather than by literal id: an id value that both begins and ends with / is a pattern, and the text between the slashes must compile as a Python re expression. /san-.* / is a pattern; san-a stays a literal id and behaves exactly as before. The form is additive — no schema_version bump — and it cannot collide with a literal one: PVE storage IDs never contain /, so a value wrapped in slashes can only have been meant as a pattern.

```
groups:
- name: fc-tier1
  storages:
  - id: /san-.* /          # pattern: every san-* LUN, present and future
    capability_weight: 1.0 # the entry's options apply to EVERY storage it matches
  - id: san-b              # literal: takes precedence over the pattern above
    capability_weight: 0.5 # the documented exception for one slower array
```

Four rules, each shaped so that the one new failure mode — a pattern that matches too much or nothing at all — stays checkable instead of silent:

- **Matching is re.fullmatch, case-sensitive.** The pattern must match the *entire* storage id. /prod/ names exactly the storage prod; under substring semantics it would also capture preprod, which is the classic too-greedy pattern and the reason whole-id matching is the rule. Python re syntax throughout: inline flags such as (?i) work, and there is no flag suffix after the closing slash.
- **The entry's options apply to every storage it matches.** A pattern entry accepts the same per-storage options as a literal one (capability_weight, reserve_factor, saturation_load, free_space), and every matched storage inherits them. This is the point of the feature: one entry weights or reserves a whole LUN family. A pattern-level free_space is resolved per matched storage — a "10%" demands a tenth of *each* LUN's own capacity (§5.3.1).
- **A literal entry beats a pattern.** Within one group, a storage named by a literal entry uses the literal's options even when a pattern also matches it — pattern as the default, literal as the exception, and deliberately not an error, because without this rule a catch-all pattern would make it impossible to single out one member of its own group. Two *patterns* matching the same storage in one group are a hard error (§11.1): which entry's options should apply would be arbitrary, and the config refuses to guess. Precedence is semantic, not positional — entry order never matters.
- **Across groups, disjointness is checked on the expanded set.** A storage matched — literally or by pattern — by entries of two different groups is a hard error naming the storage and both entries. A disk's group must stay unambiguous, exactly as §11.1 already demands for literals.

Expansion happens at run start, against the live cluster. Every pattern is matched once per run against the storage definitions (GET /storage, §3.5 — the same call that already validates a literal id's existence and content type, so pattern and literal ids are checked against one consistent source), in the same step that checks literal ids for existence, and the result is part of the per-run topology cache. A storage a pattern matches that has a definition but is reported active by no node (disabled) is also a hard error,

naming the pattern and the storage — the same fail-safe outcome a literal reference to it would hit, but with a message that does not require the operator to already know a pattern was involved. Which storages a pattern matches is a property of the cluster, not of the file: a LUN added to the cluster joins its group on the next run with no config edit — half the reason to use a pattern — and a pattern that matches nothing is handled exactly like a literal id that does not exist, a hard error before anything is planned, because a typo is the likelier cause and a silently shrunken group the likelier consequence. Nothing downstream ever sees a pattern: `S` (§5.1), the (C2) eligibility pass, cooldown keys and the `state.json` keys of §11.2 all carry real, expanded storage ids. Those keys embed the group name, so a later config change that moves a storage into a different group leaves its old cooldown keys as stale entries that simply stop matching — harmless. Group names themselves, `--group` (§11.3) and the `exclude.*` lists stay literal; only `storages[]` entries accept the `/.../` form.

Validation and visibility. `config.py` compiles every pattern at load time and reports a malformed one with the compiler’s own message; the zero-match and overlap rules run every run once the inventory is loaded, because they compare the file against the cluster. Because an over-broad pattern is the one new way to misconfigure a group, the expansion is always visible: every run logs entry `→ matched ids` at INFO, and `pve-storage-drs verify-storages` (§3.5) prints the expansion next to each storage’s properties and lists cluster storages matched by no group — the same “ungrouped, not managed” visibility (C2) gives `show-load`. One consequence to keep in mind when writing a pattern: a matched storage becomes a full member of the group, exactly as a literal entry, and `u*` (§5.3 C6) divides by every member’s `c_s` — so a pattern that captures an ISO-only or otherwise unusable LUN skews the balance target even though (C2) keeps any disk from ever landing there. Eligibility is unchanged and still per storage; safety never depends on the pattern being precise, but the quality of the balance does.

12. Implementation phases

Each phase is independently testable and useful on its own.

#	Phase	Done when
1	<code>config.py</code> , <code>metrics.py</code> , <code>pve-storage-drs verify-metrics</code>	Real metric/label names confirmed against the live Prometheus; per-disk load printed
2	<code>pve.py</code> , <code>topology.py</code>	<code>pve-storage-drs show-load</code> prints every storage with its disks (all buses), sizes, loads and reserve status; pinned disks flagged with their reason; <code>pve-storage-drs verify-storages</code> reports <code>saferemove</code> , implied wipe times and the expansion of every <code>/.../</code> storage pattern (§11.4)
3	<code>loadmodel.py</code> + gates	Correct act/no-act decision per group, with the reasoning shown
4	<code>heuristic.py</code> + <code>schedule.py</code>	End-to-end plan in dry-run, ordered and transient-feasible; reproduces <code>expected_order</code> and <code>expected_final_reserve</code> in the §14 fixture
5	<code>payback.py</code>	Plans rejected/trimmed on cost grounds, arithmetic shown
6	<code>optimize.py</code> (MILP)	Matches or beats the heuristic on the §14 fixture; CP-SAT and CBC agree on every β case, and the §5.5 coefficient assertions pass

#	Phase	Done when
7	execute.py	confirm mode against a lab cluster, including a VM locked mid-run and a source storage with saferemove on — the run must wait, not fail
8	auto mode + time windows	Unattended operation
9	forecast.py beyond p95	Seasonal-naïve validated by backtest
10	anonymize.py, collect.py, replay.py, tests/corpus/ (§16)	A bundle collected from a live cluster replays to the same plan the live run produced; the scrub audit and the determinism test pass on it
11	Logging policy (§2.3)	Done. A clean read-only run prints nothing on stderr; apply --mode auto logs the full §2.3 audit trail (gate, load, plan, payback, every UPID) without being asked; --log-format/--log-level behave as specified; the verification tests of §2.3 pass
12	Capacity-spread objective and gate, one-year payback horizon (§5.3 (C7), §5.4 6, §6, §7.2)	Done. Fixtures regenerated with the delta_values sweep and the 365d horizon; a replayed bundle shows the capacity gate deciding; explain reports the fill deviation; the manual documents objective.delta_capacity_spread, gates.capacity_spread_threshold and the new payback_horizon default (the manpage documents no individual knob, by §11's own established convention)

#	Phase	Done when
13	Free-space requirements (§5.3.1, §5.3 (C5), §6 override, §7.3 repair exemption, §8.1 hard floor) and the (C2) format-compatibility eligibility it needs	config_schema.json gains the block first — the schema is closed (additionalProperties: false throughout, deliberately: it is where a typo'd key is caught, §11.1's structural pass), so a free_space: key is rejected before config.py ever sees it: a top-level free_space object and a free_space property on groups[].storages[], each with soft/hard typed ["string", "number", "null"] for §5.3.1's grammar (integer bytes, byte-unit string, "N%", and null with its two by-level meanings); config.py then resolves free_space.soft/hard per storage (bytes, byte-unit strings, percentages; global, per-storage, per-pattern; the global snapshot_reserve.min_free_bytes scalar deprecated, folded in per storage after percent conversion as soft_s = max(soft_s_resolved, min_free_bytes) — §5.3.1), validates $\text{hard} \leq \text{soft}$ and $\text{soft} < C_s$ on the written values, before that fold (§5.3.1, “validate as written, then fold”); the per-storage soft_s/hard_s pair replaces the min_free_bytes scalar parameter across reserve.compute_reserve_status()/transient_charge_ok(), heuristic.run_heuristic() and its helpers, schedule.transient_invariant_ok()/order_moves(), optimize.py, execute.py's live execution-time re-check and every cli.py call site that threads the scalar today, and collect.py's bundle manifest (which serialises the scalar, so a replayed bundle carries the pair instead — §16); topology.Storage gains the type/format fields (C2) needs and both solver backends fix $x_{\{d,s\}}=0$ for format-incompatible targets; payback.py's repair detection (ScheduledMove.resolves_reserve_violation, set by schedule.py's “source presently violating” test) is replaced by §7.3's outcome trigger (exempt iff the plan's final Σr_s is strictly below the current assignment's) plus a per-move repair marker computed by the revert test — re-scoring Σr_s on the final assignment with one x held — while order_moves()'s internal priority-1 test keeps §8.2's “source currently violating” form (a current-state rule, not a plan-outcome one); the outcome trigger is a signature and data-flow change, not a flag swap: evaluate_plan_payback(move_costs, benefit_load_seconds, payback_ratio) has no access to Σr_s, so the current and final

#	Phase	Done when
14	Forecast-driven decision statistic (§10.1's target design; REVIEW.md T-03 and AL-01)	The per-disk statistic the gates, solver, payback and ordering consume is the backtest-gated forecaster's upper bound $\hat{u}_d(W)$, with $W = \text{window.lookback}$ — the same horizon §10.2's backtest already validates — computed per group by <code>loadmodel</code> from the per-disk series the saturation guard's own <code>compute_disk_load_series()</code> already fetches, gated by the same per-group backtest with the same quantile fallback (quantile itself needs no gate: it fits nothing); with the default <code>forecast.model: quantile</code> this is a deliberate default change — the decision statistic moves from <code>window.quantile (p95)</code> to <code>window.upper_quantile (p99)</code>, so gates, plans and payback all turn conservative by one quantile, and the §14 fixtures and every committed corpus <code>expected.json</code> are regenerated to match; a seasonal model drives placement only where it demonstrably predicted this group's own last window (within <code>gates.imbalance_threshold</code>), quantile otherwise with the existing warning; <code>explain</code> reports which statistic and which forecaster produced the run; the manual documents the default change and when a seasonal model is worth selecting — strong diurnal structure or trending demand, the cases where a trailing p99 lags what the VMs are about to need

#	Phase	Done when
15	Single-source configuration defaults (§11.1; REVIEW.md AL-03)	Every default exists exactly once, on the dataclass field ; the loader constructs each config class from the raw mapping by passing only the keys the operator actually wrote , through field-level converters (duration/byte/percent strings, list→tuple), so no <code>.get(key, default)</code> ever restates a default — today’s twin copies in <code>config.py</code> (e.g. <code>model: str = "quantile"</code> on <code>ForecastConfig</code> beside <code>fc_raw.get("model", "quantile")</code>) in the loader, and the same shape for every other knob) are gone; <code>config_schema.json</code> , the third copy of the shape, is generated from the same field/type/enum source — or, if generation proves heavier than checking, a check target fails on drift between schema and dataclasses — so it cannot rot either; zero operator-visible behaviour change : <code>--help</code> , the manual’s option tables and <code>config/drs.example.yaml</code> values are byte-identical before and after, proven by the fixture and corpus checks running green untouched

Phase 4 before phase 6 is deliberate: a working heuristic makes the MILP verifiable, and it is the production fallback for large groups. Do not start with the solver.

Phase 6’s done-when said “CP-SAT and CBC agree on every β case”; that agreement was proven while both backends existed. The CP-SAT backend has since been removed (REVIEW.md AL-02 — not in Debian, never installable on the deployment target), and the fixture now binds CBC to the exhaustively-enumerated optimum alone. Phases 14 and 15 were added after the phase 13 implementation by the twenty-eighth review pass (REVIEW.md section 56): 14 closes §10.1’s own target design and is a behaviour change; 15 is a pure internal refactor with no behaviour to specify beyond §11.1’s existing validation rules.

Phase 11 is last only because it was found last — dogfooding the finished tool, where the noise on a clean run and the silence on an auto run are both obvious in a way they never were while the engine underneath was still being built.

13. Failure modes and safety

Hazard	Handling
Metric gap / disk below <code>min_coverage</code>	Use last known load from state; flag in output; never treat as zero
Counter reset (VM reboot, node migration)	Handled by <code>rate()</code> ; sum by (vmid, device) collapses node labels
VM live-migrated between nodes mid-plan	Re-fetch node before each move (§9.2); mismatch → abort move, re-plan

Hazard	Handling
Disk has an existing snapshot chain	Pinned, load and bytes still counted, reported at WARN every run with the pinned load/bytes per VM (§3.7). <code>move_disk delete=1</code> is rejected by PVE on such volumes and would not carry the snapshots anyway
Snapshot created between planning and execution	Re-checked immediately before every move (§9.2 step 5); the move is dropped and the plan re-planned
Target storage filled, or a disk created or moved onto it, by another user or tool after planning	The live provisioned re-check (§9.2 step 2) refuses the move; the run re-plans from fresh state, up to <code>execution.max_replans_per_run</code> , then bails out (exit 0, next run starts fresh) — §9.2 re-plan protocol
PVE API error while re-reading the VM or the target before a move, or an unsized volume on the target	Not a mismatch: the run fails (exit 1), no further group is visited — §9.2 “Errors are not mismatches”
Prometheus error while computing a group’s load, in the first plan or a re-plan	The run fails (exit 1); plan/explain report every group they can and exit 1 — §9.2 “Errors are not mismatches”
VM has <code>efidisk0</code> / <code>tpmstate0</code>	Ordinary movable disks on PVE 9.2 (verified on a live cluster). Below <code>migration.tiny_disk_bytes</code> they move free of β , γ and the payback test, so κ reunites them with their VM (§3.6, §5.4, §7.3). Do not assume drive-mirror semantics for <code>tpmstate0</code> ; §8.1’s both-storages invariant holds either way (§3.6)
VM has disks on <code>ide/sata/virtio</code> , not just <code>scsi</code>	Full bus regex in §3.5; enumerating only <code>scsi*</code> silently mis-accounts capacity
unused{N} volumes	Movable with $\ell_d = 0$, so the solver relocates them only to repair a reserve violation — the intended policy
VM is locked (backup, snapshot, migrate, ...)	Pinned at planning time, waited for at execution time up to <code>execution.locks.wait_timeout</code> ; the lock value set is treated as open-ended and never whitelisted (§9.3)
Source space not reclaimed after a successful move	<code>saferemove</code> zeroes the old volume at ~10 MiB/s; the move stays in the draining state and keeps charging the source until the volume is observed gone (§8.2, §9.3)
Next move blocked by the previous move’s wipe	Completion requires task OK and source volume absent and lock clear; <code>cooldown_per_storage</code> validated against the wipe time (§9.3)
Thin provisioning	Never considered (§5.1): every disk counts at its provisioned size, so a thin pool’s “used” is $\sum z_d + U^{\text{ext}}$, which can be several times what the pool reports allocated. <code>migration.assume_thick_provisioning</code> survives only as an accepted-true, refused-false key; there is no allocated-size mode
Foreign volumes on a storage	Counted via <code>count_foreign_volumes</code> ; otherwise the reserve silently overstates free space
Orphaned target volume after a failure	Detected and reported, never auto-deleted (§9.3)
Storage already violating the reserve	Soft slack <code>r_s</code> keeps the model feasible; violation bypasses gates and is scheduled first
Storage below its configured free-space requirement (<code>free_space.soft</code> , §5.3.1)	Same handling as a reserve violation: slack keeps the model feasible, the §6 override bypasses drift/imbalance, a repairing plan is payback-exempt (§7.3) and the <i>direct</i> repair — the move off the short storage — is scheduled first (§8.2 exception 1); an <i>indirect</i> repair, one that frees the space the direct repair needs, waits for the spec-only exception 2, as §14.8’s second move shows; an unrepairable shortfall is reported with the byte amount
Group I/O-balanced but data concentrated on few storages	The capacity gate (§6) triggers planning anyway and the δ term (§5.4) does the spreading; it still honours cooldowns, payback and the transient invariant
Plan’s entire value is affinity repair ($\Delta\text{imbalance} \approx 0$)	Affinity improvement counts in the payback benefit and tiny moves are payback-exempt (§7.2, §7.3); acting with near-zero balance benefit is a correct outcome, not a defect

Hazard	Handling
Two DRS instances running	Advisory lock in <code>state.json</code> plus a startup scan for in-flight <code>move_disk</code> UPIDs owned by the DRS user. The lock is <code>node-local</code> ; only the UPID scan crosses the cluster (§11)
Config edited mid-run, cluster-wide	The config is read once at startup and never re-read; the resolved path and its SHA-256 are logged, so a plan can be traced to the exact file that produced it
Storage <code>/.../</code> pattern matches nothing in the cluster	Hard error before planning, exactly like a literal id that does not exist: the likelier cause is a typo, and the alternative is a silently shrunk group (§11.4)
Cluster storage set changes after the config is written	Patterns are re-expanded against the live inventory every run, so a new LUN joins its group with no config edit; the expansion is logged and shown by <code>verify-storages</code> (§11.4)
Solver infeasible or timing out	Fall back to the heuristic; never emit a partial/unvalidated assignment
<code>bwlimit</code> misunderstood	It is KiB/s in the API; config is bytes/s and must be converted
PVE Dynamic Load Balancer moves a VM mid-plan	Node re-fetched before every move (§9.2); mismatch triggers a bounded re-plan
Engine crashes mid-move	<code>inflight_upids</code> in <code>state.json</code> + startup scan; the PVE task continues regardless
Concurrent moves onto one storage	Generalized transient invariant over the in-flight set (§8.1)
Config knob with no effect	§11.1 validation; every knob maps to exactly one formula (§15)
A diagnostic bundle carries an identifier or a secret	Allowlist, never denylist: a field reaches a bundle only if <code>anonymize.py</code> names it, and an identifier with no mapping drops its whole record rather than passing through. The corpus scrub audit re-checks every committed bundle against the same allowlist plus IP/email/IQN/PEM/hex patterns, on the assumption that the collector has a bug (§16.3, §16.6)
Two unrelated clusters produce the same pseudonyms	The salt is 32 bytes of <code>os.urandom()</code> persisted per host, never <code>/etc/machine-id</code> — which is routinely cloned by templates and golden images and would turn the mapping into a cross-bundle correlation key (§16.3)
A bundle replays to a different plan than the live run	The replay clients key on the anonymized query text the engine itself regenerates, and a miss is a loud error naming the query and the range, never an empty result (§16.5). <code>collect-testdata</code> captures the superset of every forecaster's <code>required_range()</code> , not the configured model's (§16.2)
A bundle looks complete but a permission silently emptied a response	Every captured call records its outcome (<code>ok/http_error/empty/refused/skipped</code>) and the bundle ships <code>verify-metrics</code> findings from capture time (<code>verify-storages</code> 's own report replays from the topology/config the bundle already carries, needing no separate capture) — the <code>Datastore.Allocate</code> false negative of §3.5 must be visible in the bundle, not inferred from it (§16.2)

Overarching rule: **the reserve constraint is never traded against balance**. Stated precisely: with the lexicographic solve of (C5) this is exact — the reserve shortfall is minimized in a prior stage that the balance objective cannot influence at any weight. With the single-stage big-M alternative it is *effectively* rather than *provably* hard, because P is a calibrated constant; §5.3 gives the bound P must clear. Either way $\sum r_s > 0$ means physically impossible, not merely unattractive, and (C5) is enforced before, during and after every move. The same rule covers the configured free-space requirement (§5.3.1): `soft_s` enters the same `max()`, the same slack, the same lexicographic stage, so a byte of configured free space is exactly as non-negotiable as a byte of snapshot reserve — and `hard_s` is the floor the transient states of §8.1 may not cross.

14. Worked example

A complete, self-consistent fixture. Implementations must reproduce these numbers exactly.

The machine-readable form lives in `tests/fixtures/fc-tier1.yaml` (input) and `tests/fixtures/fc-tier1.expected.json` (expected derivations for every (β, δ) pair in the input's `beta_values × delta_values` sweeps, the execution order with its transient checks, the post-plan reserve state, and both payback calculations). Assert against those files in CI rather than transcribing the tables below.

The expected file is **generated, not written**: `tests/fixtures/generate_expected.py` enumerates all $3^6 = 729$ assignments per (β, δ) pair, so the recorded optimum is proven rather than hand-worked, and derives the order with the §8.2 rule and the §8.1 transient predicate. Run it with `--check` in CI to assert the committed file is current; that check is also the regression test for §5.5's coefficient scaling, since a scaling bug shows up as a different optimum.

This example cannot test everything, and one gap is worth naming. In `fc-tier1` every reserve-violating assignment is *also* worse on balance, so the lexicographic solve and the single-stage big-M solve agree here at **any** $P \geq 0$ — the fixture simply never exercises the distinction the two options of §5.3 exist to make. `tests/fixtures/reserve-tradeoff.yaml` is the companion fixture that does; see §14.6. §14.7's `affinity-repair.yaml` covers a third gap of the same kind: the payback rule's treatment of plans whose value is affinity. §14.8's `free-space-repair.yaml` covers a fourth: the free-space repair mandate of §5.3.1, a plan payback rejects and the requirement executes anyway.

14.1 Input

Group `fc-tier1`, three storages of 8.0 TiB each, all `capability_weight = 1.0`, $f = 2.0$, no foreign volumes. `config/drs.example.yaml` uses the same group and storage names with the same weights, so the example config and this fixture agree; the differing-weight feature is illustrated on `fc-tier2` there instead. ℓ_d is in average in-flight I/O requests (§4), *not* normalized to sum to one — which is what makes the $\omega = 1.0$ per mirror endpoint in §14.5 commensurable with it.

Disk	VM	z_d (TiB)	ℓ_d	On
101:scsi0	101	2.0	3.0	san-a
101:scsi1	101	1.0	1.0	san-a
102:scsi0	102	1.5	2.5	san-a
103:scsi0	103	0.5	0.4	san-b
104:scsi0	104	1.0	0.3	san-b
105:scsi0	105	0.5	0.2	san-c

$\Sigma \ell = 7.4$, so $u^* = 7.4 / 3 = 2.4667$.

14.2 Initial state

Storage	L_s	used	Z_s	used + $f \cdot Z_s$	vs C_s
san-a	6.50	4.5	2.0	8.5	8.0 → violates by 0.5 TiB
san-b	0.70	1.5	1.0	3.5	8.0 ✓
san-c	0.20	0.5	0.5	1.5	8.0 ✓

- Spread $(6.50 - 0.20) / 2.4667 = 2.554 \rightarrow$ **255%**, far above the 20% gate.
- $E_{\text{before}} = |6.50 - 2.4667| + |0.70 - 2.4667| + |0.20 - 2.4667| = 4.0333 + 1.7667 + 2.2667 = 8.0667$
- Data spread (§5.3 (C7)): fills 56%/19%/6% against $\bar{b} = 0.2708$, $F_{\text{before}} = 2.154$ — san-a alone holds 69% of the group's bytes. $(0.5625 - 0.0625) / 0.2708 = 1.85$, far above the 25% capacity gate.

- san-a already breaches the snapshot reserve. This is why (C5) carries slack r_s rather than being hard — a hard constraint would report *infeasible* here and refuse to help.

14.3 Solution

With default weights ($\alpha=1.0$, $\beta=0.25$, $\gamma=0.05/\text{TiB}$, $\kappa=0.5$, $\delta=0.5$) the optimum is **two** moves:

1. 102:scsi0 san-a → san-c
2. 101:scsi1 san-a → san-b

Storage	L_s	used	Z_s	used + f·Z_s	fill	✓
san-a	3.00	2.0	2.0	6.0	25%	✓
san-b	1.70	2.5	1.0	4.5	31%	✓
san-c	2.70	2.0	1.5	5.0	25%	✓

$E_{\text{after}} = 0.5333 + 0.7667 + 0.2333 = 1.5333$, spread $(3.00-1.70)/2.4667 = 53\%$; the fill deviation drops from $F_{\text{before}} = 2.154$ to $F_{\text{after}} = 0.308$. The reserve violation is repaired.

The β knob, demonstrated at $\delta = 0$. Switch the capacity term off and a third move becomes worth taking: 105:scsi0 san-c → san-b improves the imbalance by $1.5333 - 1.1333 = 0.400$, and its objective contribution is

$$\begin{aligned} \alpha \cdot \Delta E + \beta \cdot 1 + \gamma \cdot 0.5 \text{ TiB} &= -0.400 + 0.250 + 0.025 = -0.125 \rightarrow \text{accepted at } \beta=0.25 \\ &= -0.400 + 0.500 + 0.025 = +0.125 \rightarrow \text{rejected at } \beta=0.50 \end{aligned}$$

So with `delta_capacity_spread: 0` the solver returns the **three-move** plan — (3.00, 1.90, 2.50), $E = 1.1333$, spread 44.6%, fills 25%/38%/19% — for every `beta_move_count < 0.375`, and the two-move plan above for everything past it. This is exactly the “minimal number of migrations” trade-off made explicit and tunable; both plans are correct, and β chooses.

The δ knob, demonstrated at the defaults. The third move concentrates data — san-b’s fill rises to 38% while san-c’s falls to 19%, F goes from 0.308 to 0.769, $\delta \cdot \Delta F = +0.231$ — and that regression outweighs the move’s net I/O gain:

$$\alpha \cdot \Delta E + \beta \cdot 1 + \gamma \cdot 0.5 \text{ TiB} + \delta \cdot \Delta F = -0.400 + 0.250 + 0.025 + 0.231 = +0.106 \rightarrow \text{rejected}$$

The two-move plan is therefore the optimum at the defaults (full objectives: 3.664 against 3.769 — both split VM 101, so both carry the same $\kappa \cdot w_{101} = 1.351$ and this particular comparison is unchanged by the weighting), and it is also ahead on migrations ($2 < 3$), bytes moved ($2.5 < 3.0$ TiB) and data spread. That is the intended shape of the term: α decides how much imbalance to remove; δ helps decide when a further move would concentrate data more than it evens load. (At `beta_move_count: 0.5` the two-move plan wins with or without δ .)

The affinity trade-off, demonstrated. The two-move plan splits VM 101 (scsi0 on san-a, scsi1 on san-b). VM 101 carries $\ell_v = 4.0$ against a group mean of $\ell^* = 7.4/5 = 1.48$, so $w_{101} = 2.703$ and the split incurs $\kappa \cdot w = 1.351$ where an unweighted term would charge 0.5 — the group’s heaviest VM is worth 2.7 average ones to keep together (§5.4). Keeping VM 101 together forces san-a to $L = 4.0$ and the best reachable E becomes 3.133. Comparing on the persistent terms ($E + \kappa \cdot w + \delta \cdot F$): $3.133 + 0 + 0.5 \cdot 0.769 = 3.52$ (together) against $1.5333 + 1.351 + 0.5 \cdot 0.308 = 3.04$ (split). Splitting still wins — high I/O legitimately overrides the preference — but by 0.48 rather than the 1.3 an unweighted κ would have given, and a somewhat busier VM 101 would flip the comparison entirely, which is what the weighting is for.

14.4 Ordering

102:scsi0 is scheduled first: it alone repairs san-a's reserve violation ($4.5 \rightarrow 3.0$ used, so $3.0 + 4.0 = 7.0 \leq 8.0$), and it also has the largest persistent-objective reduction per unit cost. Transient checks for the two-move plan:

```
move 1 → san-c:  used 0.5 + 1.5 = 2.0,  max(Z_c, 1.5) = 1.5,  2.0 + 3.0 = 5.0 ≤ 8.0 ✓
move 2 → san-b:  used 1.5 + 1.0 = 2.5,  max(Z_b, 1.0) = 1.0,  2.5 + 2.0 = 4.5 ≤ 8.0 ✓
```

The $\delta = 0$ three-move variant appends 105:scsi0 → san-b — used $2.5 + 0.5 = 3.0$, $\max(Z_b, 0.5) = 1.0$, $3.0 + 2.0 = 5.0 \leq 8.0$ ✓ — recorded alongside in the expected file's delta_values sweep.

14.5 Payback

Atbwlimit = 200 MiB/s, $\omega_{\text{src}} = \omega_{\text{dst}} = 1.0$, $H = 365\text{d} = 31\,536\,000\text{ s}$, $\lambda = 10$, and **saferemove off on all three storages** so $\text{duration_wipe_d} = 0$ (\$7.1), for the two-move plan:

That last assumption is not a detail of the prose — the config default is `migration.account_saferemove_wipe: true`, and with a wipe at the PVE default of 10 MiB/s the 1.5 TiB move below would carry ~44 h of zeroing on top of its 2.2 h mirror and the arithmetic would look nothing like this. So the fixture states it in the data rather than leaving it to be inferred: `saferemove: false` on each storage, `account_saferemove_wipe: false` in the migration block, and an assumptions object echoed into the expected file. The per-move records carry `duration_mirror_seconds` and `duration_wipe_seconds` separately, so a run with wiping enabled is a different number in a named field rather than a silent discrepancy.

Move	Size	Duration	cost = 2 × duration
102:scsi0	1.5 TiB	7 864 s (2.18 h)	15 729 load·s
101:scsi1	1.0 TiB	5 243 s (1.46 h)	10 486 load·s
Σ			26 214 load·s

```
benefit = (α·ΔE + δ·ΔF + κ·ΔA) · H
         = (6.5333 + 0.5 × 1.8462 - 0.5 × 2.7027) × 31 536 000
           (ΔF = 2.1538 - 0.3077; ΔA = 0 - 2.7027: the plan splits
            VM 101 — the group's heaviest,  $w_{101} = 2.703$  — and $7.2
            charges that fragmentation against the benefit)
         = 6.1050 × 31 536 000 = 192 527 280 ≈ 1.93×108 load·s
ratio    = 192 527 280 / 26 214 ≈ 7 344 ≥ λ = 10 → ACCEPT
```

A move that fails payback. Consider instead a 4.0 TiB archive disk with $\ell = 0.1$ whose relocation would improve E by only 0.01 and leave the data spread essentially unchanged:

```
cost      = 2 × (4.0 TiB / 200 MiB/s) = 2 × 20 972 = 41 943 load·s
benefit   = 0.01 × 31 536 000 = 315 360 load·s
ratio     = 7.5 < λ = 10 → REJECT
```

Even a full year of accumulated benefit does not pay for this migration. This is the requirement that “migrating a very large disk might generate more traffic than we are trying to save”, enforced numerically — and it is the horizon that sets the bar: at the previous 7d default this same test demanded $\Delta\text{obj} \geq 0.69$ for a disk of this size (0.17 per TiB moved) and rejected a large share of the moves that were worth making, which is why H is configurable and why its default is now stated as an explicit assumption about how long a placement lasts (\$7.2).

14.6 Companion fixture: when the reserve and the balance objective disagree

tests/fixtures/reserve-tradeoff.yaml is a second, deliberately awkward group, and its only job is to separate the two solve paths of \$5.3.

Two storages: roomy (20 TiB, empty) and cramped (5 TiB, of which 3 TiB is already `Used` — volumes DRS does not manage). $f = 2.0$. Two disks, both 1.0 TiB, both $l_d = 5.0$, both on roomy, belonging to different VMs. So $u^* = 5.0$, and:

Assignment	$\sum r_s$	E	Non-reserve objective at $\beta = 0.25, \delta = 0.5$
both on roomy (current)	0	10.0	11.25
one moved to cramped	1.0 TiB	0.0	2.18
both moved to cramped	2.0 TiB	10.0	13.10

Moving one disk balances the group *perfectly* and costs a 1 TiB reserve breach on cramped ($3 + 1 + 2 \cdot 1 = 6 > 5$). That is the trade the reserve rule exists to forbid, and the three answers are:

- **Lexicographic (the default).** Stage 1 finds $\min \sum r_s = 0$, stage 2 optimises within that set — so the plan is *no moves at all*, and the group stays at $E = 10$. Correct, and it needed no calibration to be correct.
- **Big-M at the configured $P = 1000$.** Same answer: $2.18 + 1000 > 11.25$.
- **Big-M at $P = 5$.** $2.18 + 5 = 7.18 < 11.25$, so it moves the disk and breaches the reserve for balance. The exact flip point, recorded in the expected file as `big_m_agreement_threshold_p`, is $P = 9.08$: below it big-M is wrong, above it big-M is right. Note how small that number is — nothing about $P = 5$ looks obviously wrong to an operator, which is the whole argument for computing P rather than configuring it. (The δ term raises the moved plan's objective — concentrating both disks on cramped is also the worse data spread, and the gap between the two plans narrows from 9.70 to 9.08 — but not enough to matter at any sane P .)

The build-time bound of \$5.3 gives $P_{\min} = 23.6 \cdot 2^{20} \approx 2.47 \times 10^7$ for this group, four orders above the 9.08 actually needed. That is the bound doing its job: it is deliberately worst-case (it refuses to trade even one mebibyte), and being conservative in the safe direction costs nothing.

One further check the fixture records: the $P = 5$ plan is not merely undesirable, it is **unschedulable**. Its single move fails the \$8.1 transient predicate on cramped, so \$8.2 reports a deadlock rather than emitting it. The two safety mechanisms are independent, and the fixture asserts that both fire.

Disk 201:scsi0 and 202:scsi0 are interchangeable here; the recorded move names 202:scsi0 because that is how the enumerator breaks the tie. An implementation may pick either — assert on the move *count* and the resulting slack, not on the disk identity.

14.7 Companion fixture: affinity repair under the payback rule

tests/fixtures/affinity-repair.yaml isolates what \$5.4's strengthenings and \$7.2's corrected benefit exist for: a group whose I/O is perfectly balanced and whose only improving moves are a VM's tiny disks reuniting with it. Three 8 TiB storages stor-a/stor-b/stor-c, $f = 2.0$, equal capabilities, saferemove off:

Disk	VM	z_d	l_d	On
301:scsi0	301	1.0 TiB	2.0	stor-a
301:efidisk0	301	528 KiB	0.0	stor-c
301:tpmstate0	301	1 MiB	0.0	stor-b
302:scsi0	302	1.0 TiB	2.0	stor-b
309:scsi0	309	1.0 TiB	2.0	stor-c (pinned: exclude.vmid)

plus 4.0 TiB of foreign volumes on stor-c. Loads read 2.0/2.0/2.0, so the imbalance gate stays shut; fills read 0.125/0.125/0.625 against $\bar{b} = (3.0 + 4.0)/24 = 0.2917$, so the capacity gate fires on a spread of

$(0.625 - 0.125)/0.2917 = 1.714$. VM 301 sits on three storages; $\ell^- = 6.0/3 = 2.0$, so $w_{301} = 1$.

The optimum is exactly two moves — 301:efidisk0 stor-c $\rightarrow$ stor-a and 301:tpmstate0 stor-b $\rightarrow$ stor-a — worth $\kappa \cdot w_{301} \cdot 2 = 1.0$ at zero β/γ cost. The alternatives all improve less: joining scsi0 to its tiny disks costs $\beta + \gamma \cdot 1.0 \text{ TiB} = 0.3$ for half the affinity gain and a large fill-deviation regression, and every other byte-moving candidate worsens δ more than it helps anything. Payback accepts with benefit $\approx \kappa \cdot \Delta A \cdot H = 3.15 \times 10^7 \text{ load} \cdot \text{s}$ against cost $= 0$.

This fixture pins \$5.4/\$7.2's *value*, not a verdict flip: the affinity repair is worth 31 536 000 of the plan's benefit_load_seconds: 31 536 006.65 ($\kappa \cdot \Delta A \cdot H$, 99.99998 % of it) at exactly zero cost, and test_affinity_repair_fixture.py asserts that by value rather than by pass/fail. It is **not** the fixture that flips reject-to-accept — replayed through the pre-\$7.2 formula (no $\kappa \cdot \Delta A$ term, tiny_disk_bytes = 0 so both tiny disks are charged their real, nonzero mirror cost), this same two-move plan still *accepts*: $\Delta E = 0$ exactly, ΔF is positive (the tiny disks leave above-mean storages for a below-mean one), giving benefit $\approx \delta \cdot \Delta F \cdot H = +6.65 \text{ load} \cdot \text{s}$ against cost $\approx 0.0152 \text{ load} \cdot \text{s}$ — ratio ≈ 438 , the same $\checkmark$ the fix produces. The verdict flip the live cluster actually suffered belongs to the *pre-\$12* formula alone (benefit = $\Delta E \cdot H$ at the old 7d horizon, \$12's phase table): that formula scores this plan benefit = $0 < \lambda \cdot \text{cost}$ and rejects it. What a silent revert of κ/w_v does catch on this fixture is the test's exact-value assertions (benefit $\approx 31\,536\,006.65$, cost_load_seconds == 0.0), not the verdict.

14.8 Companion fixture: the free-space repair mandate

tests/fixtures/free-space-repair.yaml isolates what \$5.3.1's requirement and \$7.3's repair exemption exist for: a group whose I/O is **perfectly balanced**, whose snapshot reserve is satisfied everywhere, and where a single storage sits below its configured free-space requirement because an admin placed foreign volumes on it. The engine must repair it anyway — the Storage-DRS cluster function — and the repair is exactly the kind of plan the payback rule would reject on its own.

Three 10 TiB storages packed/roomy/swapme, $f = 2.0$, equal capabilities, saferemove off, free_space.soft: "30%" (3.0 TiB) on all three, free_space.hard swept by the fixture: null (= 3.0) and "10%" (1.0 TiB). swapme accepts only raw volumes — (C2) format eligibility — which is what makes the repair a two-move plan rather than a one-move one.

Two prerequisites this fixture states openly rather than assumes. First, the (C2) format-compatibility rule it leans on. *It was not implemented when the fixture was specified* — topology.Storage did not expose storage type/format, so both solver backends treated every group storage as an eligible target for every disk — and under that behaviour the fixture's optimum would have been the *one-move* repair 601:scsi0 packed $\rightarrow$ swapme (objective 1.081 — but **not** a plan that pays for itself: it relocates the quiet 0.05 disk off a perfectly balanced group, so $E: 0 \rightarrow 0.10$ against $F: 1.608 \rightarrow 1.412$, benefit $(-0.100 + 0.5 \times 0.196) \times 31\,536\,000 \approx -6.2 \times 10^4 \text{ load} \cdot \text{s}$ against a 5 243 load·s cost, ratio -11.8, and it clears the gate only through the same repair exemption the two-move plan needs — its own Σr_s is $0.5 \rightarrow 0$, so both the built has_reserve_override and \$7.3's outcome trigger fire on it). What depends on the format rule is the plan's *shape*, not its economics: the two-move repair, the indirect repair the revert test marks, and both hard-sweep orders below. Capacity alone cannot substitute for the format rule: \$8.1's predicate is monotone in z_d , so any storage that accepts the 1.0 TiB 603 accepts the 0.5 TiB 601, and pinning 603 (the affinity-repair.yaml mechanism) does not help either — it leaves 601 $\rightarrow$ swapme legal and optimal. **As built (AH-03):** phase 13 landed the rule (Storage.storage_type/allowed_formats, one shared storage_accepts_format(), both backends fixing $x_{\{d,s\}}=0$) *in the same commit* as the fixture, so the one-move counterfactual above is history rather than a state the fixture can observe — which is why the expected file carries no requires_format_eligibility marker: its only job was to fail loudly on a run *before* the rule existed, and no such run can exist. The one-move figures survive as the hand derivation in REVIEW.md \$48 (AG-01), not as a recorded case. Second, the group's fills (0.75/0.68/0.10) put the capacity gate at $(0.75 - 0.10)/0.51 = 127\%$, far above the 25% default — the gate is what lets the engine *plan* here, and

the fixture sets `gates.capacity_spread_threshold: null` to keep that planning decision from being the fixture’s own doing: with the gate open, the soft-less counterfactual below is exactly \$9.5’s “gate says ACT, solver’s optimum is to move nothing” case, pinned deliberately.

Disk	VM	z_d	ℓ_d	format	On
601:scsi0	601	0.5 TiB	0.05	qcow2	packed
602:scsi0	602	1.0 TiB	2.00	qcow2	packed
603:scsi0	603	1.0 TiB	2.05	raw	roomy
604:scsi0	604	1.0 TiB	2.05	raw	swapme

plus foreign volumes — the admin’s new VMs, which DRS does not manage — of 6.0 TiB on packed and 5.8 TiB on roomy. Initial state:

Storage	used	Z_s	f·Z_s	soft_s	R_s	free	r_s
packed	7.5	1.0	2.0	3.0	3.0	2.5	0.5
roomy	6.8	1.0	2.0	3.0	3.0	3.2	0
swapme	1.0	1.0	2.0	3.0	3.0	9.0	0

Loads read 2.05/2.05/2.05 — $E = 0$, the drift and imbalance gates shut — and the snapshot reserve alone is satisfied everywhere (packed: $7.5 + 2.0 = 9.5 \leq 10$). Only the configured free space is violated, by 0.5 TiB, on packed alone. This is the case the old `min_free_bytes` could not express as a *per-storage policy* and could not *repair*: nothing distinguishes it from a healthy cluster except the requirement. (The gate’s ACT is what lets the engine plan — see the capacity-gate prerequisite above — and the requirement is what makes the solver *move*. The isolation claim is about the solver’s objective, not the gate stack.)

The optimum is two moves, and it is a repair, not a balance. Exhaustive enumeration over all format-feasible assignments confirms the lexicographic optimum is unique:

1. 601:scsi0 packed → roomy — the quiet 0.5 TiB disk, chosen over the 1.0 TiB 602 by γ (both repairs cost $\beta \cdot 1$; the smaller disk moves fewer bytes);
2. 603:scsi0 roomy → swapme — **not because roomy violates anything**: roomy ends at 6.3 used, 3.7 free, compliant. It moves because 601 cannot land on swapme (qcow2 on a raw-only storage, (C2)), and roomy cannot end below its own soft with 601 added on top of 603 ($6.8 + 0.5 + 3.0 = 10.3 > 10$). The move is an *indirect repair* — it empties the destination the direct repair needs — which is why \$7.3’s per-move marker is a revert test rather than “source was in violation”: holding 603 back raises the plan’s final Σr_s from 0 to 0.3 (roomy at 7.3 used, 2.7 free, short by 0.3), so it is marked repair: true even though roomy never violated anything.

Storage	used	Z_s	R_s	free	r_s
packed	7.0	1.0	3.0	3.0	0
roomy	6.3	0.5	3.0	3.7	0
swapme	2.0	1.0	3.0	8.0	0

Σr_s : 0.5 → 0. The counterfactual is recorded in the expected file: with `free_space.soft: 0` on the same cluster, the optimum is **no moves at all** (objective 0.804 against the repair’s 5.283) — the requirement, and nothing else, is what makes the solver move. (With the capacity gate nulled, that counterfactual is the engine declining to act at all; with the gate left at its default it is \$9.5’s ACT-but-no-moves case, reported by `explain`’s closest-alternative block.)

Payback rejects this plan, and the exemption is what executes it. The repair wrecks the balance the

cluster started with: loads become 2.00/0.05/4.10, E: 0 → 4.1, and the fill spread improves only from F = 1.608 to F = 1.216:

```
benefit = (α·ΔE + δ·ΔF) · H = (-4.100 + 0.5 × 0.392) × 31 536 000 = -1.23×108 load·s
cost     = 2 × (0.5 TiB/200 MiB/s) + 2 × (1.0 TiB/200 MiB/s) = 5 243 + 10 486 = 15 729 load·s
ratio    = -7 827 < λ = 10 → REJECT – overridden: the plan repairs (Σ r_s 0.5 → 0), §7.3-exempt
```

A negative benefit is the point: this plan moves a 0.05-load disk off a perfectly balanced group and concentrates 4.10 of load on swapme. No weighting of the \$5.4 objective can make it attractive, and no payback horizon can make it pay — the mandate is the only thing that produces it, which is precisely why the exemption must be structural (§5.3) rather than a weight.

The hard sweep, and why the order flips. §8.2 schedules 601 → roomy first by its exception 1 — it is the move that resolves packed's (C5) violation — with 603 → swapme behind it: exception 2 applies to that move too (it frees the roomy space the first move needs), but rule 1 outranks rule 2. Note the ratio itself would rank both moves *last*: a repair's persistent-objective reduction is negative here (§8.2's α/δ/κ terms all get worse), which is exactly why the exceptions exist as priorities rather than as weights. (As built, only exception 1 is implemented — schedule.py's `_resolves_reserve_violation` — and exception 2 remains spec-only; the flip demonstrated here needs only exception 1 plus the §8.1 predicate, so the fixture does not depend on the gap.) The transient check on roomy at that moment: 603 has not moved yet, so roomy holds 6.8 + 0.5 incoming = 7.3 used, and the predicate demands 7.3 + max(2·1.0, hard) ≤ 10:

- hard: null (= soft = 3.0): 7.3 + 3.0 = 10.3 > 10 — **infeasible**. The scheduler reverses the order: 603 → swapme first (its own check on swapme is 1.0 + 1.0 + max(2·1.0, 3.0) = 5.0 ≤ 10 ✓), which empties roomy to 5.8 used, and then 601 lands at 5.8 + 0.5 + max(2·max(0, 0.5), 3.0) = 5.8 + 0.5 + 3.0 = 9.3 ≤ 10 ✓ — note Z_roomy has dropped to 0 with 603 gone (the 5.8 TiB left on it is foreign, and 601 has not arrived yet), so the inner term is max(0, 0.5) = 0.5 and the hard floor of 3.0 dominates the snapshot term either way. The plan still executes — the requirement is met either way, at the cost of the less preferred order.
- hard: "10%" (= 1.0 TiB): 7.3 + 2.0 = 9.3 ≤ 10 — feasible as preferred. roomy dips to **2.7 TiB free**, below its 3.0 soft requirement from 601's target allocation until 603's source volume is observed gone — the whole repair, both mirrors and the drain between them, not just the first mirror — and never below the 1.0 hard floor; the plan's endpoint pulls it back to 3.7. (The reversed order's second move is looser still: 5.8 + 0.5 + max(1.0, 1.0) = 7.3 ≤ 10.)

Both orders end compliant; the expected file records expected_order per hard value, and the hard: "10%" entry is the fixture's proof that a transient dip below soft is a real, schedulable state rather than a contradiction of (C5) — the dip is on a storage the finished plan leaves compliant, exactly as §8.1 requires.

One further assertion the fixture makes: with `free_space.hard` above `free_space.soft` on any storage, `config.py` rejects the file (§11.1) — the sweep values are chosen so the invalid configuration is a one-token edit away, and the error is part of the test.

15. Requirements traceability

Original requirement	Where addressed
Proxmox API via user/password	§3.5
Per-disk metrics for all running VMs	§3.1, §3.4 (blockstat via Prometheus, not RRD — see §3.2)
Equalize I/O across storages in a group	§4, §5.3 (C6), §5.4
VMs confined to their storage group	§5.1, §5.3 (C1, C2)
Restrict to shared storages	§5.3 (C2)
2× largest disk free for snapshots	§5.3 (C5)

Original requirement	Where addressed
...including <i>during</i> migrations	§8.1 transient invariant
Reserve factor configurable	snapshot_reserve.factor, per-storage override
Configurable free space per storage (bytes or %), kept free by migration	free_space.soft (§5.3.1), §5.3 (C5), §6 override, §7.3 repair exemption; demonstrated §14.8
Minimal number of migrations	§5.4 β term; demonstrated §14.3
Min % changed traffic before acting (10%)	§6 drift gate
% I/O difference across the group	§6 imbalance gate
Keep a VM's disks together, unless space/IO forces otherwise	§5.3 (C3), §5.4 κ (I/O-weighted, tiny disks free), §7.2 payback benefit; demonstrated §14.3, §14.7
Spread data evenly over the storages (failure risk)	§5.3 (C7), §5.4 δ , §6 capacity gate; demonstrated §14.3
Mathematical optimization formulation	§5
Migration order planned	§8
Automatic or manual confirmation	§9.1 (dry-run / confirm / auto)
Forecasting (Holt-Winters or other)	§10
Configurable decision timeframe, default 24h	window.lookback; §3.4
Migration load counted against the benefit	§7; demonstrated §14.5

15.1 Config knob → formula

Every knob must appear in exactly one formula. A knob with no formula is dead configuration and a bug waiting to happen; this table is the audit.

Knob	Where it acts
load_weights.iotime/ops/bytes	§4, ℓ_d
load_weights.read_factor/write_factor	§4, raw_X(d), applied engine-side before normalization
window.lookback	§3.4 reduction range
window.quantile / upper_quantile	§10.1, point estimate (the actual decision statistic) vs. the bound (§7.3 saturation guard only — see the “As built” note in §10.1)
window.min_coverage	§3.4, disk data rejection
groups[].storages[].id in pattern form (/.../)	§11.4 expansion into group membership; the entry's options apply to every matched storage
groups[].storages[].capability_weight	§4, $u_s = L_s / c_s$
snapshot_reserve.factor	§5.3 (C5), $R_s \geq f_s \cdot Z_s$
snapshot_reserve.min_free_bytes	§5.3 (C5), $R_s \geq \text{soft}_s$ — deprecated syntax for free_space.soft (§5.3.1)
free_space.soft (global, per-storage, per-pattern)	§5.3 (C5), $R_s \geq \text{soft}_s$; §6 reserve override; §7.3 repair exemption
free_space.hard (global, per-storage, per-pattern)	§8.1 transient invariant, $\max(f_b \cdot \dots, \text{hard}_b)$ floor
snapshot_reserve.count_foreign_volumes	§5.1.1, $U^{\text{sex t}}$
gates.drift_threshold	§6 drift gate
gates.imbalance_threshold	§6 imbalance gate
gates.capacity_spread_threshold	§6 capacity gate (fill fractions from §5.3 (C7))
gates.cooldown_per_disk/storage	§5.3 (C2) pinning, §8.1 concurrency_ok
migration.bwlimit_bytes_per_sec	§7.1 duration_d; converted to KiB/s at the API call
migration.source/target_load_weight	§7.1 $\omega_{\text{src}}, \omega_{\text{dst}}$
migration.payback_horizon / payback_ratio	§7.2, §7.3 acceptance test
migration.max_single_move_duration	§7.3 hard per-move rule; compared against duration_d including the wipe
migration.account_saferemove_wipe	§7.1 duration_wipe_d
migration.wipe_load_weight	§7.1 ω_{wipe} in cost_d; §7.3 $\omega_{\text{role}}(m, s)$ while draining
migration.tiny_disk_bytes	§5.4 D^{big} (β/γ exemption); §7.1 cost_d = 0; §7.3 aggregate-test exemption
execution.locks.*	§9.3 lock wait loop; §5.3 (C2) planning-time pin
execution.source_release.*	§9.3 completion criterion; §8.2 draining state
exclude.include_unused_disks	§3.6 membership of $\mathcal{D}$
exclude.skip_vms_with_snapshots	§3.7, §5.3 (C2) pinning

Knob	Where it acts
objective.affinity_counts_pinned_disks	§5.3 (C3) range: all of D (default) or $D^{\wedge}mov$
report.warn_pinned_load_fraction	§3.7 unreachable-goal warning
migration.saturation_ceiling	$\$7.3 L_{during}(s) \leq saturation_ceiling \cdot N_s$
groups[].storages[].saturation_load	$\$7.3 N_s$; guard skipped when unset
objective.alpha_spread/beta_move_count/gamma_move_bytes_per_tib/kappa_vm_affinity/delta_capacity_spread	§5.4 (the δ term and the I/O-weighted κ term also enter §7.2's benefit)
objective.reserve_violation_penalty	§5.3 (C5), <i>floor</i> for the single-stage P alternative
metrics.pvestatd_push_interval	§11.1 rate_window validation; §3.3 verify-metrics
objective.spread_metric	§5.3 (C6), L1 vs min-max
solver.*	§5.5
execution.max_concurrent_migrations/per_storage	§8.1 generalized invariant, concurrency_ok
execution.max_replans_per_run	§9.2 re-plan protocol
execution.time_windows	§9.1 auto mode gating
exclude.*	§5.3 (C2) variable fixing
forecast.model + holt_winters.*	§10.1
proxmox.read_workers	§3.5 read-path concurrency
support.salt_path	§16.3, the persisted anonymization salt — per node by default, cluster-wide if pointed at pmxcfs
support.bundle_dir	§16.4, default output location for collect-testdata
support.max_series_points	§16.2, hard refusal ceiling on the series capture
support.capture_range	§16.2, auto for the maximum over every forecaster's <code>required_range()</code>
monitoring.status_file	§2.4, where apply writes its <code>check_statusfile-format</code> status report; null writes none

16. Diagnostic bundles: collect-testdata and --replay

Two facts motivate this section, and neither is a hypothetical.

One cluster is not a test corpus. Everything in §14 is a hand-built fixture, and the only real cluster this tool has ever run against is the author's own. Every interesting failure so far — a metric silently missing for one disk (§3.3), `/content` returning empty under `Datastore.Audit` (§3.5), a coverage query starved by a too-short `window.lookback` — was found by an operator running the tool and then *describing* what they saw. A description is not a regression test: none of those three is in CI as the shape of data that produced it, because that data never left the cluster.

The data cannot leave the cluster as-is. A PVE cluster's inventory is a list of customer names, host-names, iSCSI IQNs, PBS fingerprints, ticket numbers in snapshot descriptions and storage comments, and credentials in storage definitions. An operator who wants to help cannot simply tar up API responses, and asking them to redact by hand guarantees either a leak or a refusal.

So: one read-only command that captures everything the engine could ever ask for, replaces every identifier with a keyed pseudonym that is stable on that host and one-way to everybody else, and writes a self-contained bundle the operator can read before they send it. And one global option that runs the engine against such a bundle with no network at all, so a submitted bundle becomes an executable test case.

16.1 The bundle

A bundle is a **directory**, plus a deterministic `.tar.gz` of that directory for sending. The directory is the canonical form: it is what the `scrub audit` (§16.6) reads, what `git diff` shows when a bundle is committed to the corpus, and what an operator opens to satisfy themselves before attaching anything to an email.

```

drs-testdata-cluster-3f8a91c2-2026-09-11/
manifest.json          # schema, versions, what was captured, what failed, counts
config.yaml           # the anonymized, credential-free effective configuration
findings.json         # verify-metrics output, as captured
pve/
  cluster-resources-vm.json
  cluster-resources-storage.json
  storage-definitions.json
  nodes.json
  cluster-tasks.json
  vm-config/<vmid>.json
  vm-snapshots/<vmid>.json
  vm-status-current/<vmid>.json
  vm-pending/<vmid>.json
  storage-status/<node>/<storage>.json
  storage-content/<node>/<storage>.json
prometheus/
  label-values/<label>.json
  instant/<sha256-of-query>.json
  range/<sha256-of-query-and-range>.json
SHA256SUMS

```

Every file is JSON (or YAML for `config.yaml`), written with sorted keys, two-space indent, `\n` endings and no trailing whitespace, so a bundle diffs line by line. The Prometheus payloads are **not** individually compressed: the directory form is meant to be read and to compress well in git, and the single gzip of the tarball does the compression once.

`prometheus/instant/<sha256>.json` and `range/<sha256>.json` each carry the query text they were recorded for alongside the response, so a bundle is self-describing and a key miss at replay time (§16.5) can name what it was looking for. The hash is over the *anonymized* query text (plus the range parameters), because that is what the engine will generate when it replays against the bundle’s own anonymized config — see §16.5 for why that closes rather than opens a gap.

Determinism. Two captures of an unchanged cluster must produce byte-identical bundles apart from the coarsened capture timestamp in `manifest.json` and the metric samples themselves. That means: every collection is sorted before it is written (by `vmid`, by storage id, by node, by series label tuple); floats are written with `repr()`’s round-trip form and never reformatted; the tar members are sorted, with `mtime=0`, `uid=gid=0`, empty `uname/gname` and fixed modes; and the gzip header carries `mtime=0`. `tests/unit/test_collect.py` asserts this by running the collector twice against the same fakes and comparing bytes — a determinism claim nobody checks is a determinism claim that is already false.

16.2 Capture: the superset, not the configured path

The point of a bundle is that the *author* can run configurations the *operator* never ran. A bundle captured for `forecast.model: quantile` and `solver.backend: heuristic` that only contains 24 h of data is worthless for reproducing a Holt-Winters misfit. So the collector deliberately captures the superset of what any supported configuration could ask for.

PVE. Every read endpoint in §3.5’s table, exactly once each, for every VM and every (node, storage) pair the topology pass finds — including `cluster_tasks`, which §13’s crash-recovery scan reads and nothing else does. This is the same read path a `plan` run performs (§3.5’s “expected call count per run”), so the cost is one planning run’s worth of API calls, not a multiple of it. The write path is never touched: this command has no code path that can issue `move_disk`, and `--replay` has none either (§16.5).

Prometheus. For every group, for each of the six raw metrics of §3.3:

- the instant query `verify-metrics` issues, so a bundle reproduces §3.3’s own checks;

- `label_values` for each of the three configured labels;
- the `quantile_over_time` reductions of §3.4 that `loadmodel.compute_group_load()` consumes, for both `window.quantile` and `window.upper_quantile`;
- the sum by `(vmid, device) (rate(...))` **range** query of §3.4 over the **capture range**:

```
capture_range = max( over every registered forecaster of required_range(),
                    2 * window.lookback )
    = max(window.lookback,
          forecast.seasonal_lookback_days,
          2 * holt_winters.seasonal_periods * metrics.step,
          2 * window.lookback)
```

at `metrics.step` resolution — the union of §10.1’s table, not the row the operator happens to have selected, **plus** `2 * window.lookback`: §10.2’s backtest gate fits on `[now-2W, now-W)` and checks against `[now-W, now]` for whichever of `seasonal_naive/holt_winters` a bundle is replayed with, so a capture sized only to a forecaster’s own minimum (which can equal `window.lookback` exactly, e.g. `holt_winters` tuned so `2 * seasonal_periods * metrics.step == window.lookback`) would replay the backtest gate as permanently “not enough history”, independent of how much real history Prometheus actually had. With the defaults that is `max(24h, 7d, 48h, 48h) = 7d`.

This is cheap in *queries* and expensive in *bytes*, which is the right way round. Each range query returns every disk in the group as one response, so the query count is `6 * |groups| * (range_chunks + 3 instant) + 3 label_values`, where `range_chunks = [capture_range / 1 day]` is the day-sized chunking above (7 at the 7 d default, not 1 — as built, REVIEW.md X-09: the printed estimate treated a multi-day range as a single range query per group per metric, undercounting the real HTTP request count by roughly the chunk count). Still tens to low hundreds of queries for any cluster — while the payload is `6 * |disks| * capture_range / metrics.step` samples, about 12000 samples per disk at the defaults, or roughly 2.4 million samples for a 200-disk cluster. So:

- the collector **computes the estimate and enforces `support.max_series_points` against it before fetching anything**, derived from the topology pass it has already done — as built (Y-05): `--estimate` prints the estimate and exits without fetching; a real capture computes it internally, purely to enforce the refusal below, and prints nothing about it before proceeding;
- `support.max_series_points` (default 5 million) is a hard refusal, not a truncation, naming the flags that would bring the capture under it;
- `--range`, `--step` and `--no-series` override the capture range, the resolution and the range queries respectively, each recorded in the manifest so the author can see what they are missing rather than inferring it from a short file.

A range longer than Prometheus will serve in one request is **chunked into day-sized sub-queries and stitched**, deterministically (chunk boundaries fall on the range’s own start, never on wall-clock “now”), because a backend refusing a `7 d × 5 m` request with `max_samples` exceeded is a configuration difference between deployments and not a reason to hand back a short bundle.

Failures are recorded, never rendered as absence. This is the direct lesson of §3.5’s Datastore. Allocate finding: a `/content` call that returns 200 and an empty list looks exactly like an empty storage. Every captured call carries its outcome in the manifest — `ok/http_error/empty/refused/skipped` — and the bundle additionally ships `findings.json`, the verbatim output of `verify-metrics` against the live cluster at capture time. (`verify-storages`’s own report needs no separate capture: it is a pure function of the topology and config the bundle already carries in `pve/` and `config.yaml`, so `--replay <bundle> verify-storages` reproduces it byte-for-byte from data already there — storing it a second time in `findings.json` would be redundant, not additional coverage. `verify-metrics` is different because its report depends on a live Prometheus response that is otherwise gone the moment capture ends.) An operator whose bundle

records “every storage reported zero volumes” has a bundle that says so, and the author reading it sees a permissions finding instead of a cluster with no disks.

A capture in which some calls failed is still written, and the command exits 1 with the failures summarized. A partial bundle is useful; a bundle that pretends to be complete is not.

16.3 Anonymization

The governing rule: allowlist, never denylist

A field reaches the bundle only if anonymize.py names it. Not “every field except the ones we strip” — every field the engine actually reads, and nothing else. The allowlist is derived from pve.py’s own accessors and is the single implementation (AGENTS.md §5) shared by the collector and the corpus scrub audit (§16.6), so the audit cannot drift from what the collector permits.

This is the one decision in this section that must not be softened for convenience. A denylist of “fields known to be sensitive” is wrong on the next PVE release, which will add a field nobody listed: GET /storage alone carries fingerprint, password, encryption-key, keyring, server, portal, target, export, monhost, username, options and comment, of which the engine reads exactly storage, type, content, shared, nodes, disable, saferemove and saferemove_throughput. A VM config carries net0 (MAC addresses), ipconfig0, sshkeys, cipassword, smbios1, description and hookscript, of which the engine reads the disk keys of §3.5’s bus regex, lock and template. Everything else is not “redacted” — it is never read into the bundle’s data model at all.

Within an allowlisted *value* the same rule applies one level down. A disk value `san-a:vm-101-disk-0,size=512G,iothread=1,discard=on` is parsed into its volume id, size, format and media and re-serialized from those four; iothread and discard do not survive because nothing reads them.

The pseudonym function, and the salt

```
pseudonym(kind, value) = HMAC-SHA256(salt, kind || "\0" || value)
```

truncated and rendered per kind (below). kind is included so that a node and a storage that happen to share a name do not collide into one pseudonym — they are different objects and must stay different.

The salt is 32 bytes from os.urandom(), generated on first use and persisted at support.salt_path (default /var/lib/pve-storage-drs/anonymization-salt, mode 0600). It is never written into a bundle. The manifest instead carries `salt_fingerprint = HMAC-SHA256(salt, "drs-salt-fingerprint")`, which lets two bundles be proven to share a mapping — so the author can diff a before-and-after pair from the same cluster — without revealing the salt or letting anyone test a guess at an original name against the bundle.

This is what makes the mapping reproducible in the sense that matters: **the same host maps the same object to the same pseudonym forever**, across runs, across releases and across bundles, until the operator rotates the salt with `--new-salt`. Two consequences worth stating plainly, because the word “reproducible” invites the wrong one:

- what is stable is the *mapping*, not the payload — a bundle taken a week later has the same names for the same VMs and storages, and different metric samples, which is precisely what makes the pair comparable;
- reproducibility is **per salt file**, so it is per node by default. An operator who wants every node in the cluster to produce the same mapping points `support.salt_path` at a `pmxcfs` path (`/etc/pve/pve-storage-drs-anon-salt`); the manual documents that as the choice it is, since it also replicates the salt to every node.

Not derived from /etc/machine-id, and not from the cluster name. Both are tempting because they

need no state file. `machine-id` is disqualified by cloning: it is generated at install time and VM templates, golden images and cloned PVE installs routinely carry the same one, so two unrelated clusters could produce the same pseudonyms for the same `vmid` — turning the mapping into a cross-bundle correlation key, the exact opposite of the goal. The cluster name is disqualified by entropy: a low-entropy, guessable input to a public HMAC construction is a mapping anyone can invert by brute force over a name list.

Per-kind mapping

Kind	Pseudonym	Notes
Cluster name	<code>cluster-<8 hex></code>	
Node name	<code>node-<8 hex></code> , or <code>node-<8 hex>.<8 hex></code> . <code>invalid</code> when the original was an FQDN	Shape preserved <i>only</i> for FQDN-ness, so §3.4's <code>_escape_promql_regex_literal()</code> dot-escaping path stays exercised by a bundle from a cluster that uses FQDNs. <code>invalid</code> per RFC 2606
Storage id	<code>stor-<8 hex></code>	Lowercase letters, digits and <code>-</code> only — deliberately the narrowest shape real ids take, so a pseudonym is accepted anywhere an id is
Group name	<code>group-<8 hex></code>	<code>groups[].name</code> , <code>--group</code> and every <code>state.json</code> key that embeds it move together
<code>vmid</code>	<code>100 + (HMAC mod 899_900)</code> , deterministic linear probing on collision	Must stay an integer: it is a PromQL label value, a config value in <code>exclude.vmid</code> s and a component of a volume id. Probing walks candidates in order of <i>original</i> <code>vmid</code> so the result never depends on iteration order
Volume id	rebuilt as <code><storage-pseudonym>:<prefix>-<vmid-pseudonym>-disk-<n>[.<ext>]</code>	The structural prefix (<code>vm-</code> , <code>base-</code>), the disk index and an optional trailing <code>.<ext></code> (e.g. <code>.qcow2</code> , PVE's own format marker on the volume name for <code>qcow2-on-shared-LVM</code> and similar cases, confirmed against a real cluster) survive because §3.5's parser and §3.6's movability rules read them, and because the same string must still match its own <code>storage-content</code> listing entry at replay time. A volume whose name matches neither the pattern nor its extension-bearing form is dropped
Tag, pool	<code>tag-<8 hex></code> , <code>pool-<8 hex></code>	Pseudonymized rather than dropped because <code>exclude.tags</code> filters on them; the same mapping rewrites <code>exclude.tags</code> in <code>config.yaml</code> so the exclusion replays
UPID	rebuilt from anonymized parts	<code>UPID: {node}:{pid}:{pstart}:{starttime}:{type}:{id}</code> the grammar confirmed against a real cluster in <code>crashrecovery.py</code> . <code>Node</code> , <code>id</code> and <code>user</code> are mapped; <code>pid/pstart</code> are replaced with fixed constants (they identify a process on a named host and nothing the engine reads)
Username / realm	<code>user-<8 hex>@realm</code>	Only ever seen inside a UPID

Kind	Pseudonym	Notes
Metric name	as built (REVIEW.md X-08): not anonymized, carried verbatim	The operator's own configured names (config.metrics.read_ops etc.) reach the bundle in config.yaml, the query text and every label_values capture, unchanged. This table used to promise canonical drs_rd_operations/drs_wr_bytes/... names instead; that mapping was never built. The as-built behaviour leaks nothing (the names are already the operator's own config, already in config.yaml) and is self-consistent by construction rather than by a second mapping that could drift from it
Prometheus label name	the configured vmid/device/node label names are kept verbatim; every other label is dropped	Label <i>names</i> are chosen by the operator's Telegraf config and can be identifying (customer, datacenter), but the three configured ones must survive or the bundle cannot be joined. They are already in config.yaml, so they leak nothing the config does not
Device name (scsi0, efidisk0, tpmstate0, unused3)	not anonymized	A closed enumerated set (\$3.5's bus regex) carrying no identity, and \$3.6's movability rules and \$3.7's pinning read them directly. Mapping them would destroy the behaviour the bundle exists to reproduce
VM name, description, notes, comment, snapshot name	dropped	Free text, nothing reads it. show-load under replay prints the vmid where it would print a name

Timestamps are rebased. Every absolute timestamp — sample timestamps, ctime on a volume, snaptime, a UPID's starttime — is shifted by a single per-bundle offset so that the capture window starts at a fixed synthetic epoch, and the manifest records the *duration* and the *wall-clock-hour alignment* rather than the date. Load has a daily and a weekly shape that the seasonal forecasters of \$10.1 must be able to learn, so hour-of-day and day-of-week are preserved modulo the week; the absolute date is not, because “this cluster's I/O spiked on the afternoon of the 3rd” is a correlation handle for anyone who reads incident reports.

Unmapped means dropped. If the collector meets an identifier of a mapped kind that is not in its map — a volume id naming a storage outside every group, a UPID for a node that has left the cluster — it drops the containing record, logs a warning and counts it in the manifest. It never passes the original through and never invents a mapping on the spot. Fail closed: the failure mode of “dropped a record the author might have wanted” is a worse bundle, and the failure mode of the alternative is a leak.

The configuration in the bundle

config.yaml is the effective configuration as config.py resolved it, with four classes of change. It matters that this file is right: it is what --replay runs on, so an inconsistency between it and the recorded responses shows up as a key miss (\$16.5) rather than as a wrong answer.

- **Credentials and endpoints are dropped, not blanked.** proxmox.host, .user, .password, .token_id, .token_secret, .fingerprint, prometheus.url and every prometheus auth field are absent from the file, not present-and-empty. Replay needs none of them (\$16.5), and a blanked field invites a future reader to wonder what was in it.
- **Every identifier is mapped with the same mapping as the data:** groups[].name,

groups[].storages[].id, exclude.vmid, exclude.storages, exclude.disks and exclude.tags. A knob the engine compares against a captured value must move together with it or the exclusion silently stops applying, which is a quiet behaviour change in a file that claims to reproduce a run. state.path, support.salt_path and support.bundle_dir are replaced with fixed placeholders — a filesystem path is free text and routinely carries an organization's name.

- **/.../ storage patterns are expanded at capture time** (§11.4) and the bundle carries the literal, anonymized ids they matched. A pattern cannot survive anonymization: its text names real storages, and a pattern rewritten over pseudonyms would match nothing on re-expansion. The manifest records each pattern entry's *existence* and the number of storages it matched, so the author can see that a group was pattern-driven; the bundle loses the ability to test §11.4's expansion itself, which is a real and deliberate gap named here rather than discovered later.
- **metrics.extra_selector is rewritten**, not carried, for the same reason: it is operator-authored PromQL over real label values. The collector replaces it with the equivalent anonymized node alternation — the selector §3.4's default tier would have built — and the manifest flags that it did. A bundle from a cluster whose selector does something the default tier cannot express is therefore not byte-faithful to its live queries, and says so in the manifest instead of quietly producing a plan from differently-scoped data.

Everything else — window, snapshot_reserve, free_space, gates, migration, objective, solver, execution, forecast, report, the per-storage capability_weight/reserve_factor/ saturation_load/free_space — is carried **verbatim**. Those knobs are the test case.

What is deliberately preserved

Sizes, capacities, used/free bytes, per-disk load samples, storage type, content, shared, disable, saferemove, saferemove_throughput, disk format, the PVE and Prometheus version strings, the count and grouping of everything, and the entire configuration except its credentials. All of it is what makes a bundle a test case rather than a shape, and none of it is an identifier.

Threat model, stated honestly

The bundle contains **no direct identifier and no secret**: no hostname, no IP, no IQN, no fingerprint, no key, no password, no customer-authored free text. That is a property the scrub audit mechanically checks (§16.6).

It is **not** anonymous against someone who already knows the cluster. A cluster with three nodes, two 8 TiB LUNs and 1214 VMs is recognizable to anyone who has seen it, and the load shape of a particular workload is a fingerprint. The tool must not claim otherwise, and the manual must say so in the same paragraph that explains how to run the command: a bundle is safe to hand to the author, it is not safe to publish, and the decision to submit one is the operator's informed decision and not a formality. tests/corpus/README.md carries the same statement for the author's side of the exchange.

16.4 The command

```
pve-storage-drs -c /etc/pve/drs.yaml collect-testdata [options]
```

Option	Default	Effect
-o, --output DIR	support.bundle_dir (/var/lib/pve-storage-drs/testdata)	Where the bundle directory and its tarball are written
--estimate	off	Print the query count and payload estimate, then exit without fetching
--range DURATION	the §16.2 capture range	Override the range of the series capture
--step DURATION	metrics.step	Override the series resolution

Option	Default	Effect
--no-series	off	Capture topology, instant queries and findings only. Produces a small bundle that cannot exercise §10's forecasters
--no-archive	off	Write the directory only, no tarball
--salt-file PATH	support.salt_path	Use a different salt file
--new-salt	off	Generate a fresh salt, replacing the persisted one. Logged at warning: bundles made before and after no longer share a mapping

It is a **read-only command and a dry-run-only command**: --mode confirm or --mode auto alongside it is a usage error, not a silently ignored flag, because the one thing an operator collecting diagnostics must never risk is having started a migration. Exit 0 when every capture step succeeded, 1 when a bundle was written with recorded failures, 2 on a usage error.

Human output names the bundle path, its size, the counts (groups, storages, VMs, disks, series, samples), the salt fingerprint, and every recorded failure. --json emits the manifest.

16.5 Replay: the --replay global option

```
pve-storage-drs --replay ./drs-testdata-cluster-3f8a91c2-2026-09-11 plan --json
```

--replay PATH is a **global** option (§11.3), not a subcommand, and that is the whole design: it substitutes both clients at the one place each is constructed, so every read-only command — verify-metrics, verify-storages, show-load, plan, explain — runs unchanged against the bundle. One option, five commands under test, and no second copy of the pipeline to keep in step with the real one.

- The configuration comes from the bundle's own config.yaml unless -c is also given, in which case the file's knobs are used against the bundle's data — that is how the author runs the operator's cluster through a solver backend, a spread_metric or a forecaster the operator never selected.
- **apply refuses.** So does any --mode above dry-run. Exit 2.
- **No network is reachable.** The replay clients are constructed *instead of* the real ones and hold no session, and a bundle's config.yaml carries no credentials or endpoints, so a bug cannot fall back to a live cluster. A unit test asserts that constructing the replay path never constructs a requests.Session or a ProxmoxAPI.
- **A key miss is a loud, specific error**, never an empty result: “bundle has no recorded response for sum by (vmid, instance) (rate(drs_rd_operations[600s])) over 7 d at 300 s — captured with --step 60s?” This is the failure mode that would otherwise make a replay silently diverge from the live run, and §13 gains a row for it. It is also why the cache key is the anonymized query text: the engine under replay reads the anonymized config, generates the anonymized query, and hits or misses deterministically — a key computed over the *original* text could only ever miss.
- state.json under replay is read from the bundle if present and written nowhere. A replay never touches the host's real state.
- **vm-pending/<vmid>.json is the one deliberate exception to the “key miss is loud” rule above.** A bundle captured before section 3.8 added this call has no vm-pending/ directory at all; a missing file there is treated as “nothing pending”, not a bundle defect, so an older committed bundle keeps replaying rather than every one of them needing recapture the moment this file was added. A bundle captured after 3.8 always writes the file for every considered VM, even when its own list is empty, so this fallback can only ever trigger on a genuinely older bundle.

--replay ships with the tool rather than living in tests/, for two reasons: the operator needs it to check

their own bundle before sending it (“does plan against this bundle show the problem I am reporting?”), and a debugging aid that only exists inside a test harness is one the author cannot hand back to the operator when the answer is “your bundle replays fine, so the difference is in your live cluster”.

16.6 The corpus: tests/corpus/

Submitted bundles land in **tests/corpus/**, one directory per bundle, and become part of the test suite.

```
tests/corpus/
  README.md           # what goes here, how to add one, and the honesty statement
  <bundle-name>/      # the unpacked bundle directory, exactly as collected
  <bundle-name>.submission.yaml # provenance and consent (below)
  <bundle-name>.expected.json  # generated, never hand-edited
  validate_corpus.py   # the generator and the --check gate
```

`<name>.submission.yaml` records what the repository needs to know about data it did not produce: who submitted it and under what terms (redistribution under AGPL-3.0-or-later, as the rest of the repository), the PVE and Prometheus versions, the collector version, whether the submitter reviewed the bundle before sending, and what the bundle is *for* — the behaviour it is expected to reproduce. A bundle with no such file is not a test case, it is an unattributed dump, and the suite fails on it. `debian/copyright` gains a stanza for `tests/corpus/*` when the first bundle lands; the corpus is not installed by the package.

What the suite asserts

The central difficulty is that **a real bundle has no known-correct answer**. Nobody can enumerate 3^{1214} assignments, so unlike §14’s fixtures a corpus bundle cannot assert “the plan is optimal”. Four kinds of assertion that do hold:

1. **The scrub audit** — cheap, and it runs on every bundle in the corpus on every `make check`, before anything else. It walks every `pve/` and `prometheus/` file, every key (X-03: this used to be true of only the five top-level `pve/` files; a `prometheus/` series’ metric dict is checked against the bundle’s own configured label names, read from its `config.yaml`) and fails on: any key not in `anonymize.py`’s allowlist (the same allowlist the collector uses, so the two cannot drift); any value matching an IPv4 or IPv6 literal, an email address, an `iqn./naa./wwn.` prefix, a PEM block, a 64-hex-or-longer run, a JWT-shaped string, a public-suffix hostname (`.com/.net/.org/.local/.internal/.corp/.lan/.home/.example/.test` — X-02’s own reproduction found the original four-suffix list missed a transport failure’s own `.internal/.corp/.example` endpoint), or an unredacted transport `host='...'` literal (X-02’s own backstop for the same class); any node or storage name not matching the pseudonym grammar; any absolute timestamp outside the synthetic epoch window. `manifest.json/findings.json` have no `anonymize.py` allowlist of their own (hand-authored bundle metadata, not a captured PVE/Prometheus object shape) and get the value-pattern checks only. It is a second line of defence that assumes the collector has a bug, which is the only useful assumption to make about a privacy control.
2. **Invariants, not optima** — reconstructed from what `plan --json`’s own group report already records per variant (X-07): every move in the plan is to a storage (C2) permits for that group; no accepted move carries `exceeds_max_duration: true` (§7.3’s duration rule); a disk payback rejected or deferred never also appears as an accepted move (§7.3’s saturation guard, structurally). These are checkable without knowing the optimum, and they are what `check_invariants()` actually asserts. Three properties this bullet used to claim as checked and is not: §8.1’s per-step transient predicate needs the emitted *order*, which no `plan --json` field carries (the Σr_s half of this gap is closed as of phase 13 — the payback block’s `reserve_shortfall_bytes_before/_after` are the current and final Σr_s , and `check_invariants()` asserts the part of it that is an invariant: the plan never *raises* the shortfall. `Not = 0`: an oversized deprecated `min_free_bytes`, or a group with no feasible repair, legitimately ends above zero); “the objective the scheduler was handed equals the objective

recomputed from the final assignment” needs the six-term breakdown, which today only explain --json emits. A real and deliberate gap, named here rather than discovered later (the same shape as this section’s own pattern-expansion gap above) — either sweep explain --json too or add the missing fields to plan --json’s group report to close it.

3. **MILP versus heuristic, on real data.** Run the same bundle through CBC and the heuristic and assert the MILP backend’s after_spread (plan --json’s already-computed post-plan spread fraction) is not worse than the heuristic’s. This is the cross-check §14 can only perform on six disks, and it is the single highest-value thing a real bundle buys: a heuristic that beats the MILP means the two have drifted apart on the shared feasibility or objective functions, which AGENTS.md §5 exists to prevent and which no synthetic fixture of this size can detect. **CBC-versus-CP-SAT agreement was never checked, and is now uncheckable** (X-07; the CP-SAT backend was removed, AL-02): a first attempt comparing after_spread against solver.mip_gap as a relative tolerance produced real disagreement on a committed bundle under --full-matrix (cbc 0.0016 vs. cpsat 0.0034-0.0112 across several variants) that was legitimate under mip_gap on the *objective* the solvers actually optimize — mip_gap bounds suboptimality of the six-term objective, not of any one derived quantity taken in isolation, and a tiny baseline spread turns a small absolute gap into a large relative one. Getting this right needs the objective breakdown itself, the same gap named in check 2 above.
4. **Regression.** <name>.expected.json records, per variant, the gate verdict, the plan (as a sorted list of moves with their per-move costs), the payback arithmetic and the findings — as built: plan --json’s group report carries neither an emitted *order* nor the six-term objective breakdown (Y-04; the exact gaps checks 2 and 3 above name), so no expected file can record those two — it *does* record the payback block’s reserve_shortfall_bytes_before/_after and repair_exempt since phase 13. It is generated by validate_corpus.py and asserted current by validate_corpus.py --check, exactly as tests/fixtures/generate_expected.py is for §14 — and for the same reason: an expected file that a human may edit is an expected file that will be edited to match a bug.

The variant matrix

Per bundle, the matrix the generator sweeps: solver.backend $\in$ {cbc, heuristic} $\times$ objective.spread_metric $\in$ {l1, minmax} $\times$ forecast.model $\in$ {quantile, seasonal_naive, holt_winters} $\times$ objective.beta_move_count over a small sweep, with everything else from the bundle’s own config.yaml. A variant whose forecaster is unavailable in the running environment is **skipped and recorded as skipped**, never silently dropped: statsmodels is an optional dependency (AGENTS.md §9.1) and a corpus result that quietly means “quantile only” is a corpus result that lies.

Size, and bundles too big to commit

A committed bundle is reviewable data, so there is a ceiling. Rule: a bundle is committed only if its directory is under **8 MiB**, which at the §16.2 estimate is a cluster of a few dozen disks or a --no-series capture of a large one. Anything bigger is kept outside the repository and found via DRS_CORPUS_DIR, a colon-separated list of directories the suite also walks; CI sets it where a large corpus is available and the suite runs the committed bundles alone where it is not. An empty tests/corpus/ is a clean pass, not a failure — the suite must be green on a fresh clone with no bundles at all.

make corpus runs the full matrix over every discoverable bundle. make check runs the scrub audit over every bundle plus the invariant and regression assertions for the committed ones, because those are bounded by the 8 MiB rule; the full matrix over a large external corpus is a CI job of its own.

16.7 Modules, phases and the honest status

Three new modules, split along the line that matters — the part that must be right is pure:

Module	Responsibility
<code>anonymize.py</code>	The allowlists, the pseudonym function, the per-kind mappings, the timestamp rebase. Pure, no I/O, and the shared implementation behind both the collector and the scrub audit
<code>collect.py</code>	Capture orchestration, the estimate, chunking, the manifest, the deterministic writer
<code>replay.py</code>	The bundle reader and the two replay clients, satisfying the same interfaces <code>metrics.py</code> and <code>pve.py</code> define

`config.py` gains a support block: `support.salt_path`, `support.bundle_dir`, `support.max_series_points`, `support.capture_range` (auto for §16.2's computed maximum, or an explicit duration). All four appear in §15.1, in `config/drs.example.yaml` and in the manual, in the commit that implements them and not before.

§12 gains **phase 10**: `collect-testdata + --replay + tests/corpus/`, done when a bundle collected from the author's own cluster replays to the same plan the live run produced, the scrub audit passes on it, and the determinism test is green.

`anonymize.py`, `collect.py`, `replay.py`, `cli.py`'s wiring (`collect-testdata`, the global `--replay`) and `tests/corpus/validate_corpus.py` are implemented, with unit and CLI-level tests covering capture, every anonymization rule, the deterministic writer, and a full capture → write → replay round trip. `tests/corpus/` and its `README.md` existed from the commit that added this section, ahead of the code, so the place to put a bundle was already there and documented; phase 10's own "done when" — a bundle collected from the author's own cluster replays to the same plan the live run produced, the scrub audit passes on it, and the determinism test is green — is met: `bzed-dev-cluster-24h`, a `--range 24h` capture from the project's own dev cluster, is in `tests/corpus/`, replays all five read-only commands to the same result the live run produced, and passes the scrub audit with zero violations.